

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ

«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет автоматики и вычислительной техники

Кафедра электронных вычислительных машин

О. В. КАРАВАЕВА

Г. А. ЧИСТЯКОВ

ПЛАНИРОВАНИЕ И СИНХРОНИЗАЦИЯ ПРОЦЕССОВ В МНОГОПРОЦЕССОРНЫХ ОПЕРАЦИОННЫХ СИСТЕМАХ

Учебно-методическое пособие

Киров

2014

УДК 004.451(07)
К 21

Рекомендовано к изданию методическим советом факультета автоматизации и вычислительной техники ФГБОУ ВПО «ВятГУ» в качестве учебно-методического пособия для студентов для студентов направления 09.03.01 «Информатика и вычислительная техника», всех профилей, подготовки всех форм обучения

Рецензент

доцент кафедры АТ ФГБОУ ВПО «ВятГУ»,
кандидат технических наук,
В. Вал. Куклин

Караваева, О. В.

К 21 Планирование и синхронизация процессов в многопроцессорных операционных системах: учебно-методическое пособие / О. В. Караваева, Г. А. Чистяков. – Киров: ФГБОУ ВПО «ВятГУ», 2014. – 127 с.

УДК 004.451(07)

Пособие предназначено для студентов направления 09.03.01 «Информатика и вычислительная техника» всех профилей подготовки, всех форм обучения для выполнения лабораторных работ по дисциплине «Операционные системы».

Тех. редактор А. В. Куликова

© ФГБОУ ВПО «ВятГУ», 2014

Содержания

Введение.....	5
1. Планирование в многопроцессорных системах.....	11
1.1 Общие сведения об интерфейсе лабораторной установки	13
1.2 Задачно-независимые алгоритмы планирования.....	16
1.2.1 Алгоритм планирования FCFS	16
1.2.2 Круговой алгоритм планирования	29
1.2.3 Алгоритм планирования SPF	30
1.3 Задачно-ориентированные алгоритмы планирования	41
1.3.1 Алгоритм планирования SNPF	41
1.3.2 Круговой алгоритм планирования	53
1.3.3 Алгоритмы копланирования	54
1.3.4 Алгоритм динамического разделения	68
2. Синхронизация в распределённых системах.....	83
2.1. Алгоритмы синхронизации часов	85
2.1.1 Алгоритм Кристиана	86
2.1.2 Алгоритм Беркли	90
2.1.3 Логические часы	91
2.2. Алгоритмы голосования.....	95
2.2.1 Алгоритм «забияки»	96
2.2.2 Кольцевой алгоритм	101
2.3 Взаимные исключения.....	104
2.3.1 Централизованный алгоритм.....	104
2.3.2 Распределенный алгоритм	108

2.3.3 Алгоритм маркерного кольца	113
2.4 Распределенные транзакции	116
2.4.1 Закрытое рабочее пространство	118
2.4.2 Журнал с упреждающей записью	120
Заключение	126
Библиографический список.....	127
Электронные ресурсы	127

Введение

Вычислительная техника в своем развитии по пути повышения быстродействия ЭВМ приблизилась к физическим пределам. Время переключения электронных схем достигло долей наносекунды, а скорость распространения сигналов в линиях, связывающих элементы и узлы машины, ограничена значением 30 см/нс (скоростью света). Поэтому дальнейшее уменьшение времени переключения электронных схем не позволит существенно повысить производительность ЭВМ. В этих условиях требования практики (сложные физико-технические расчеты, многомерные экономико-математические модели и другие задачи) по дальнейшему повышению быстродействия ЭВМ могут быть удовлетворены только путем распространения принципа параллелизма на сами устройства обработки информации и создания многомашинных и многопроцессорных (мультипроцессорных) вычислительных систем. Такие системы позволяют производить распараллеливание во времени выполнения программы или параллельное выполнение нескольких программ.

В настоящее время исключительно важное значение приобрела проблема обеспечения высокой надежности и готовности вычислительных систем, работающих в составе различных АСУ и АСУ ТП, особенно при работе в режиме реального времени. Эта проблема решается на основе использования принципа избыточности, который ориентирует также на построение многомашинных или многопроцессорных систем (комплексов). Появление дешевых и небольших по размерам микропроцессоров и микро-ЭВМ облегчило построение и расширило область применения многопроцессорных и многомашинных ВС разного назначения.

Различие понятий многомашинной и многопроцессорной ВС поясняет рисунок 1.1. Многомашинная ВС (ММС) содержит несколько ЭВМ, каждая из которых имеет свою ОП, средства обмена информацией между машинами и работает под управлением своей операционной системы. Реализация обмена информацией происходит, в конечном счете, путем взаимодействия операционных систем машин между собой. Это ухудшает динамические характеристики процессов межмашинного обмена данными. Применение многомашинных систем позволяет повысить надежность вычислительных комплексов. При отказе в одной машине обработку данных может продолжать другая машина комплекса. Однако можно заметить, что при этом оборудование комплекса недостаточно эффективно используется для этой цели. Достаточно в системе, изображенной на рис. 1.1 а, в каждой ЭВМ выйти из строя по одному устройству (даже разных типов), как вся система становится неработоспособной.

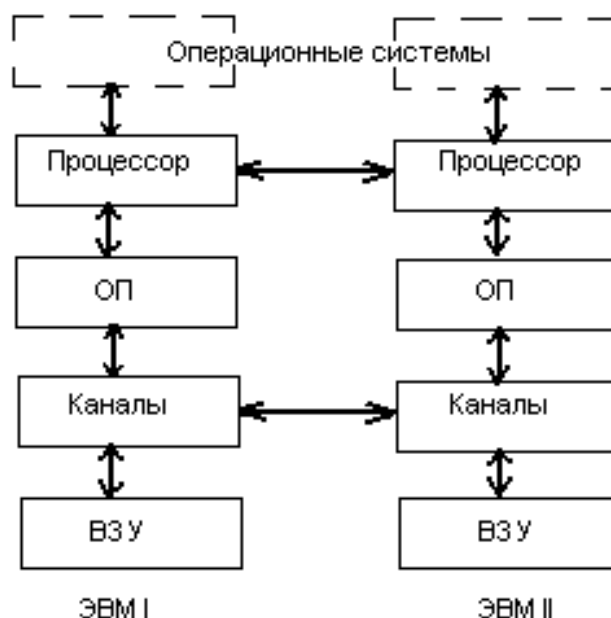


Рис. 1.1. Многомашинные системы

Этих недостатков лишены многопроцессорные системы (МПС). В таких системах (рис. 1.2.) процессоры обретают статус рядовых агрегатов вычислительной системы, которые подобно другим агрегатам, таким, как модули памяти, каналы, периферийные устройства, включаются в состав системы в нужном количестве.

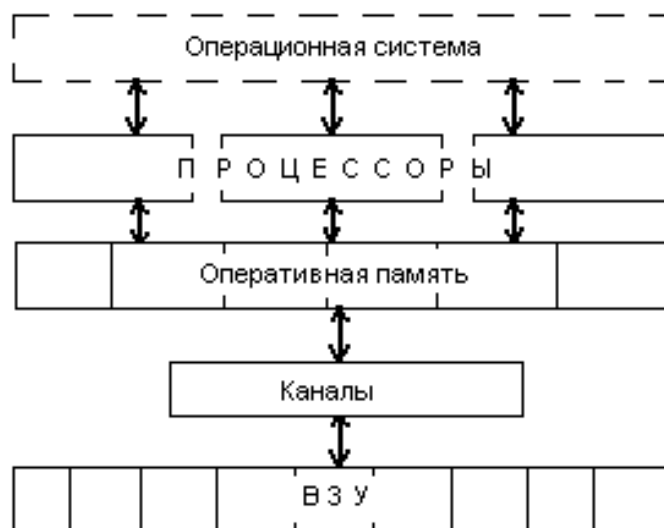


Рис. 1.2. Многопроцессорные системы

Вычислительная система называется *многопроцессорной*, если она содержит несколько процессоров, работающих с общей ОП (общее поле оперативной памяти) и управляется одной общей операционной системой. Часто в МПС организуется общее поле внешней памяти.

Под общим полем понимается равнодоступность устройств. Так, общее поле памяти означает, что все модули ОП доступны всем процессорам и каналам ввода-вывода (или всем периферийным устройствам в случае наличия общего интерфейса); общее поле ВЗУ означает, что образующие его устройства доступны любому процессору и каналу.

В МПС по сравнению с ММС достигается более быстрый обмен информацией между процессорами. Благодаря этому может быть получена

более высокая производительность, более быстрая реакция на ситуации, возникающие внутри системы и в ее внешней среде, и более высокие надежность и живучесть, так как система сохраняет работоспособность, пока работоспособны хотя бы по одному модулю каждого типа устройств.

Многопроцессорные системы представляют собой основной путь построения ВС сверхвысокой производительности. При создании таких ВС возникает много сложных проблем, таких как:

- распараллеливание вычислительного процесса (программ) для эффективной загрузки процессоров системы;
- преодоление конфликтов при попытках нескольких процессоров использовать один и тот же ресурс системы (например, некоторый модуль памяти) и уменьшение влияния конфликтов на производительность системы;
- осуществление быстродействующих экономичных по аппаратурным затратам межмодульных связей.

Указанные вопросы необходимо учитывать при выборе структуры МПС.

На основе многопроцессорности и модульного принципа построения других устройств системы возможно создание отказоустойчивых систем, или, другими словами, систем повышенной живучести.

Однако построение многомашинных систем из серийно выпускаемых ЭВМ с их стандартными операционными системами значительно проще, чем построение МПС, требующих преодоления определенных трудностей, возникающих при реализации общего поля памяти, и, главное, трудоемкой разработки специальной операционной системы.

Многомашинные и многопроцессорные системы могут быть однородными и неоднородными. Однородные системы содержат однотипные ЭВМ или процессоры. Неоднородные ММС состоят из ЭВМ

различного типа, а в неоднородных МПС используются различные специализированные процессоры, например процессоры для операций с плавающей запятой, для обработки десятичных чисел, процессор, реализующий функции операционной системы, процессор для матричных задач и др.

Многопроцессорные системы и ММС могут иметь одноуровневую или иерархическую (многоуровневую) структуру. Обычно менее мощная машина (машина-сателлит) берет на себя ввод информации с различных терминалов и ее предварительную обработку, разгружая от этих сравнительно простых процедур основную, более мощную ЭВМ, чем достигается увеличение общей производительности (пропускной способности) комплекса. В качестве машин-сателлитов используют малые или микро-ЭВМ.

Важной структурной особенностью рассматриваемых ВС является способ организации связей между устройствами (модулями) системы. Он непосредственно влияет на быстроту обмена информацией между модулями, а следовательно, на производительность системы, быстроту ее реакции на поступающие запросы, приспособленность к изменениям конфигурации и, наконец, размеры аппаратных затрат на осуществление межмодульных связей. В частности, от организации межмодульных связей зависят частота возникновения конфликтов при обращении процессоров к одним и тем же ресурсам (в первую очередь модулям памяти) и потери производительности из-за конфликтов.

Используются следующие способы организации межмодульных (межустройственных) связей:

- регулярные связи между модулями;
- многоуровневые связи, соответствующие иерархии интерфейсов ЭВМ;
- многовходовые модули (в частности, модули памяти);

- коммутатор межмодульных связей (Рис. 1.3);
- общая шина (Рис. 1.4).

Принципы организации МПС и ММС существенно отличаются в зависимости от их назначения. Поэтому целесообразно различать:

- ВС, ориентированные в первую очередь на достижение сверхвысокой производительности;
- ВС, ориентированные в первую очередь на повышение надежности и живучести.

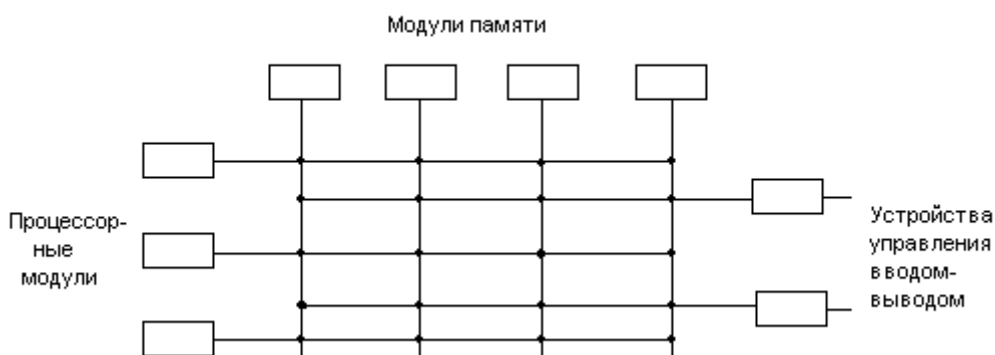


Рис. 1.3. Коммутатор межмодульных связей «Эльбрус»

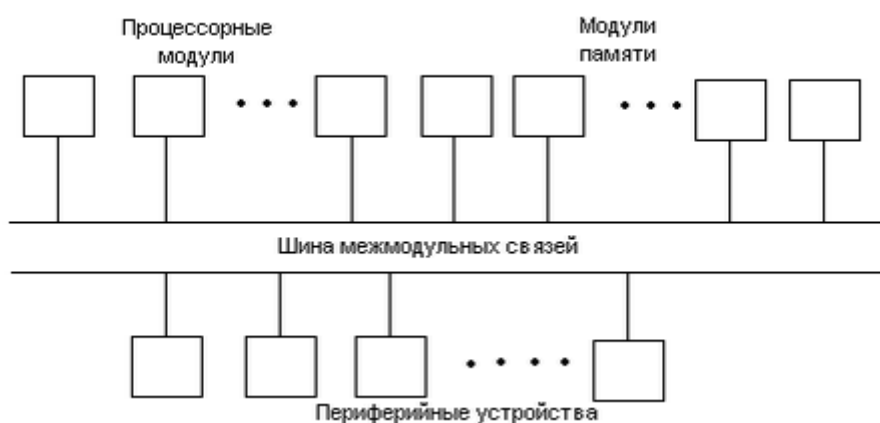


Рис. 1.4. Многопроцессорные ВС с общей шиной межмодульных связей
«СМ ЭВМ»

1. Планирование в многопроцессорных системах

Планирование в операционных системах – это предоставление доступа к процессору процессов или потоков в зависимости от алгоритмов планирования.

Часть операционной системы, отвечающая за планирование называется планировщиком.

В различных операционных системах используются разные алгоритмы планирования. Это зависит от назначения системы, ее возможностей и других факторов, в том числе и от видения разработчиков.

Планирование бывает необходимо в разные моменты времени в зависимости от ситуации, складывающейся в системе.

Например, планировщик начинает свою работу при создании или завершении процесса, при блокировании процесса на операции ввода-вывода или на семафоре, при возникновении прерывания от таймера или от внешнего устройства и т. д.

Основной задачей планировщика в однопроцессорных многозадачных системах является обеспечение выполнения всех без исключения процессов, находящихся в системе.

У планирования в многопроцессорных системах те же цели, что и в однопроцессорных. Оно должно обеспечивать максимальную производительность и минимальное время реагирования для всех процессов. Кроме того, система должна гарантировать соблюдение приоритетов процессов. В отличие от алгоритмов планирования, используемых в однопроцессорных системах, алгоритмы для многопроцессорных систем должны определять не только порядок выполнения процессов, но и то, на каких процессорах эти процессы должны выполняться. За счет этого растет сложность алгоритмов. Алгоритмы планирования для многопроцессорных систем должны

гарантировать, что процессоры в системе не будут простаивать, если есть процессы, ожидающие выполнения.

Когда нужно определить, на каком процессоре должен выполняться процесс, планировщик должен принять во внимание несколько моментов: например, в некоторых стратегиях планирования целью является максимальное распараллеливание работы, полное использование возможностей системы по одновременному выполнению программ. Взаимодействующие процессы в системах часто объединяются в задачи. Параллельное выполнение процессов задачи позволяет повысить производительность, поскольку процессы выполняются действительно одновременно. Существует несколько алгоритмов планирования с временным разделением, пытающихся воспользоваться такой параллельностью при планировании размещения взаимодействующих процессов на разные процессоры. Это позволяет процессам более эффективно организовать свою одновременную работу. Другие стратегии сосредотачиваются на привязке задач к процессорам, т. е. взаимосвязи процесса и процессора, его локальной памяти и кэша.

Процесс, привязанный к процессору, выполняется на этом процессоре в течении большей части своего жизненного цикла. Алгоритмы планирования, пытающиеся разместить процесс на один процессор на все время выполнения этого процесса, обеспечивают мягкую привязку, а алгоритмы, размещающие процесс всегда на одном и том же процессоре, обеспечивают жесткую привязку.

Алгоритмы планирования с пространственным разделением пытаются добиться максимальной привязки задач к процессорам, направляя взаимодействующие процессы для выполнения на один процессор. Эти алгоритмы подразумевают, что взаимодействующие процессы будут работать с одними и теми же данными, и эти данные скорее всего удастся разместить в кэше процессоров и в их локальной

памяти. Поэтому алгоритмы планирования с пространственным разделением увеличивают частоту успешных обращений к кэшу и локальной памяти. Однако они могут ограничивать производительность, поскольку при их использовании взаимодействующие процессы обычно не могут выполняться одновременно.

Алгоритмы планирования в многопроцессорных системах обычно делятся на задачно-ориентированные и задачно-независимые.

Во многих алгоритмах планирования для многопроцессорных систем процессы организуются в глобальную очередь выполнения. Каждая такая очередь содержит все процессы в системе, которые готовы к выполнению. Глобальные очереди выполнения можно использовать для сортировки процессов по их приоритетам, по задачам, к которым они относятся, или по тому, какой процесс выполнялся последним.

В системах могут также использоваться процессорные очереди выполнения. Использование таких очередей характерно для слабо связанных систем, в которых нужно добиться как можно большего уровня успешных обращений к кэшу и локальной памяти. В процессорных очередях выполнения процессы ассоциируются с определенным процессором, и система реализует заданный алгоритм планирования для каждого процессора. В некоторых системах используются узловые очереди выполнения. В таких системах в каждом узле может находиться по несколько процессоров. Эти очереди эффективны в системах, в которых процессы связаны с определенными группами процессоров. С проблемой планирования связана и проблема миграции процессов, т. е. переноса процессов из одной процессорной или узловой очереди в другую.

1.1. Общие сведения об интерфейсе лабораторной установки

Лабораторная работа «Планирование в многопроцессорных системах» преследует своей целью изучение алгоритмов работы

планировщиков заданий, используемых в различных операционных системах.

Для изучения данных алгоритмов используется программная модель, которая является разработкой кафедры ЭВМ и зарегистрирована

В самом начале работы необходимо зарегистрироваться, то есть ввести свою фамилию и выбрать из списка группу (рис. 1.5).

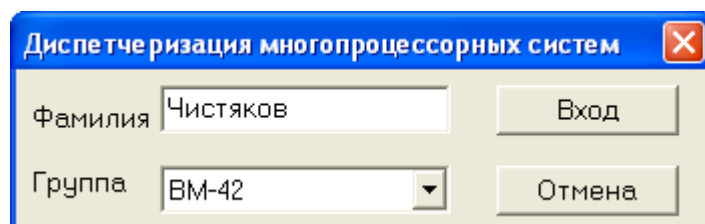


Рис. 1.5. Регистрация в программе

Главное меню программы состоит из трёх пунктов: «Файл», «Задания» и «Справка» (рис. 1.6).

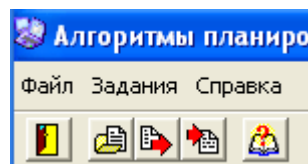


Рис. 1.6. Главное меню

Для того, чтобы приступить к выполнению работы необходимо пройти тест.

Для успешного прохождения теста необходимо ответить на ряд вопросов по теме лабораторной работы. Каждый вопрос может иметь более одного правильного ответа. Успешно выполненным считается тест с оценкой более 90 % верных ответов.

После прохождения теста можно приступать к выполнению заданий лабораторной работы. Задания генерируются автоматически и выбираются из пункта меню «Задания» главного меню. Необходимо промоделировать

работу четырёх алгоритмов планирования в многопроцессорных системах (рис. 1.7).

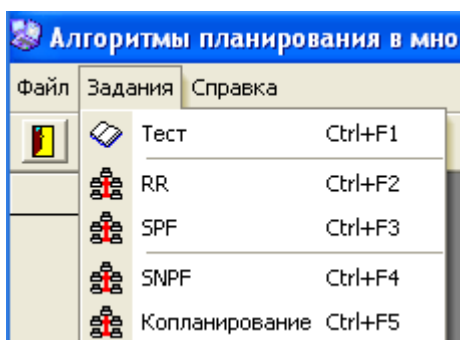


Рис. 1.7. Вкладка «Задания»

По мере решения заданий автоматически генерируется отчёт о ходе выполнения лабораторной работы. Установка не предусматривает возможности сохранения прогресса работы, однако можно сохранить отчёт (рис. 1.8).

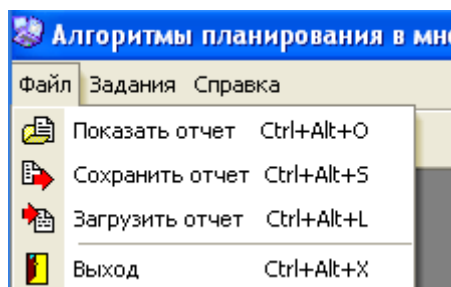


Рис. 1.8. Вкладка «Файл»

Все пункты меню продублированы кнопками быстрого доступа.

Выполняя работу, следует помнить:

- во всех дисциплинах процессы и задачи следует добавлять в том порядке, в котором они приходят в сообщениях;
- если в таблице слева у процессов есть маленький прямоугольник, то процессы можно менять местами и они должны быть упорядочены в соответствии с приоритетом.

1.2. Задачно-независимые алгоритмы планирования

Задачно-независимые алгоритмы требуют для своего использования минимальных накладных расходов, поскольку они не пытаются добиться максимального распараллеливания или воспользоваться привязкой задачи к процессору. Эти алгоритмы размещают задачи и процессы для выполнения на любом доступном процессоре. В общем случае, алгоритм планирования для однопроцессорных систем можно использовать как задачно-независимый алгоритм планирования для многопроцессорных систем.

К основным задачно-независимым алгоритмам относятся FCFS, круговой алгоритм и SPF.

1.2.1. Алгоритм планирования FCFS

FCFS расшифровывается как first-come-first-served («первым пришел – первым обслужен»). В этом алгоритме используется одна глобальная очередь. Если появляется свободный процессор, планировщик размещает на этом процессоре первый процесс из очереди выполнения. Процесс выполняется до тех пор, пока он не завершится или не произойдет ошибка.

При использовании данного алгоритма все процессы выполняются в том порядке, в котором они поступают в систему, поскольку ни один из процессов не имеет преимущества перед остальными. Поэтому при использовании алгоритма FCFS исключено бесконечное ожидание. Алгоритм использует одну глобальную очередь процессов. Однако такое отношение можно считать и неоптимальным, поскольку долго выполняющийся процесс заставляет ждать все остальные процессы, включая и быстро выполняющиеся. Кроме того, может возникнуть ситуация, при которой процессу с высоким приоритетом придется ожидать окончания выполнения процесса с низким приоритетом. Обычно алгоритм

FCFS непригоден для планирования интерактивных процессов, поскольку он не может обеспечить малого времени реагирования. Однако этот алгоритм прост в реализации, и в нем нет опасности бесконечного откладывания выполнения процесса – как только процесс встал в очередь на выполнение, никакой процесс не сможет попасть в очередь впереди него.

Описание лабораторной установки

Интерфейс окна для алгоритма FCFS представлен на рис. 1.9.

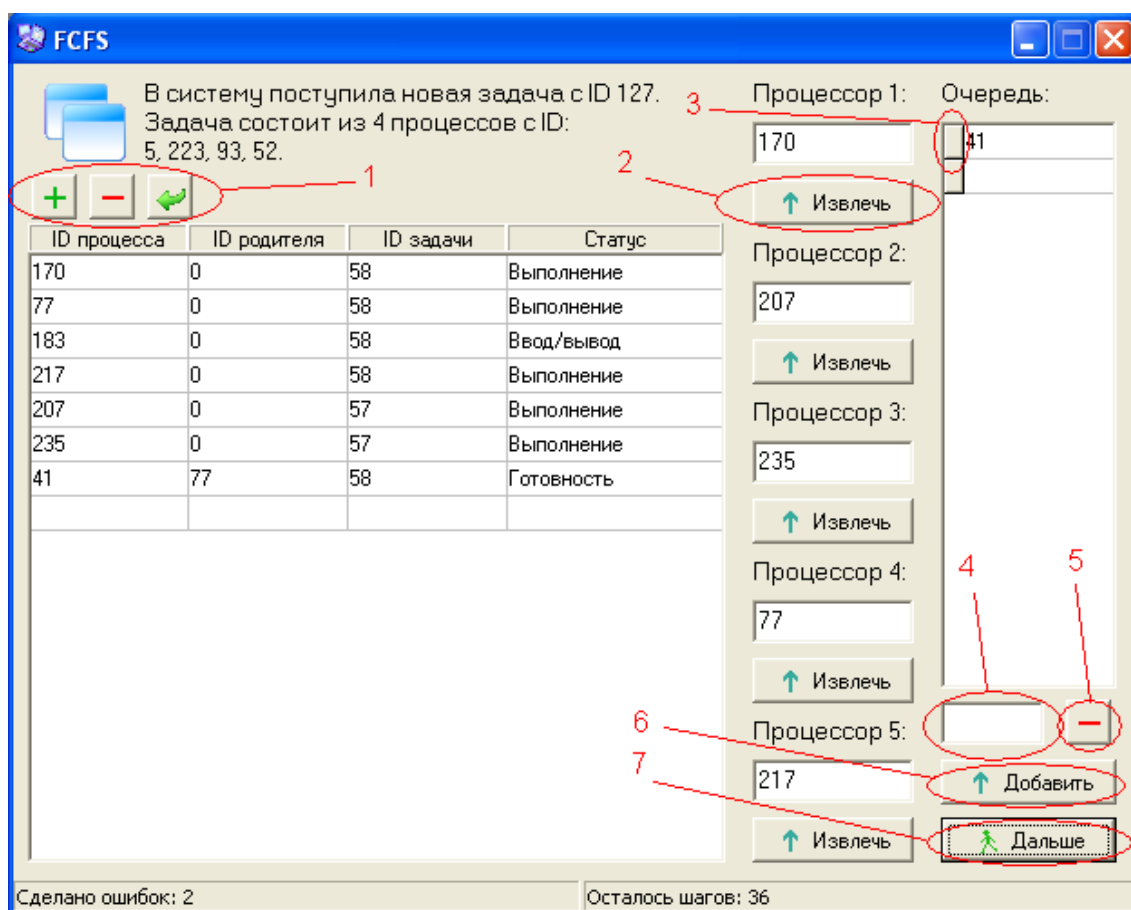


Рис. 1.9. Интерфейс окна для алгоритма FCFS

В левой верхней части окна отображается событие, произошедшее в системе.

Элементы группы (1) предназначены для добавления новой строки в таблицу процессов, удаления выделенной строки в таблице процессов, а также сброса всех значений в начальное состояние (на момент последнего успешного шага).

Кнопки «Извлечь» (2) позволяют переместить первый в очереди процесс на выполнение.

Боковые кнопки очереди (3) предназначены для переупорядочивания процессов в очереди (необходимо зажать левую кнопку мыши и перемещать).

Для добавления в очередь нового процесса необходимо вписать его номер в поле (4), а затем нажать кнопку (6). Процесс будет добавлен в конец очереди.

Удалить выделенный в очереди процесс можно с помощью кнопки (5).

Кнопка (7) позволяет перейти к следующему шагу.

В таблице процессов поле «ID процесса» должно содержать номер процесса, «ID родителя» – номер родительского процесса (или 0, если процесс не имеет родителя), «ID задачи» – идентификатор задачи. Поле «Статус» представляет собой выпадающий список с тремя значениями: «Готовность», «Выполнение», «Ввод/вывод».

Выполнение лабораторной работы

В соответствии с алгоритмом FCFS, при поступлении новой задачи, среди пары процессов больший приоритет имеет тот, который указан раньше в событии.

Выполнение шагов:

1) Добавление процессов.

С помощью кнопки «Добавить» (+) процессы заносятся в таблицу (рис. 1.10).

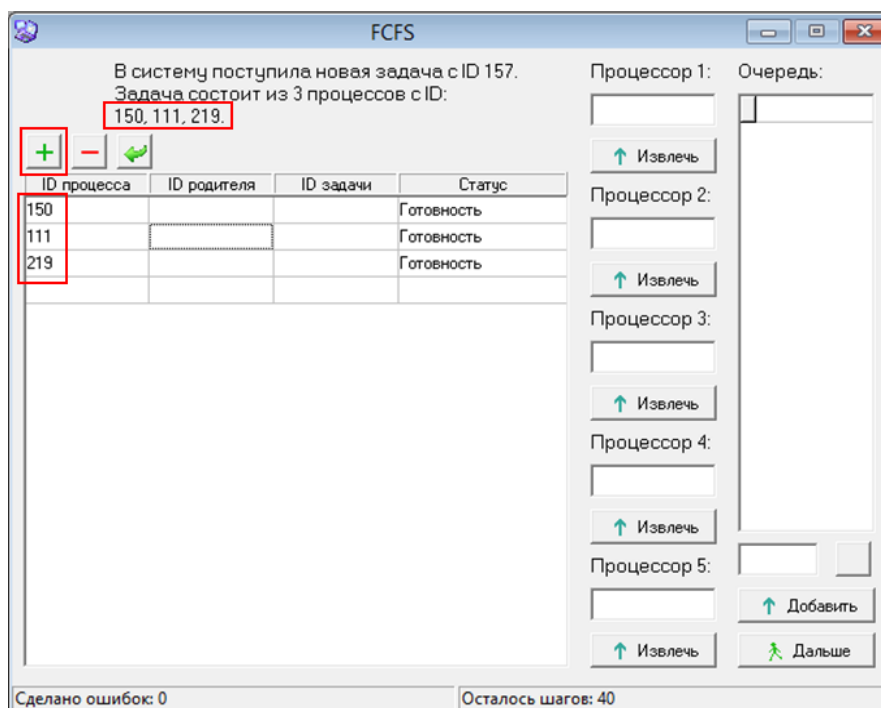


Рис. 1.10. Добавление процессов в таблицу

Так как у процессов не указан родитель, в поле «ID родителя» записывается 0 (рис. 1.11).

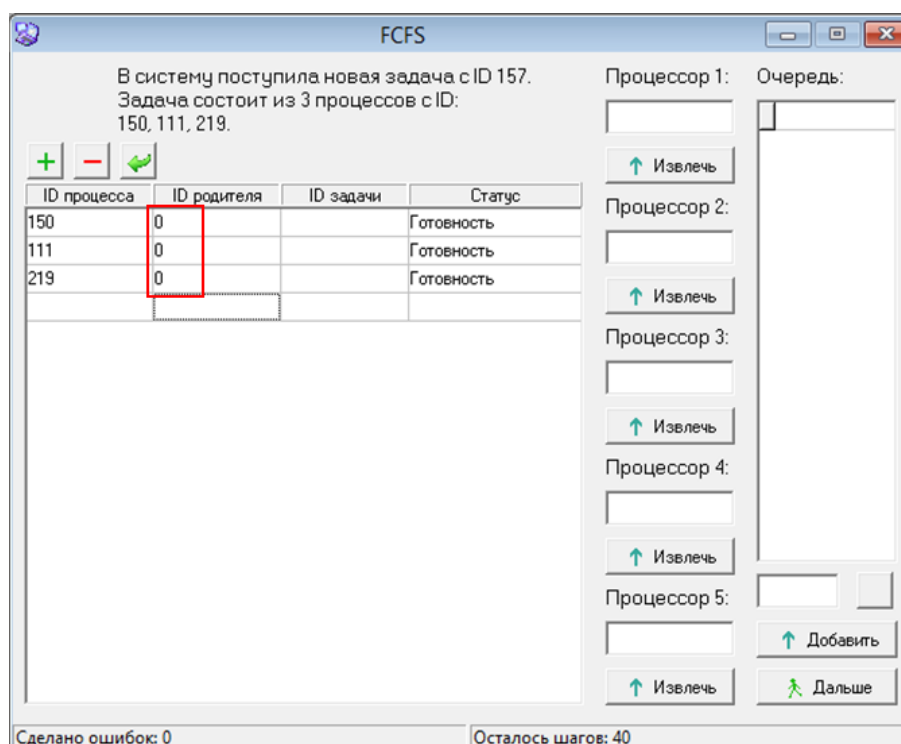


Рис. 1.11. Заполнение поля «ID родителя»

В поле «ID задачи» указывается номер задачи, к которой относится процесс (рис. 1.12).

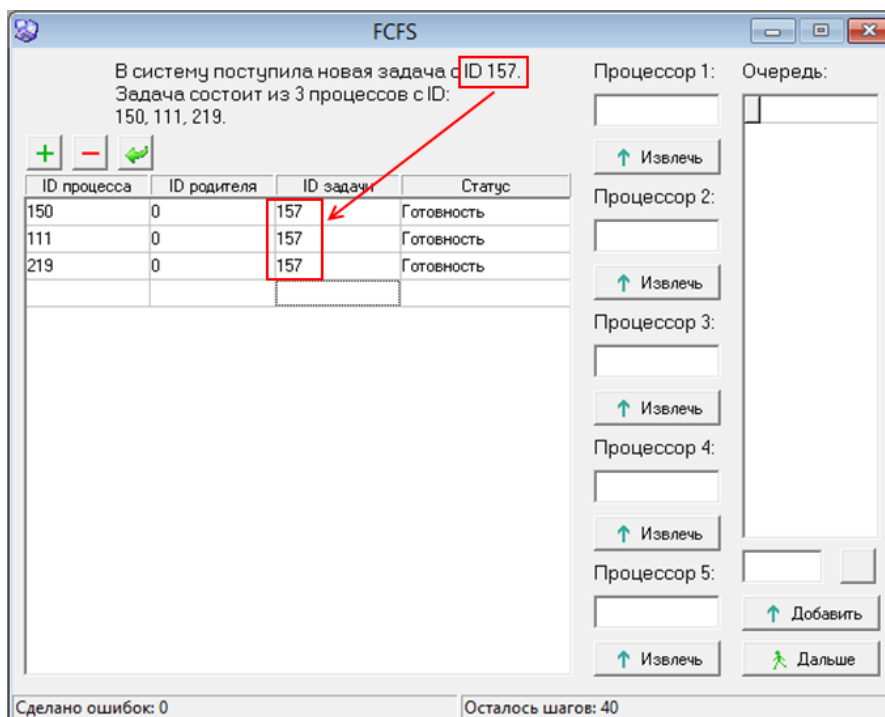


Рис. 1.12. Заполнение поля «ID родителя»

С помощью кнопки «Добавить» процессы по порядку заносятся в очередь на выполнение (рис. 1.13).

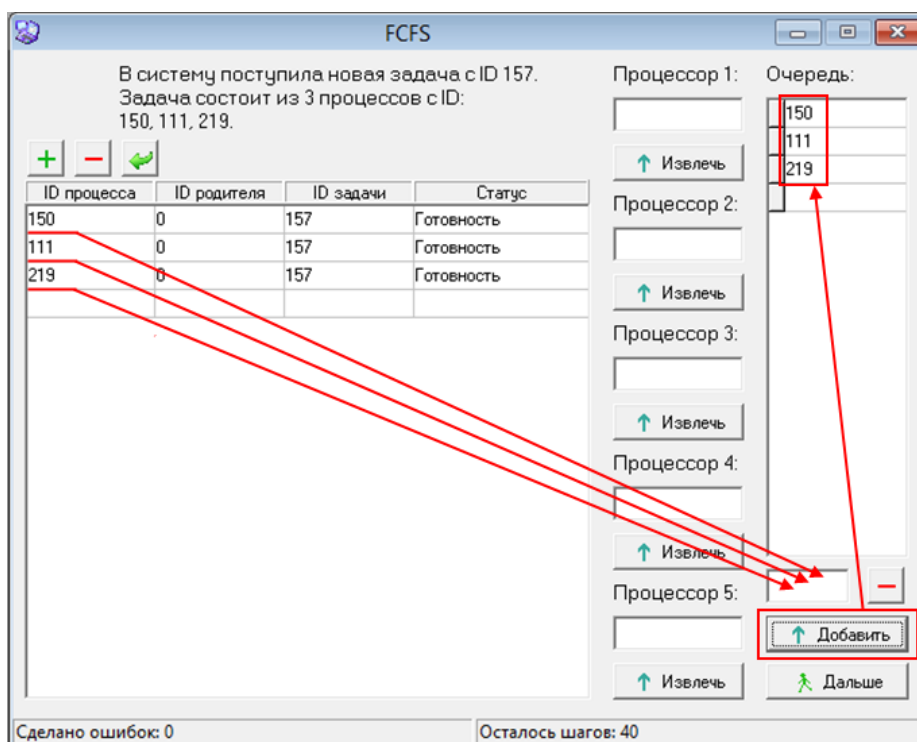


Рис. 1.13. Занесение процессов в очередь

С помощью кнопок «Извлечь» процессы уходят на выполнение (рис. 1.14, 1.15).

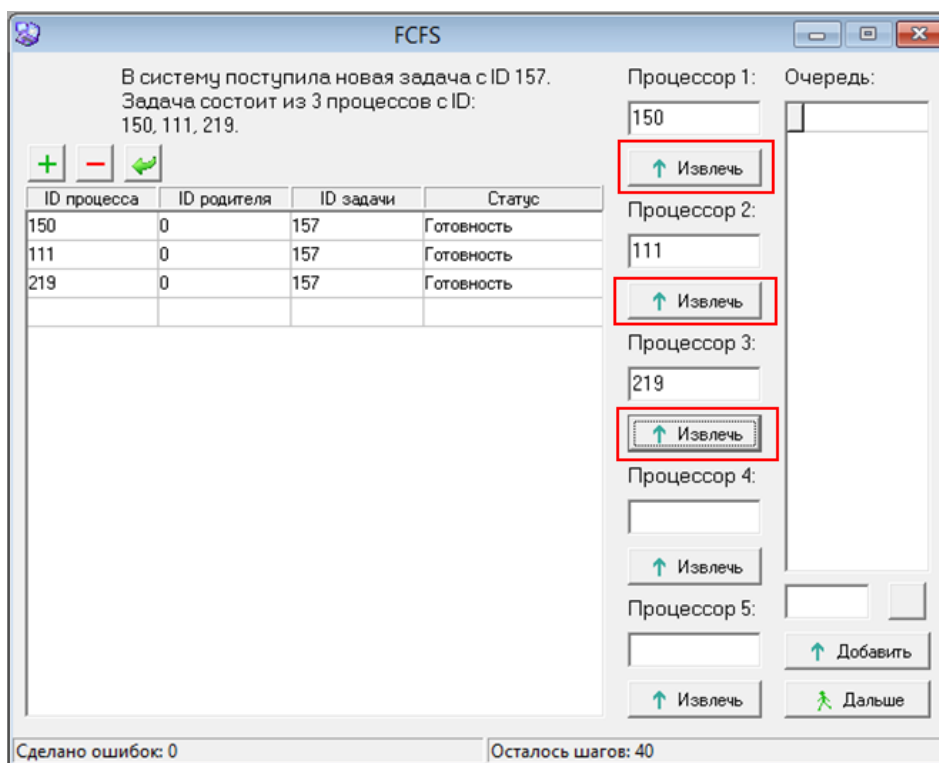


Рис. 1.14. Использование кнопок «Извлечь»

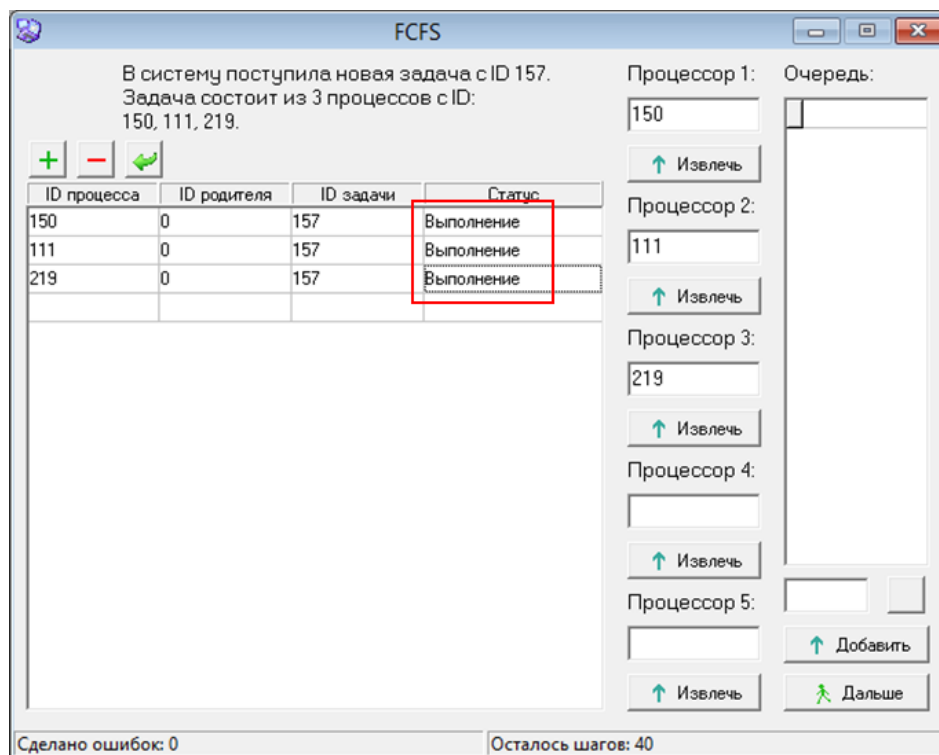


Рис. 1.15. Все процессы добавлены

2) Появление новой задачи

Выполняется аналогично добавлению процессов, в поле «ID задачи» прописывается номер новой задачи, процессы помещаются в очередь и, если свободны процессоры, уходят на выполнение (рис. 1.16).

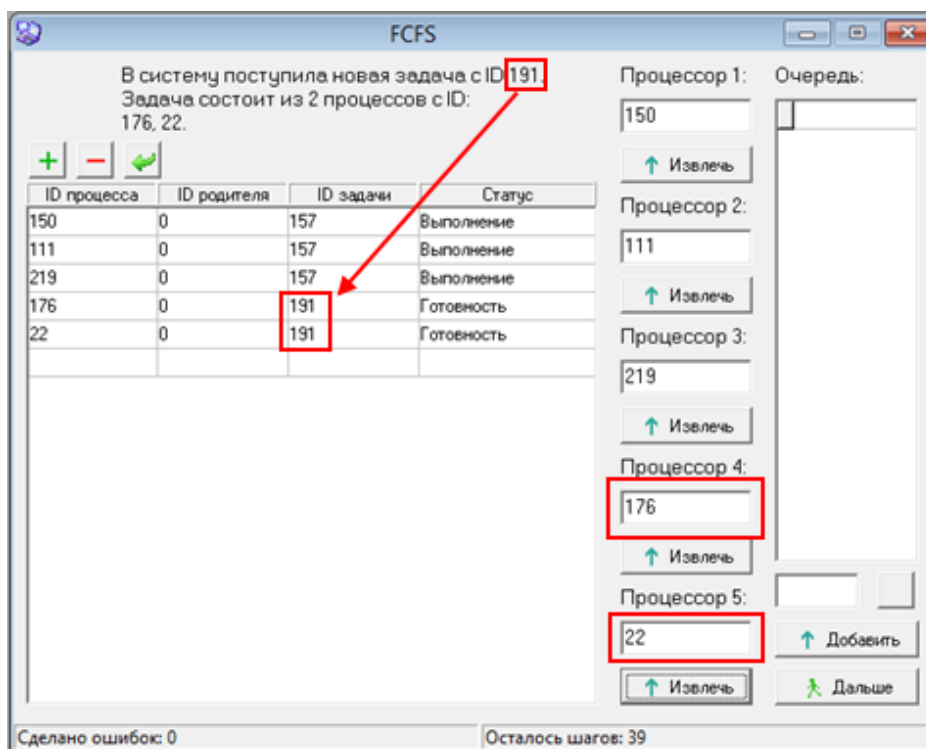


Рис. 1.16. Добавление новой задачи

Если при добавлении новых процессов все процессоры заняты другими задачами, то процессы нужно поместить в очередь на выполнение и присвоить им статус «Готовность» (рис. 1.17).

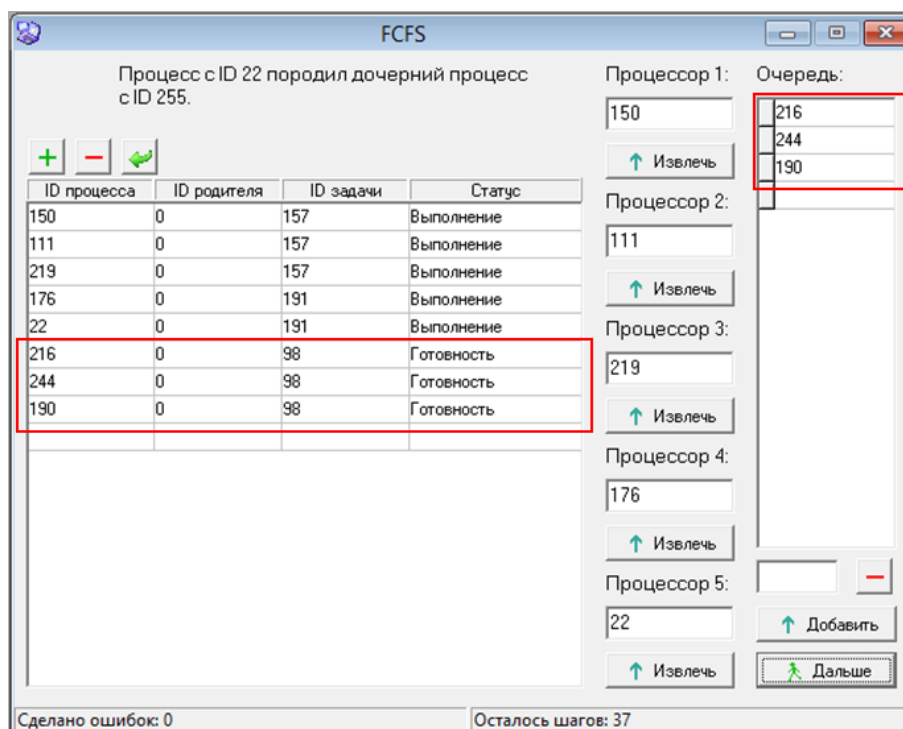


Рис. 1.17. Добавление новых процессов в очередь

3) Порождение дочернего процесса

При добавлении дочернего процесса в поле «ID родителя» записывается номер родительского процесса (рис. 1.18).

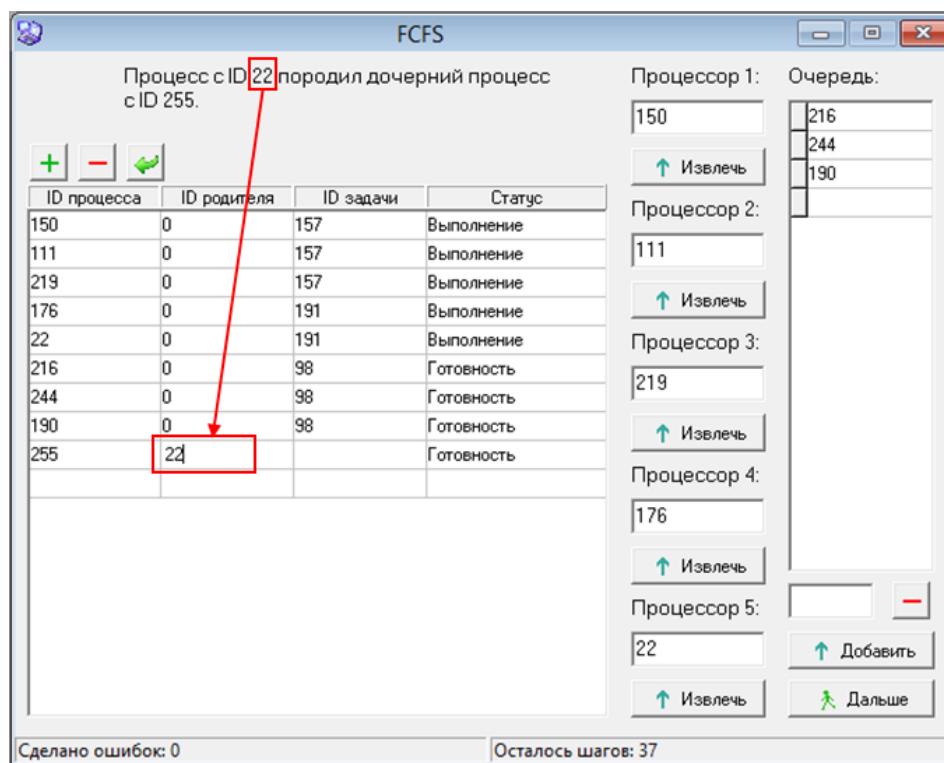


Рис. 1.18. Заполнение поля «ID родителя»

В поле «ID задачи» записывается номер задачи, к которой относится родительский процесс. Затем, если нет свободных процессоров, процесс заносится в очередь на выполнение (рис. 1.19).

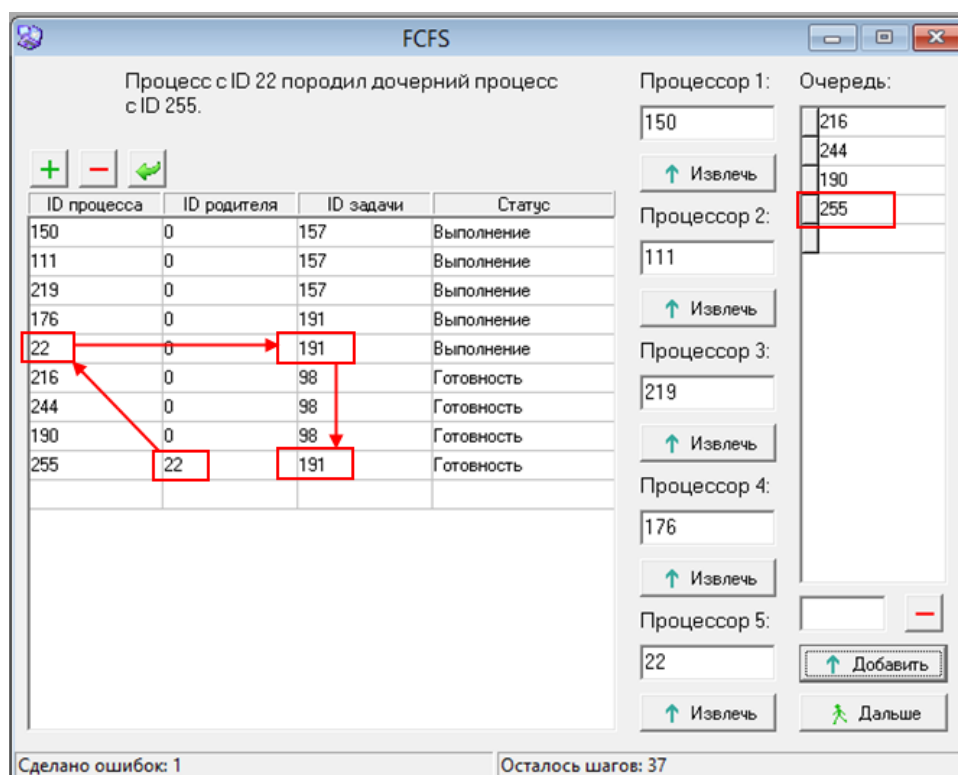


Рис. 1.19. Заполнение поля «ID задачи» и занесение процесса в очередь на выполнение

4) Завершение работы процесса

Чтобы удалить процесс, необходимо удалить его из таблицы процессов и освободить процессор, который был занят этим процессом (рис. 1.20).

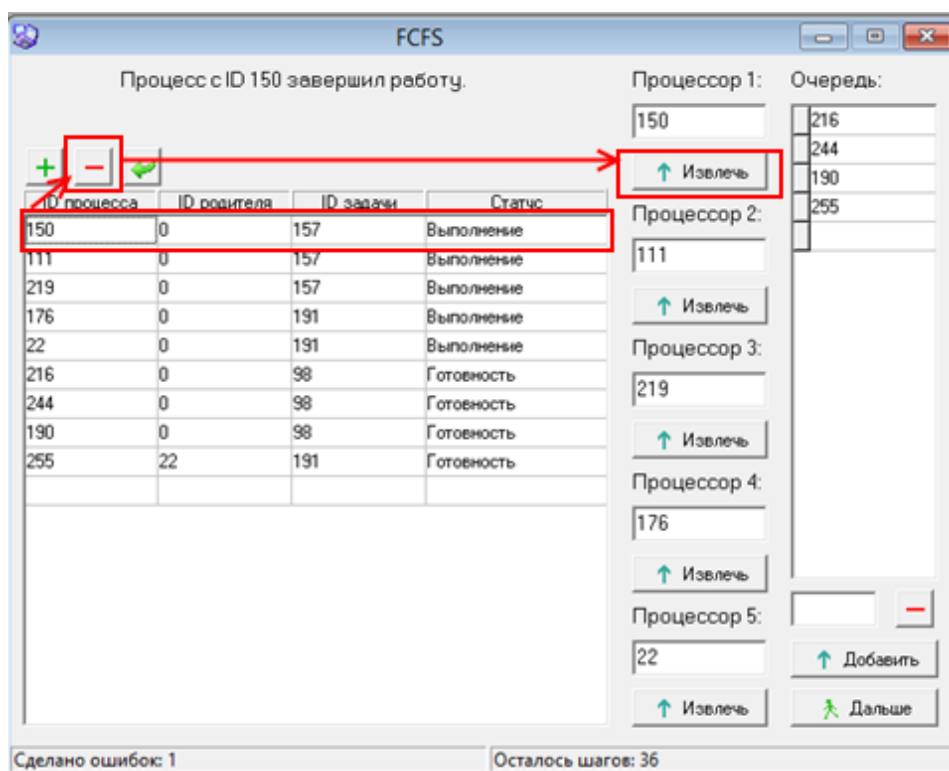


Рис. 1.20. Удаление процесса

Затем необходимо занять освободившийся процессор процессом, находящимся первым в очереди на выполнение (рис. 1.21).

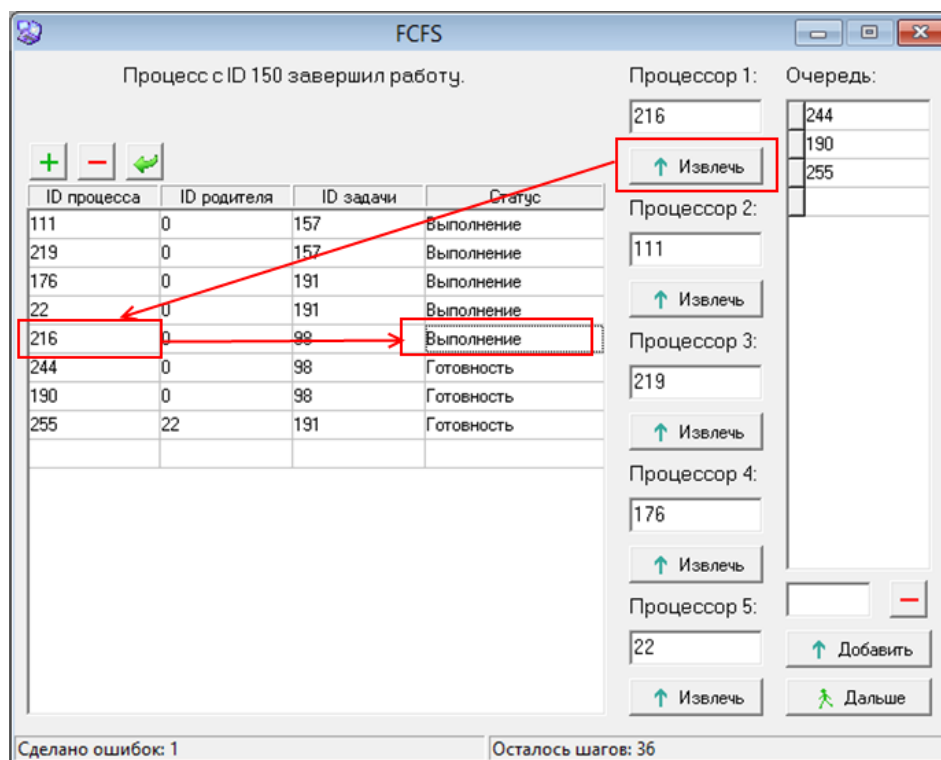


Рис. 1.21. Перемещение процесса из очереди на выполнение

5) Вход процесса в состояние ожидания ввода/вывода

Процессу присваивается статус «Ввод/вывод», а процессор занимается первым элементом из очереди на выполнение (рис. 1.22).

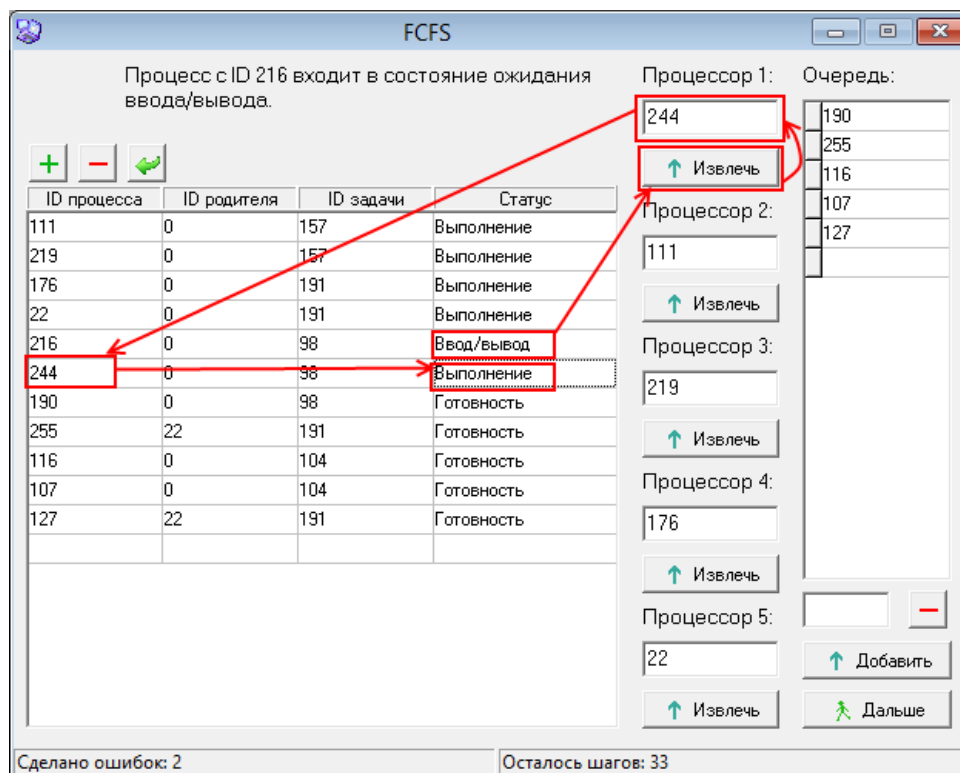


Рис. 1.22. Вход процесса в состояние ожидания ввода/вывода

6) Выход процесса из состояния ожидания ввода/вывода

С помощью выпадающего меню процессу устанавливается тот статус, который был до состояния ожидания ввода/вывода (рис. 1.23).

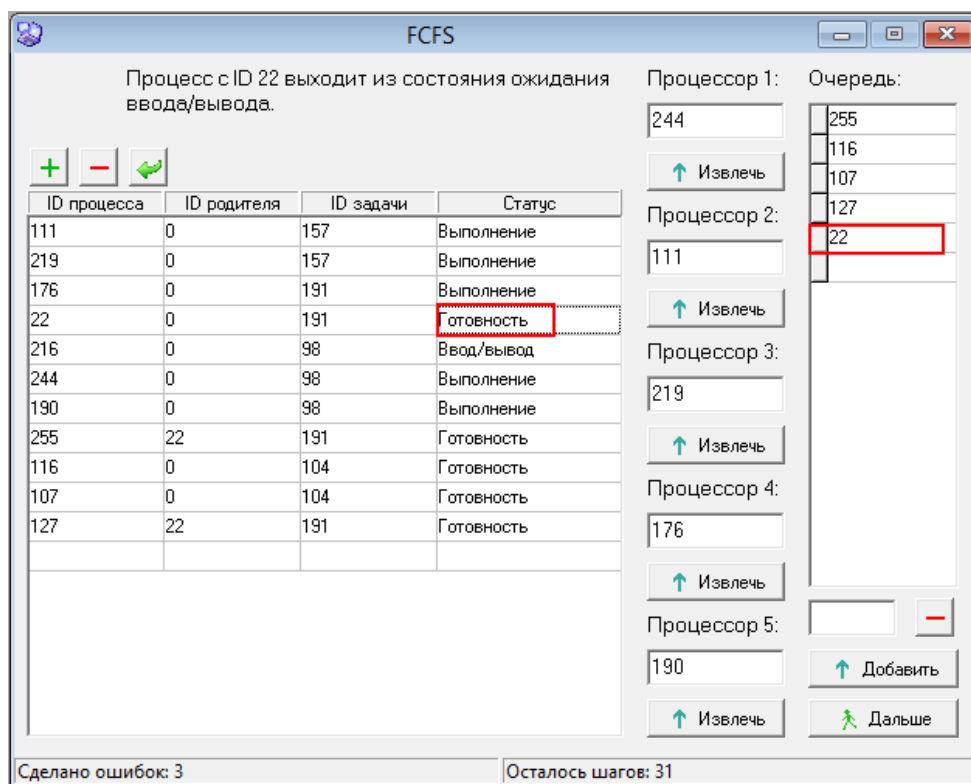


Рис. 1.23. Выход процесса из состояния ввода/вывода

7) Снятие задачи с выполнения

Чтобы удалить задачу, необходимо удалить все процессы, относящиеся к ней (рис. 1.24). Если они находятся вне очереди, нужно освободить процессоры и поместить на выполнение ожидающие в очереди элементы.

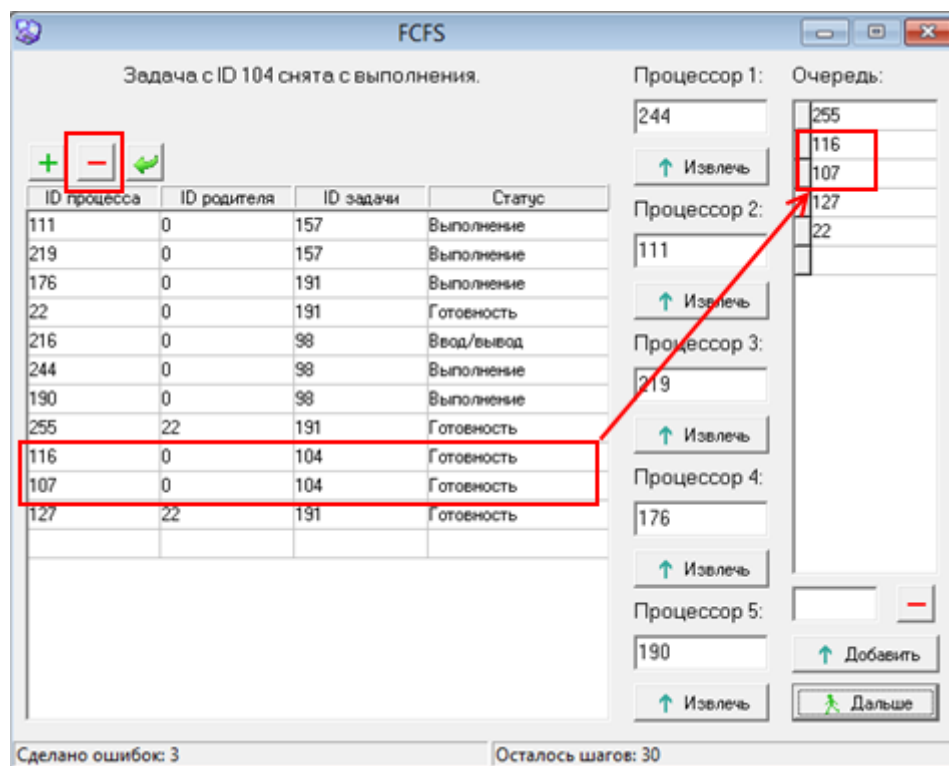


Рис. 1.24. Снятие задачи с выполнения

1.2.2. Круговой алгоритм планирования

Круговой алгоритм планирования процессов (round-robin process scheduling) помещает все готовые к выполнению процессы в глобальную очередь выполнения. Этот алгоритм в многопроцессорных системах работает почти так же, как и в однопроцессорных – процесс выполняется в течение максимум одного полного кванта времени, а затем приходит очередь другого процесса. Ранее выполнявшийся процесс помещается в конец глобальной очереди выполнения. Использование этого алгоритма также предотвращает бесконечное откладывание выполнения процесса, но он не обеспечивает высокую степень параллельности или хорошей привязки к процессорам, поскольку игнорирует связи между процессами.

Описание лабораторной установки

Интерфейс окна для алгоритма RR идентичен интерфейсу для алгоритма FCFS и представлен на рис. 1.8.

Выполнение лабораторной работы

При поступлении новой задачи, среди пары процессов больший приоритет имеет тот, который указан раньше в событии;

Кроме того, по истечению кванта времени выполняемые процессы снимаются с выполнения, а на выполнение встают первые из очереди элементы.

Выполнение шагов:

- 1) Добавление процессов, появление новой задачи, порождение дочернего процесса, завершение работы процесса, вход процесса в состояние ожидания ввода/вывода, выход процесса из состояния ожидания ввода/вывода, снятие задачи с выполнения выполняются так же, как и в дисциплине FCFS.

- 2) Истечение кванта времени

Процесс выполняется в течение максимум одного полного кванта времени, а затем заменяется на первый процесс из очереди. Ранее выполнявшийся процесс помещается в конец глобальной очереди выполнения.

1.2.3. Алгоритм планирования SPF

SPF расшифровывается как *shortest-process-first* («кратчайший процесс – первым»). При использовании этого алгоритма планирования первым запускается процесс, на полное выполнение которого требуется меньше всего времени. Как вытесняющая, так и невытесняющая версии

данного алгоритма демонстрируют меньшее время ожидания для интерактивных процессов, чем алгоритм FCFS, поскольку интерактивные процессы обычно являются «короткими». Однако при использовании алгоритма SPF запуск на выполнение «длинного» процесса может бесконечно откладываться, если в системе один за другим появляются более «короткие» процессы. Как и все задачно-независимые алгоритмы планирования, SPF не обращает внимания на параллелизм и привязку к процессорам.

Описание лабораторной установки

Интерфейс окна для алгоритма SPF практически полностью совпадает с интерфейсом окна для алгоритмов FCFS и RR и представлен на рис. 1.25.

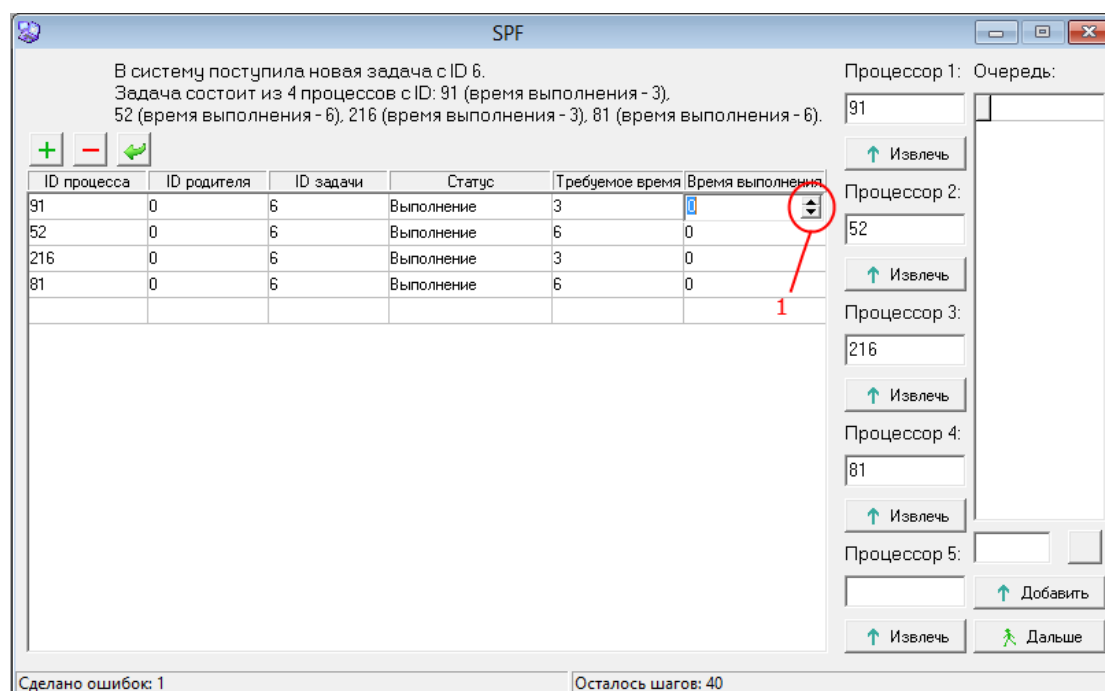


Рис. 1.25. Интерфейс окна для алгоритма SPF

Таблица процессов содержит два новых поля: «Требуемое время» и «Время выполнения» (счетчик).

Кнопки (1) предназначены для увеличения/уменьшения содержимого поля «Время выполнения».

Выполнение лабораторной работы

Приоритеты процессов определяются следующим образом.

- 1) При поступлении новой задачи среди пары процессов с равным требуемым временем работы больший приоритет имеет тот, который указан раньше в событии.
- 2) Среди двух процессов, которым до завершения требуется одинаковое количество времени, больший приоритет имеет тот, который поступил в систему раньше.
- 3) Среди двух процессов, один из которых превысил ожидаемое время работы, а второй нет, больший приоритет имеет второй.
- 4) Среди двух процессов, которые превысили ожидаемое время работы, больший приоритет имеет тот, который поступил в систему раньше.

В конце каждого такта необходимо выполнять увеличение «Времени выполнения».

В лабораторной установке используется вытесняющая версия алгоритма.

Выполнение шагов:

- 1) Добавление процессов

Выполняется так же, как и в алгоритмах FCFS и RR (рис. 1.9–1.14).

В таблице процессов появляется два новых столбца – «Требуемое время» и «Время выполнения».

- 2) Перераспределение процессов

После добавления новой задачи процессорное время расходуется на процесс с наименьшим требуемым временем. Остальные процессы помещаются в очередь по возрастанию требуемого времени (рис. 1.26).

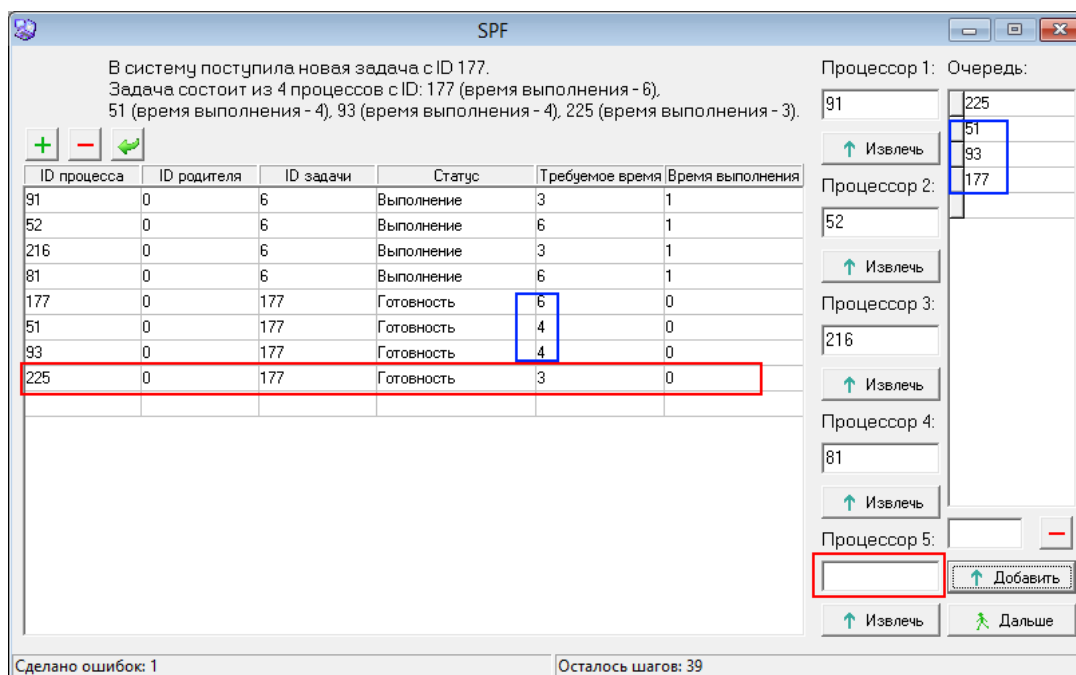


Рис. 1.26. Добавление новой задачи

Затем просматриваются выполняющиеся процессы. Процессы с наибольшим требуемым временем вытесняются процессами с меньшим временем, находящимися в очереди на выполнение (рис. 1.27-1.31).

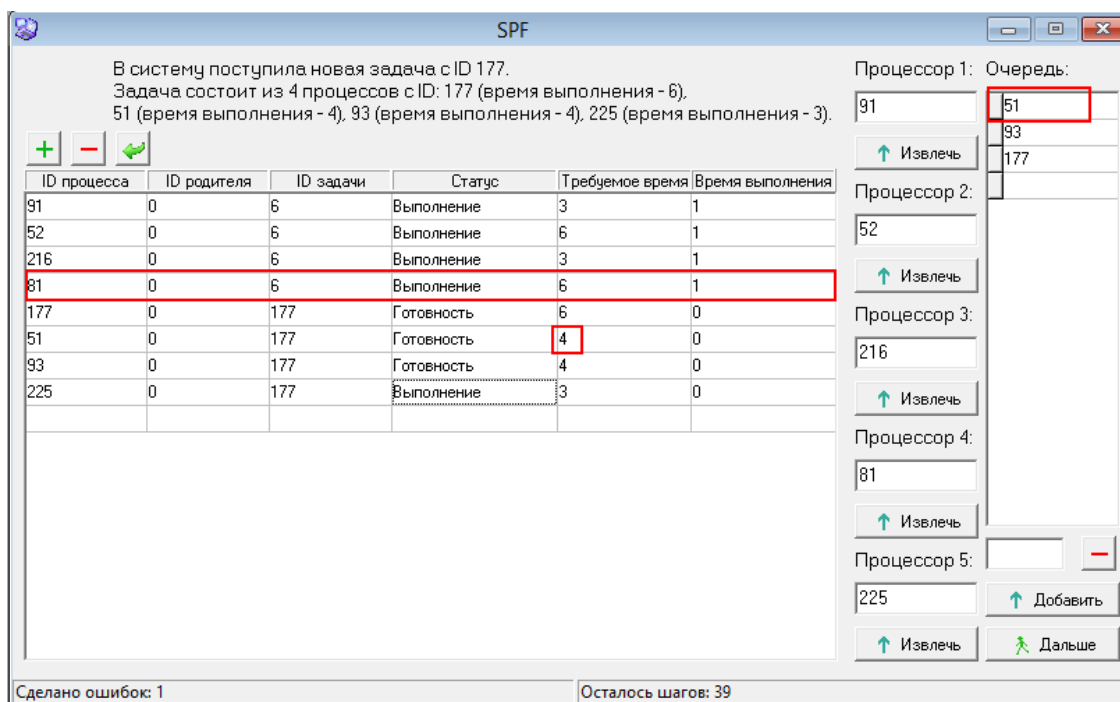


Рис. 1.27. Выбор процесса, который вытеснит процесс с ID 81

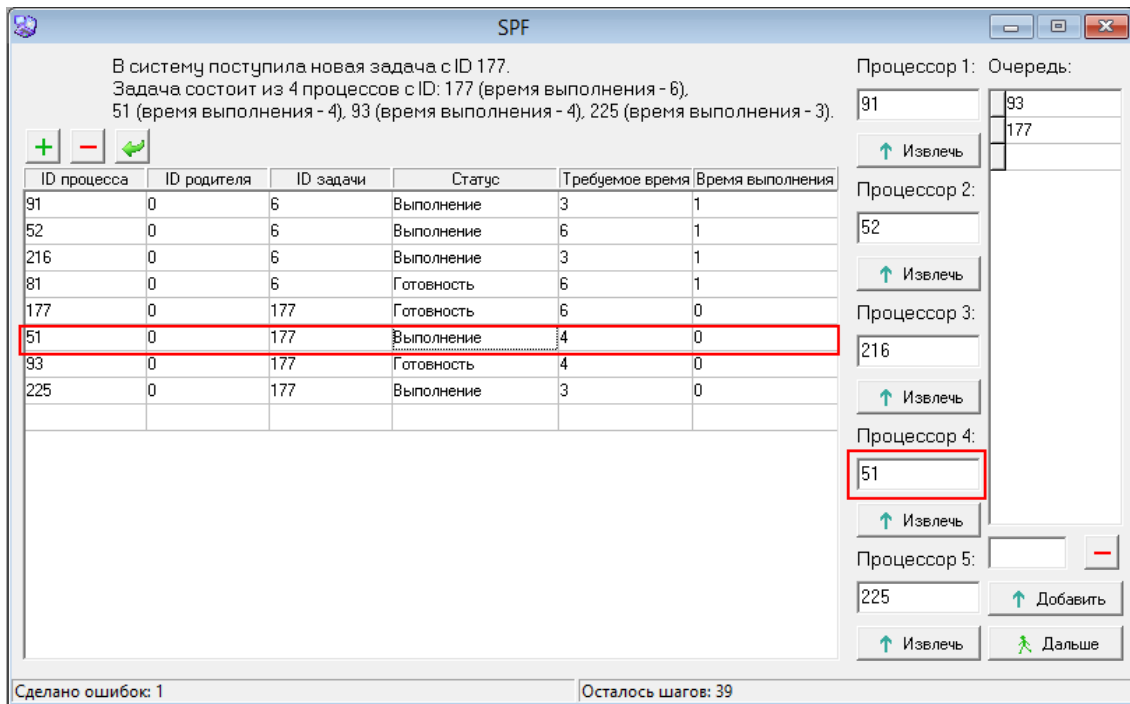


Рис. 1.28. Процесс с ID 51 выполняется

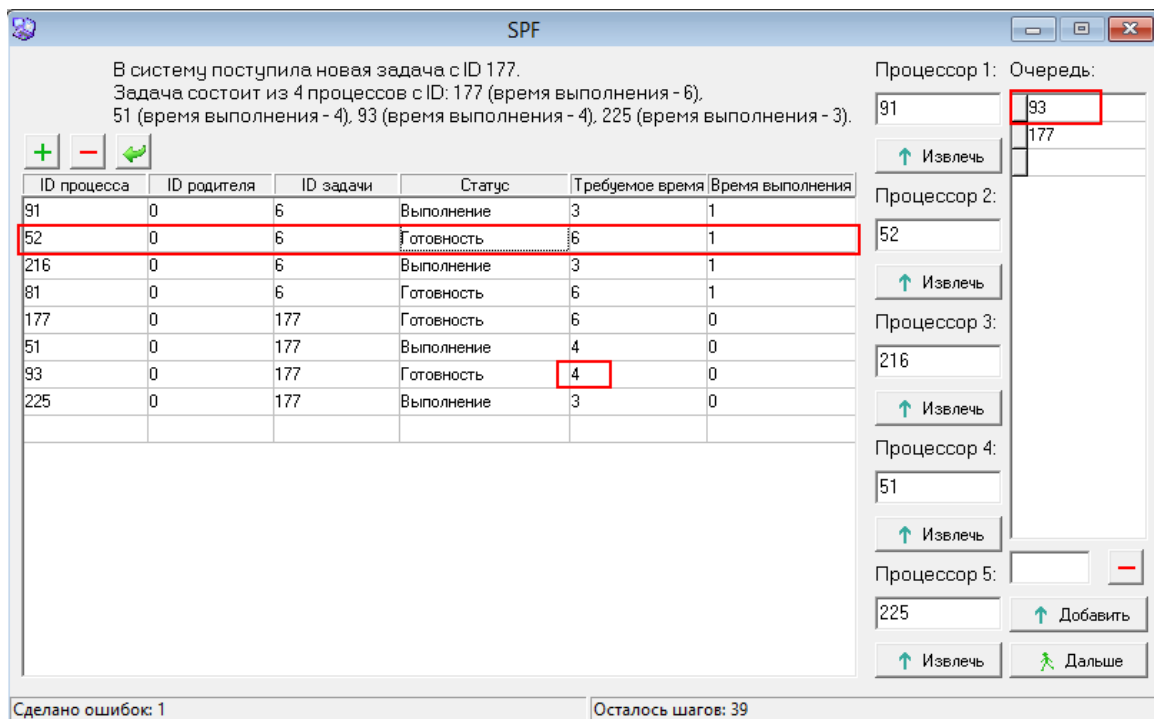


Рис. 1.29. Выбор процесса, который вытеснит процесс с ID 52

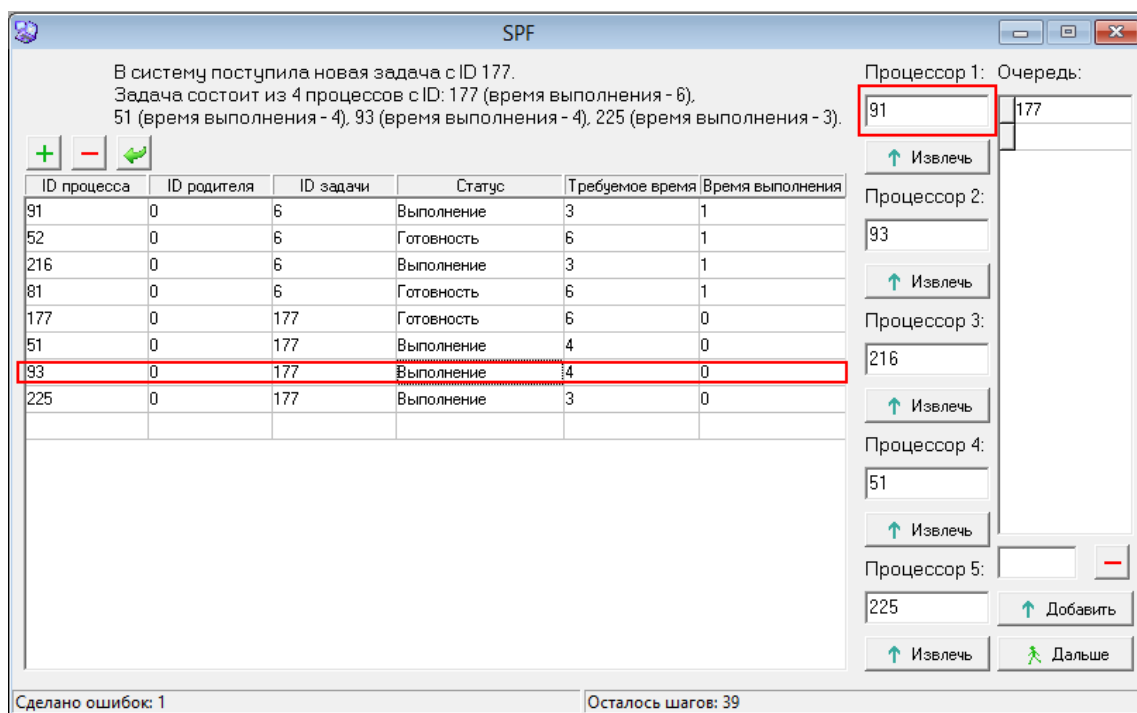


Рис. 1.30. Процесс с ID 93 выполняется

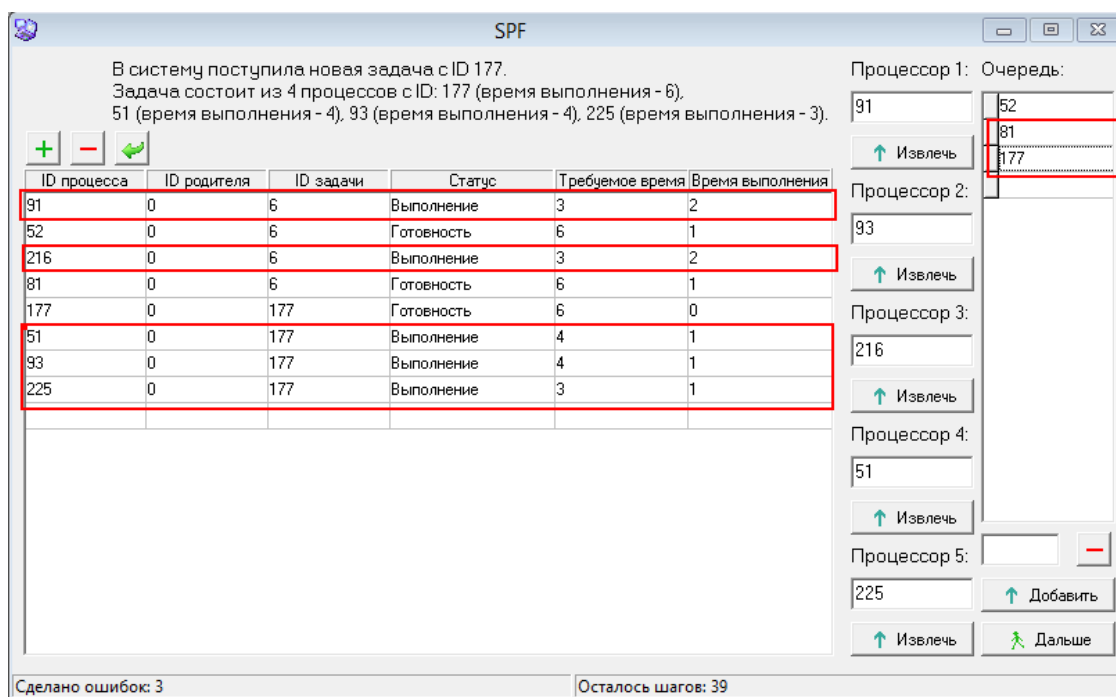


Рис. 1.31. Результат перераспределения процессов

Вытесненные процессы помещаются в очередь по возрастанию требуемого времени.

3) Процессы, проработавшие дольше, чем заявлено

Если появляются такие процессы (рис. 1.32), они освобождают процессор, попадая в конец очереди на выполнение (несмотря на требуемое время выполнения). Свободный процессор занимается процессом, находящимся первым в очереди (рис. 1.33).

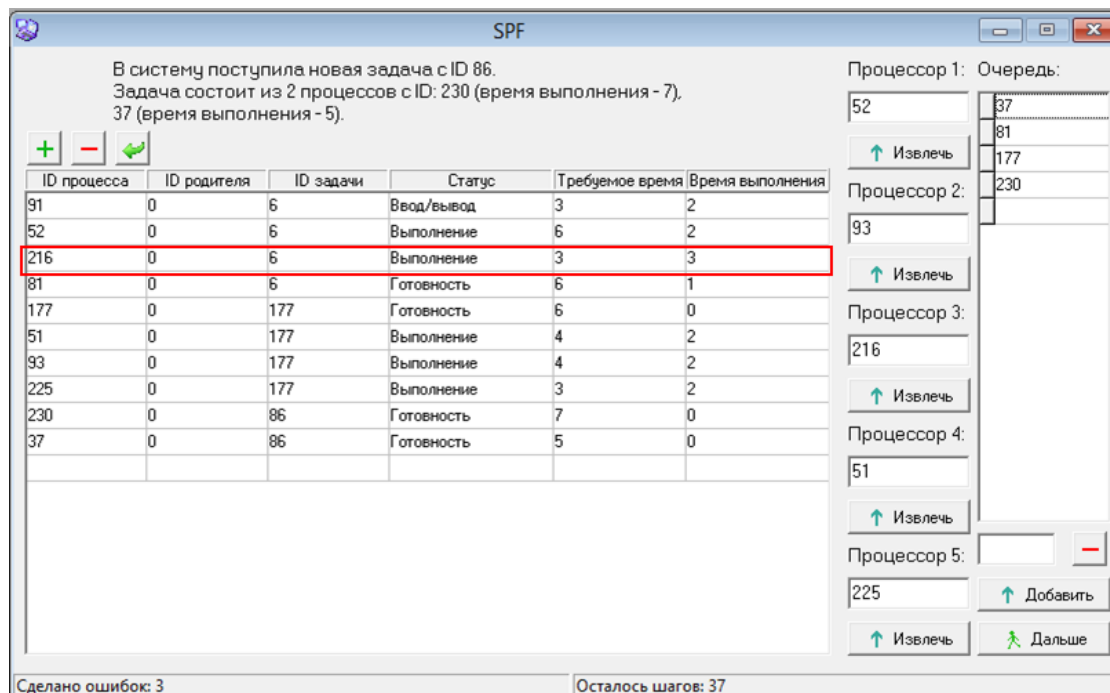


Рис. 1.32. Процесс с ID 216 работает дольше запланированного

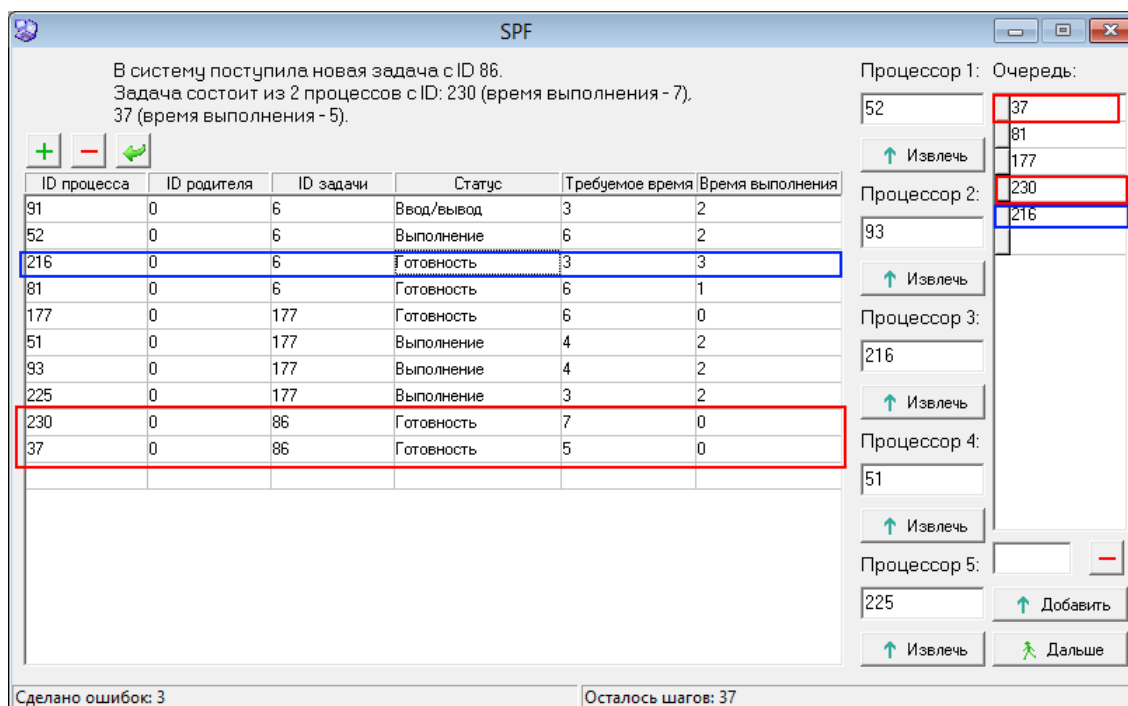


Рис. 1.33. Замена процесса

Затем смотрится следующий в очереди процесс и время его выполнения. На рис. 1.34 замена процесса с ID 52 (с наибольшим временем выполнения) на процесс с ID 81 не будет иметь смысла.

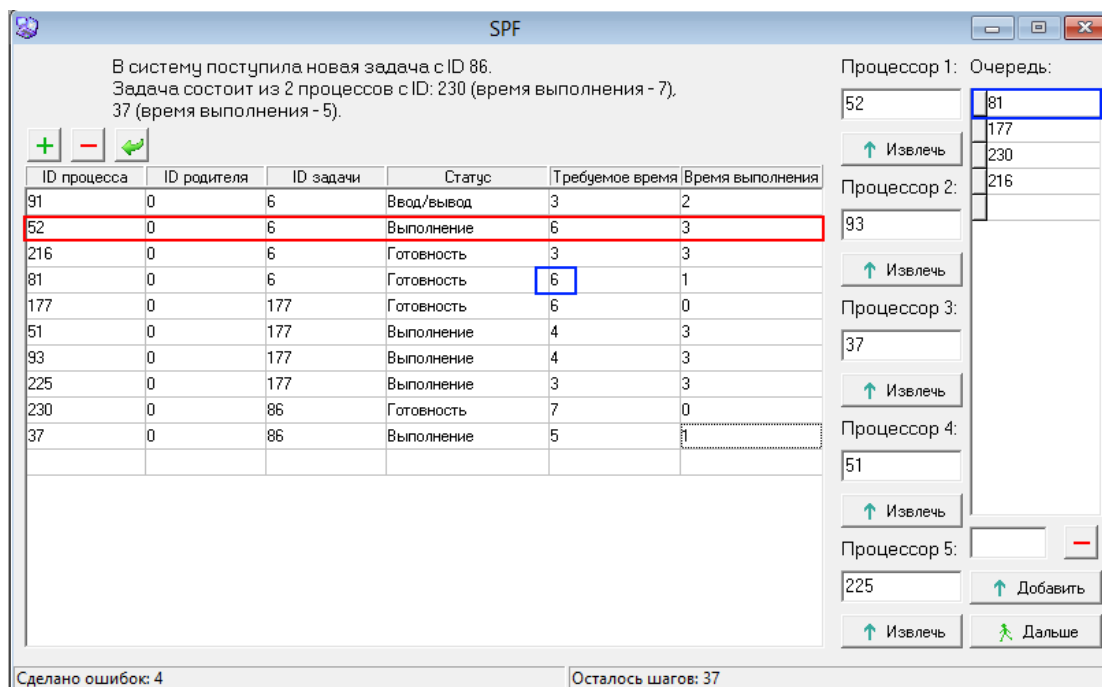


Рис. 1.34. Сравнение процессов с ID 52 и с ID 81

В том случае, если процесс уже в состоянии готовности, единственный в очереди, и какой-либо из процессоров свободен, он снова переходит в состояние выполнения (рис. 1.35 и 1.36).

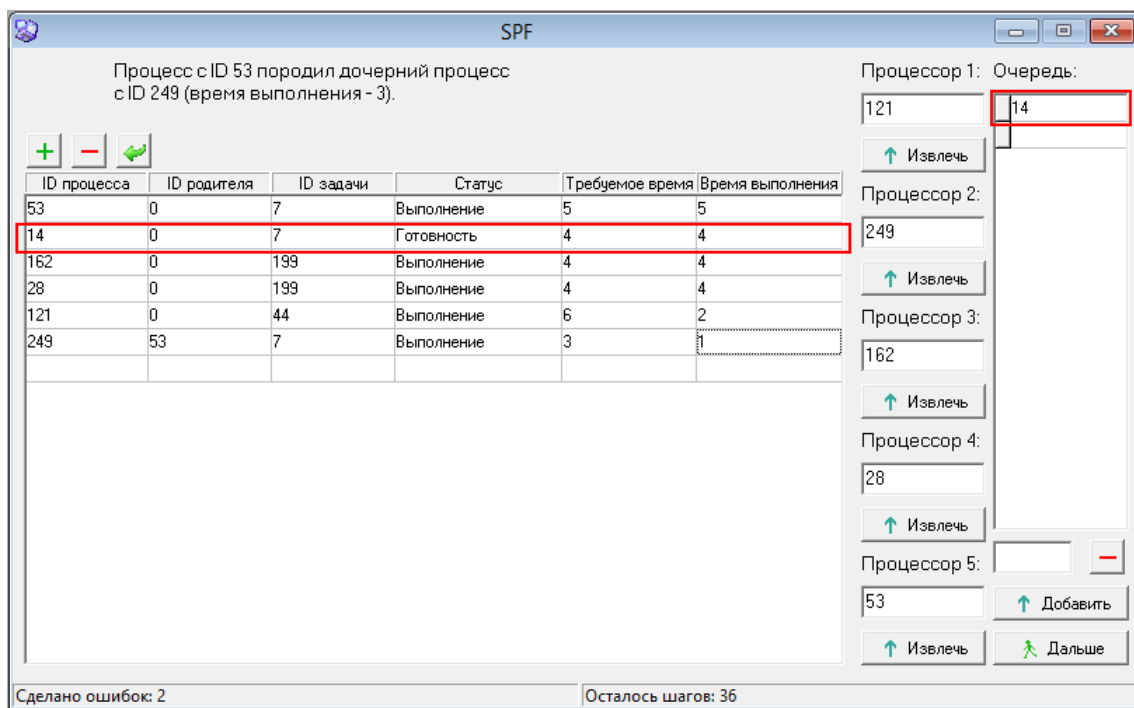


Рис. 1.35. Процесс с ID 14 в состоянии готовности

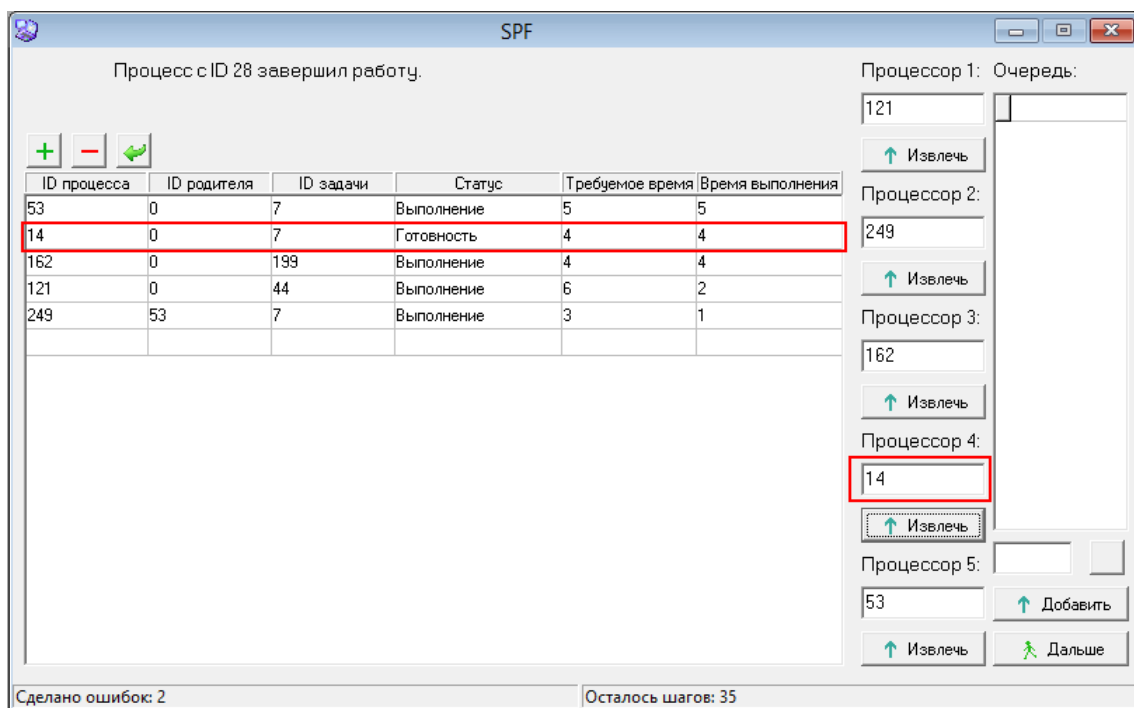


Рис. 1.36. После завершения работы процесса с ID 28, процесс с ID 14 снова выполняется

Если в один такт процесс переходит в состояние готовности, и возникает новый процесс, приоритет на выполнение будет у вновь поступившего процесса.

1.3. Задачно-ориентированные алгоритмы планирования

Задачно-ориентированные алгоритмы оценивают свойства каждой задачи и пытаются достичь максимального её распараллеливания или мягкой привязки к процессору, таким образом повышая производительность за счет снижения накладных расходов. Но они не учитывают моментов, специфичных для планирования процессов в многопроцессорных системах. Например, если два процесса, интенсивно взаимодействующие друг с другом, будут выполняться не одновременно, то существенную часть времени они будут просто ожидать ответа на свои запросы. Следовательно, производительность системы упадет. Кроме того, в большинстве многопроцессорных систем у каждого процессора есть собственный кэш. Процессы из одной задачи часто обращаются к общим данным в памяти, и расположение этих процессов на одном и том же процессоре может увеличить частоту попаданий в кэш, что приведет к повышению производительности работы с памятью. В целом задачно-ориентированные алгоритмы нацелены на достижение максимального распараллеливания и привязки к процессорам за счет увеличения сложности алгоритмов.

К основным задачно-ориентированным алгоритмам относятся SNPF, круговой алгоритм, алгоритм совместного планирования и динамическое разделение.

1.3.1. Алгоритм планирования SNPF

Теоретическая часть

SNPF расшифровывается как *smallest-number-of-processes-first* («наименьшее количество процессов – первым»). Этот алгоритм может быть как вытесняющим, так и невытесняющим, но в обоих вариантах он

использует глобальную очередь выполнения. Приоритет задачи в такой очереди будет обратно пропорционален количеству процессов в этой задаче. Если задачи с одинаковым количеством процессов конкурируют, пытаясь занять один и тот же процессор, задача, которая дольше ожидала, получает приоритет. В невытесняющей версии алгоритма SNPF, когда процессор становится доступным, алгоритм выбирает процесс из задачи в начале очереди выполнения и позволяет ему выполняться до завершения. В вытесняющей версии, если в очереди появляется новая задача с малым количеством процессов, она получает высокий приоритет и ее процессы начинают выполняться так скоро, как это возможно. Алгоритмы SNPF улучшают параллелизм, поскольку процессы из одной задачи часто могут выполняться одновременно. Однако эти алгоритмы не улучшают привязку к процессорам. Кроме того, при их использовании возможна ситуация, когда выполнение задачи с большим количеством процессов будет откладываться бесконечно.

Описание лабораторной установки

Интерфейс окна для алгоритма SNPF представлен на рис. 1.37.

Группа кнопок (1) предназначена для добавления процесса в таблицу процессов, удаления выделенной строки в таблице, а также сброса всех значений в начальное состояние (на момент последнего успешного шага).

Кнопки (2) позволяет выполнять переупорядочивание процессов в таблице.

Кнопка (3) осуществляет переход к следующему событию.

Таблица процессов содержит следующие поля: «ID процесса» – идентификатор процесса, «ID родителя» – номер родительского процесса (или 0, если процесс не имеет родителя), «ID задачи» – идентификатор задачи. Поле «Статус» представляет собой выпадающий список с тремя значениями: «Готовность», «Выполнение», «Ввод/вывод».

В нижней части окна указано количество процессоров в системе.

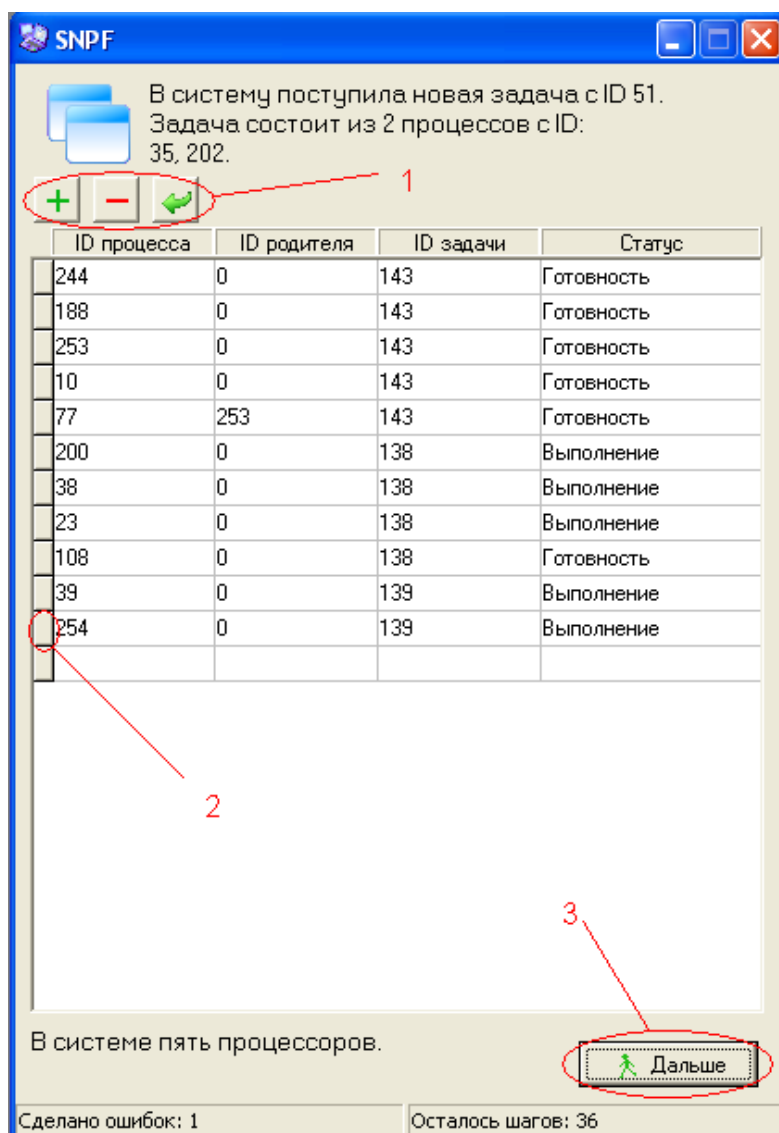


Рис. 1.37. Интерфейс окна для алгоритма SNPF

Выполнение лабораторной работы

При подсчете числа процессов задачи учитываются как процессы, находящиеся в состоянии готовности, так и процессы, находящиеся в состоянии ввода/вывода;

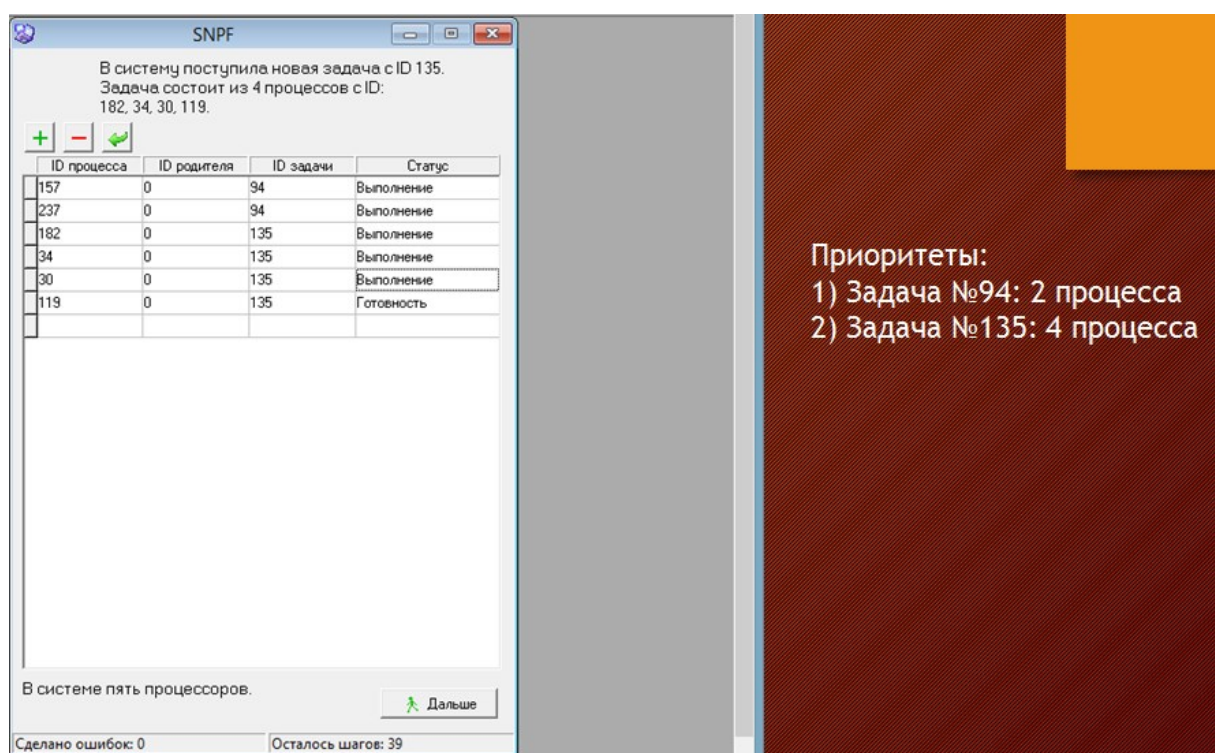
Среди двух задач с равным количеством процессов больший приоритет имеет та, которая раньше поступила в систему;

В лабораторной установке используется вытесняющая версия алгоритма.

Выполнение шагов:

1) Добавление новой задачи

При добавлении новой задачи сравнивается количество процессов в задачах. Более высокий приоритет имеют задачи с наименьшим количеством процессов, а также, при их одинаковом количестве, задачи с большим временем ожидания. Примеры представлены на рис. 1.38–1.41.



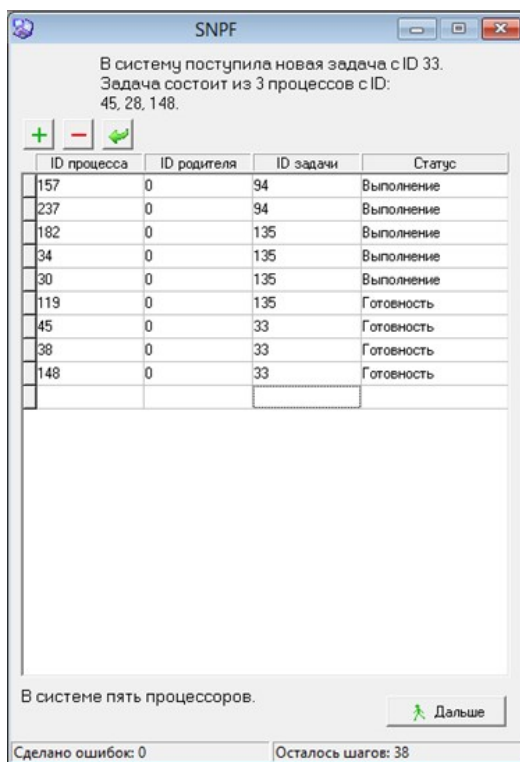
The screenshot shows the SNPF software interface. At the top, a message states: "В систему поступила новая задача с ID 135. Задача состоит из 4 процессов с ID: 182, 34, 30, 119." Below this is a table with the following data:

ID процесса	ID родителя	ID задачи	Статус
157	0	94	Выполнение
237	0	94	Выполнение
182	0	135	Выполнение
34	0	135	Выполнение
30	0	135	Выполнение
119	0	135	Готовность

Below the table, it says "В системе пять процессоров." and there is a "Далее" button. At the bottom, it shows "Сделано ошибок: 0" and "Осталось шагов: 39". To the right of the screenshot, a red box contains the following text:

Приоритеты:
1) Задача №94: 2 процесса
2) Задача №135: 4 процесса

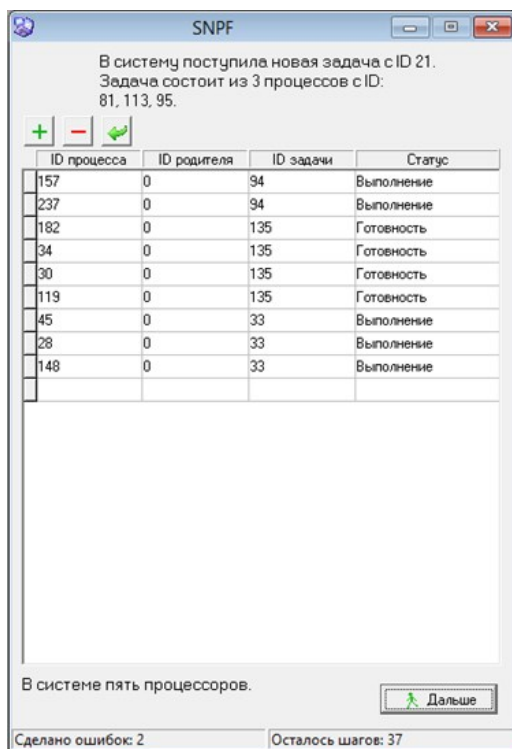
Рис. 1.38. Пример № 1



Приоритеты:

- 1) Задача №94: 2 процесса
- 2) Задача №33: 3 процесса
- 3) Задача №135: 4 процесса

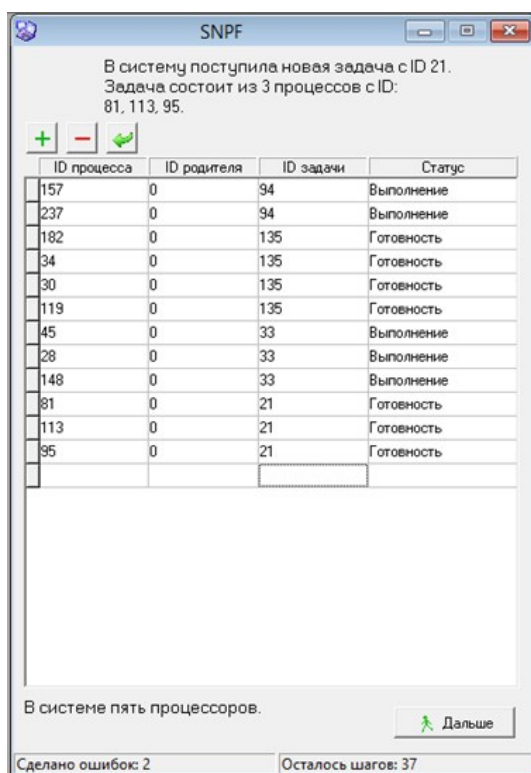
Рис. 1.39. Добавление процессов задачи с ID 33



Приоритеты:

- 1) Задача №94: 2 процесса
- 2) Задача №33: 3 процесса
- 3) Задача №135: 4 процесса

Рис. 1.40. Изменение статусов задач с ID 135 и ID 33



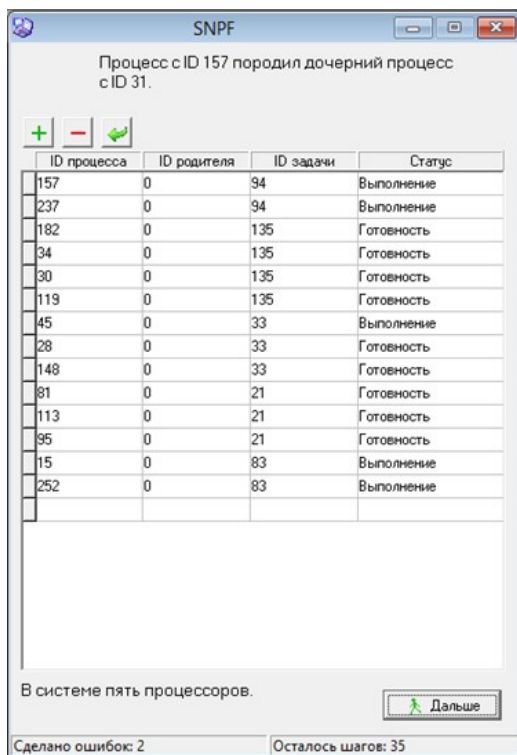
Приоритеты:

- 1) Задача №94: 2 процесса
- 2) Задача №33: 3 процесса
- 3) Задача №21: 3 процесса
- 4) Задача №135: 4 процесса

Рис. 1.41. Пример № 4

2) Порождение дочернего процесса

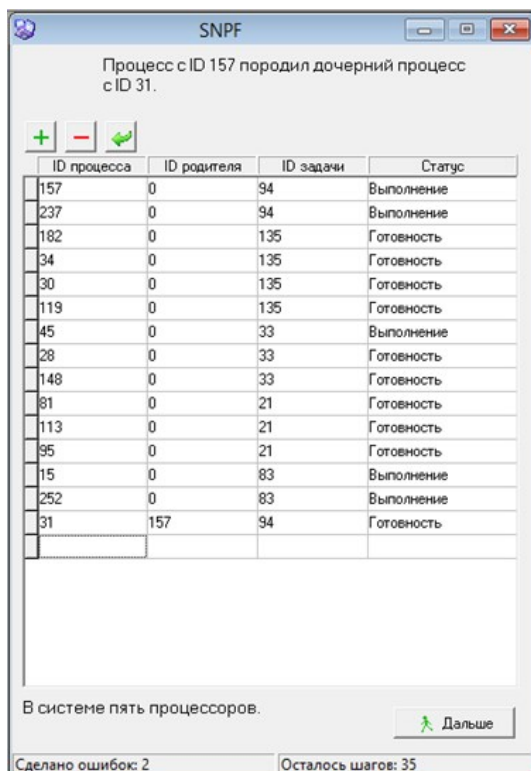
При создании дочернего процесса количество процессов задачи родителя увеличивается на 1, что влечет за собой сравнение задач и расставление новых приоритетов. Пример приведен на рис. 1.42–1.44.



Приоритеты:

- 1) Задача №94: 2 процесса +1
- 2) Задача №33: 3 процесса
- 3) Задача №21: 3 процесса
- 4) Задача №135: 4 процесса

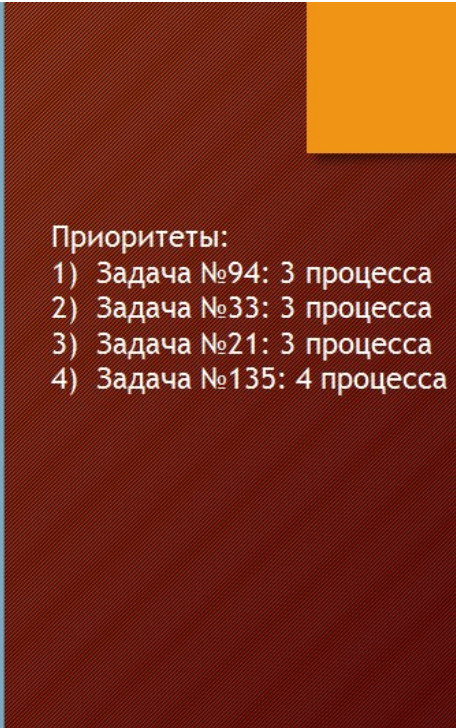
Рис. 1.42. Запрос на создание дочернего процесса



Приоритеты:

- 1) Задача №94: 3 процесса
- 2) Задача №33: 3 процесса
- 3) Задача №21: 3 процесса
- 4) Задача №135: 4 процесса

Рис. 1.43. Дочерний процесс создан



3) Завершение работы процесса

Процесс с ID 157 завершил работу.

ID процесса	ID родителя	ID задачи	Статус
157	0	94	Выполнение
237	0	94	Выполнение
31	157	94	Выполнение
182	0	135	Готовность
34	0	135	Готовность
30	0	135	Готовность
119	0	135	Готовность
45	0	33	Готовность
28	0	33	Готовность
148	0	33	Готовность
81	0	21	Готовность
113	0	21	Готовность
95	0	21	Готовность
15	0	83	Выполнение
252	0	83	Выполнение

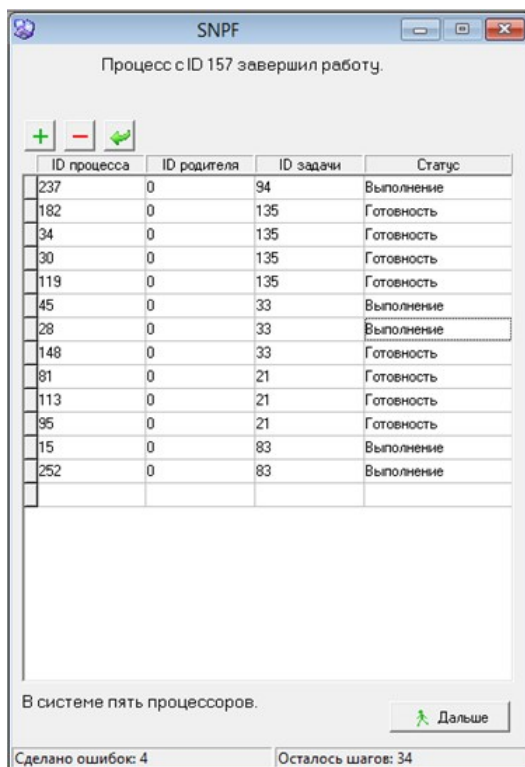
В системе пять процессоров.

Осталось шагов: 34

Дальше

- 1) Задача №94: 3 процесса
- 2) Задача №33: 3 процесса
- 3) Задача №21: 3 процесса
- 4) Задача №135: 4 процесса

48



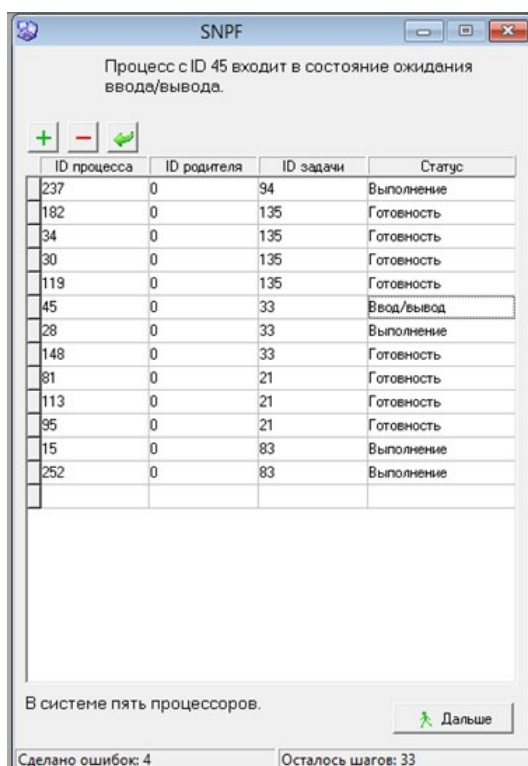
Приоритеты:

- 1) Задача №94: 1 процесс
- 2) Задача №33: 3 процесса
- 3) Задача №21: 3 процесса
- 4) Задача №135: 4 процесса

Рис. 1.46. Изменение приоритетов после завершения процесса с ID 157

4) Вход процесса в состояние ожидания ввода/вывода

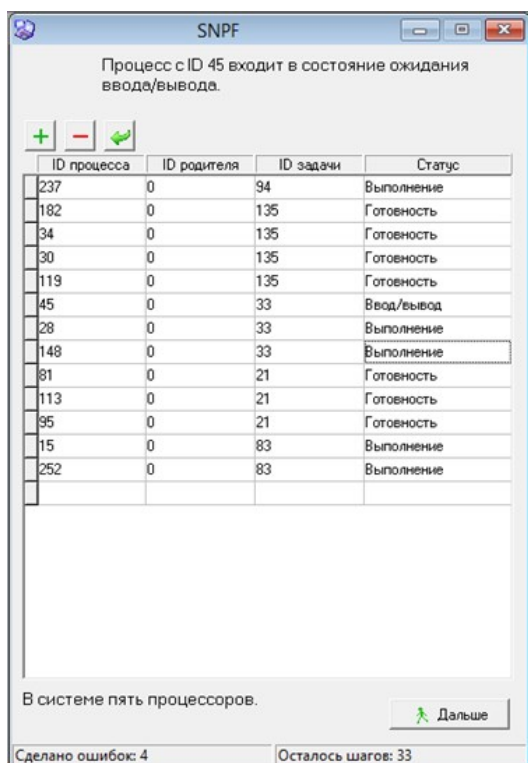
Если процесс входит в состояние ожидания ввода/вывода, на выполнение уходит первый находящийся в состоянии готовности процесс, относящийся к той же задаче, либо к задаче с более низким приоритетом (если все процессы текущей задачи уже выполняются). Пример представлен на рис. 1.47, 1.48.



Приоритеты:

- 1) Задача №94: 2 процесса
- 2) Задача №33: 3 процесса
- 3) Задача №21: 3 процесса
- 4) Задача №135: 4 процесса

Рис. 1.47. Вход в состояние ожидания ввода/вывода



Приоритеты:

- 1) Задача №94: 2 процесса
- 2) Задача №33: 3 процесса
- 3) Задача №21: 3 процесса
- 4) Задача №135: 4 процесса

Рис. 1.48. Процесс с ID 148 уходит на выполнение

5) Появление нескольких задач с одинаковым количеством процессов

При появлении нескольких задач с одинаковым количеством процессов, наибольший приоритет получает та задача, которая дольше всего находится в состоянии готовности. Пример представлен на рис. 1.49–1.52.

ID процесса	ID родителя	ID задачи	Статус
237	0	94	Выполнение
182	0	135	Готовность
34	0	135	Готовность
30	0	135	Готовность
119	0	135	Готовность
45	0	33	Ввод/вывод
28	0	33	Выполнение
148	0	33	Выполнение
81	0	21	Готовность
113	0	21	Готовность
95	0	21	Готовность
15	0	83	Выполнение
75	15	83	Выполнение
58	15	83	Готовность
51	0	1	Готовность
243	0	1	Готовность
23	0	1	Готовность

В системе пять процессоров.

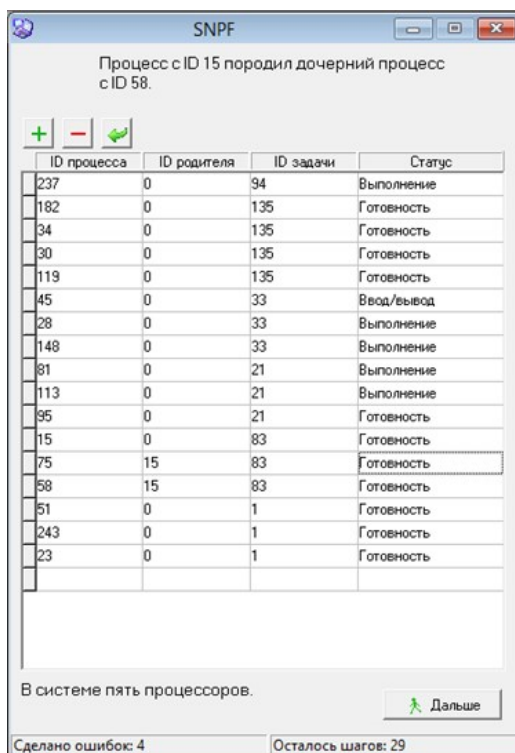
Сделано ошибок: 4 Осталось шагов: 29

Дальше

Приоритеты:

- 1) Задача №94: 1 процесс
- 2) Задача №83: 2 процесса+1
- 3) Задача №33: 3 процесса
- 4) Задача №21: 3 процесса
- 5) Задача №1: 3 процесса
- 6) Задача №135: 4 процесса

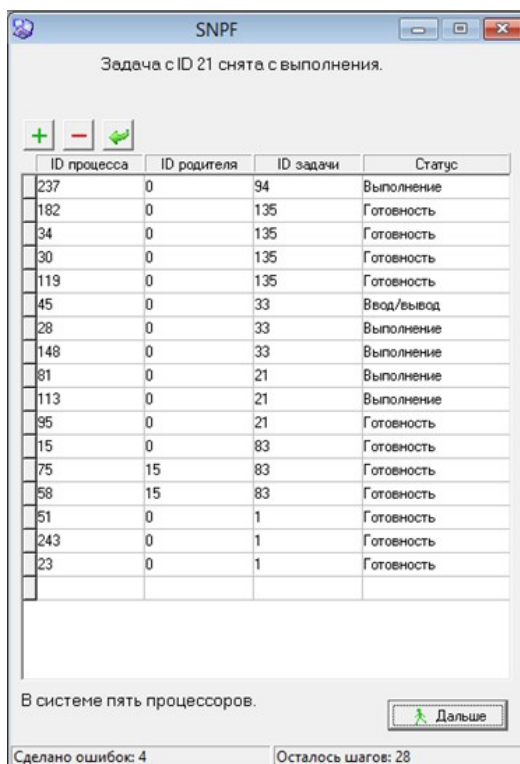
Рис. 1.49. Создание нового дочернего процесса



Приоритеты:

- 1) Задача №94: 1 процесс
- 2) Задача №33: 3 процесса
- 3) Задача №21: 3 процесса
- 4) Задача №83: 3 процесса
- 5) Задача №1: 3 процесса
- 6) Задача №135: 4 процесса

Рис. 1.50. Смена приоритетов



Приоритеты:

- 1) Задача №94: 1 процесс
- 2) Задача №33: 3 процесса
- 3) Задача №21: 3 процесса
- 4) Задача №83: 3 процесса
- 5) Задача №1: 3 процесса
- 6) Задача №135: 4 процесса

Рис. 1.51. Снятие задачи с выполнения

ID процесса	ID родителя	ID задачи	Статус
237	0	94	Выполнение
182	0	135	Готовность
34	0	135	Готовность
30	0	135	Готовность
119	0	135	Готовность
45	0	33	Ввод/вывод
28	0	33	Выполнение
148	0	33	Выполнение
15	0	83	Выполнение
75	15	83	Выполнение
58	15	83	Готовность
51	0	1	Готовность
243	0	1	Готовность
23	0	1	Готовность

Приоритеты:

- 1) Задача №94: 1 процесс
- 2) Задача №33: 3 процесса
- 3) Задача №83: 3 процесса
- 4) Задача №1: 3 процесса
- 5) Задача №135: 4 процесса

Рис. 1.55. Смена приоритетов

1.3.2. Круговой алгоритм планирования

Теоретическая часть

Круговой алгоритм планирования задач (round-robin job scheduling) помещает все готовые к выполнению задачи в глобальную очередь выполнения. Из этой очереди каждая задача назначается для выполнения группе процессоров (хотя и необязательно одной и той же группе каждый раз, когда до задачи доходит очередь выполняться). У каждой задачи есть собственная очередь процессов. Если в системе есть p процессоров, и она использует кванты продолжительности q , то задача получает в общей сложности $p \times q$ процессорного времени при ее отправке на выполнение. Обычно в задаче не точно p процессов, каждый из которых расходует целиком по одному кванту (то есть выполнение процесса может блокироваться до истечения кванта времени). Поэтому в алгоритме кругового планирования процессы задачи отправляются на выполнение,

пока задача не израсходует в общей сложности $p \times q$ времени, пока она не выполнится до конца или пока не заблокируются все ее процессы. Кроме того, алгоритм может поровну разделить $p \times q$ времени между всеми процессами задачи, позволяя каждому процессу выполняться, пока он не израсходует свой лимит времени, не завершится или не заблокируется. Альтернатива: если в задаче больше, чем p процессов, то алгоритм может выбрать p процессов для выполнения в течение кванта времени длиной q .

Как и в задачно-независимой версии алгоритма, бесконечное откладывание выполнения задачи исключено. Кроме того, поскольку процессы из одной задачи будут выполняться одновременно, этот алгоритм обеспечивает хороший уровень параллелизма. Однако дополнительные накладные расходы на переключение контекста могут снизить производительность системы.

1.3.3. Алгоритмы копланирования

Теоретическая часть

Алгоритмы копланирования, или совместного планирования (coscheduling или gang scheduling) используют глобальную очередь выполнения с циклическим доступом. Цель этих алгоритмов – выполнять процессы из одной задачи одновременно, вместо того, чтобы обеспечивать их максимальную привязку к процессорам. Есть несколько реализаций алгоритмов копланирования – матричные, непрерывные и неразделяющие. В данной работе рассматривается только неразделяемая реализация, поскольку она лишена некоторых слабых мест матричных и непрерывных реализаций.

Неразделяющий алгоритм копланирования (undivided coscheduling algorithm) помещает процессы из одной задачи в смежные позиции в глобальной очереди выполнения (рис. 1.53). Планировщик использует «окно», размер которого равен количеству процессоров в системе. Все

процессы, попадающие в окно, выполняются параллельно в течение максимум одного кванта. Выполнив обработку группы процессов, окно перемещается к следующей группе, которая тоже выполняется параллельно в течение одного кванта. Чтобы добиться максимальной загрузки процессоров, используется следующий подход: если процесс, попавший в окно, не может выполняться, окно расширяется на один процесс вправо, и в течение одного кванта выполняется также следующий процесс в очереди, даже если он не принадлежит к задаче, выполняемой в данный момент.

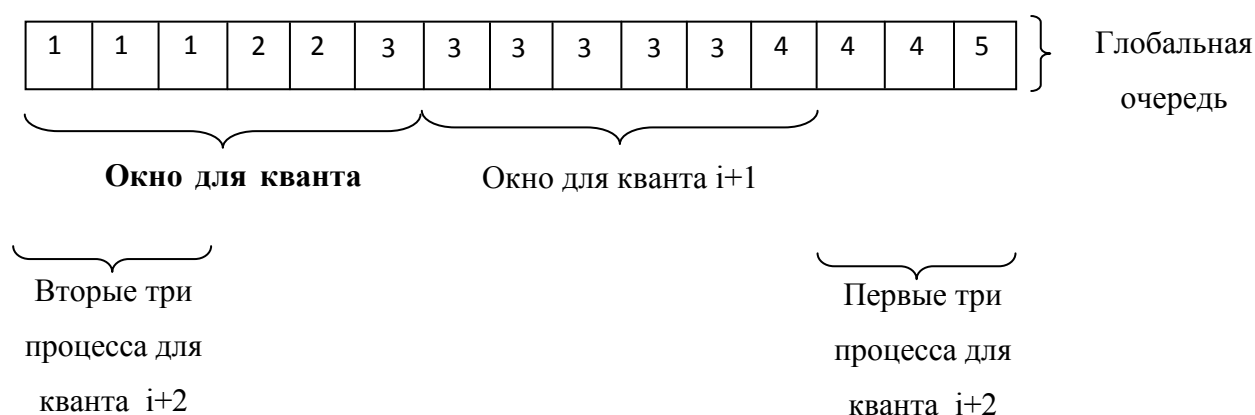


Рис. 1.53. Алгоритм копланирования (неразделяющая версия)

Поскольку в алгоритмах копланирования используется циклический доступ к очереди выполнения, при их использовании исключено бесконечное откладывание выполнения задачи. Кроме того, процессы из одной задачи часто выполняются одновременно. Поэтому алгоритмы копланирования позволяют программам, рассчитанным на параллельное выполнение, получать преимущество от работы в многопроцессорных системах. К сожалению, процесс может выполняться на разных процессорах каждый раз, когда до него доходит очередь, поэтому привязка его к процессору будет слабой.

Описание лабораторной установки

Интерфейс окна для алгоритма копланирования представлен на рис. 1.54.

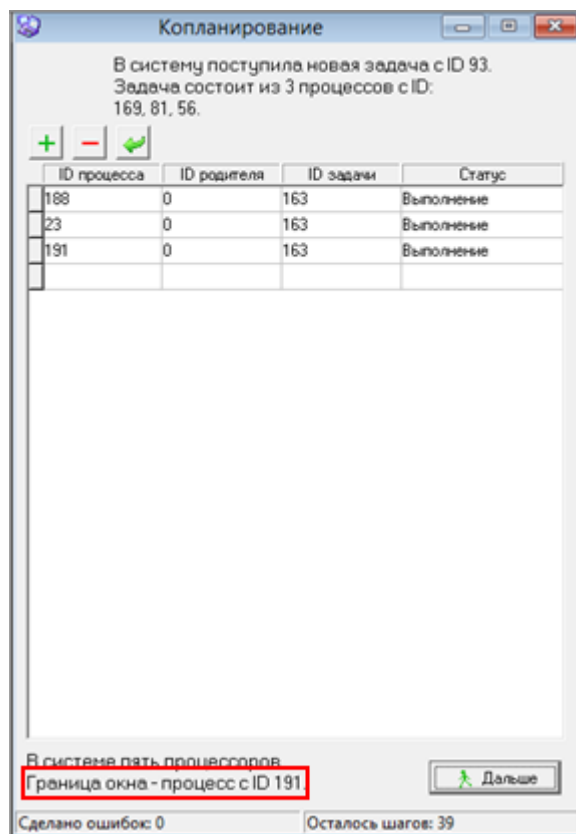


Рис. 1.54. Интерфейс окна для алгоритма копланирования

Интерфейс окна полностью совпадает с интерфейсом окна для алгоритма SNPF. В нижней части дополнительно указывается текущая граница процессорного окна.

Выполнение лабораторной работы

При поступлении новой задачи процессы должны добавляться в таблицу в том порядке, в котором они перечислены в событии;

В лабораторной установке используется неразделяющая версия алгоритма.

Выполнение шагов:

1) Добавление задачи

Добавление задач представлено на рис. 1.54, 1.55. Параллельно может выполняться пять процессов, остальные находятся в состоянии готовности. Граница окна показывает, какие процессы выполняются в текущий квант времени.

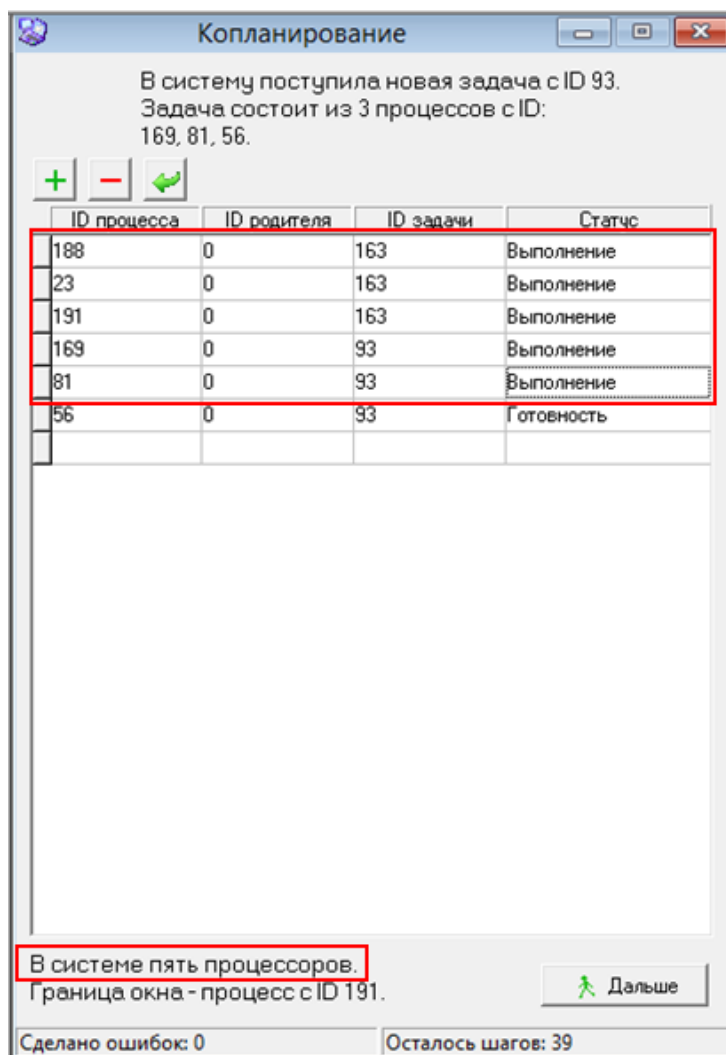


Рис. 1.55. Добавление новых процессов

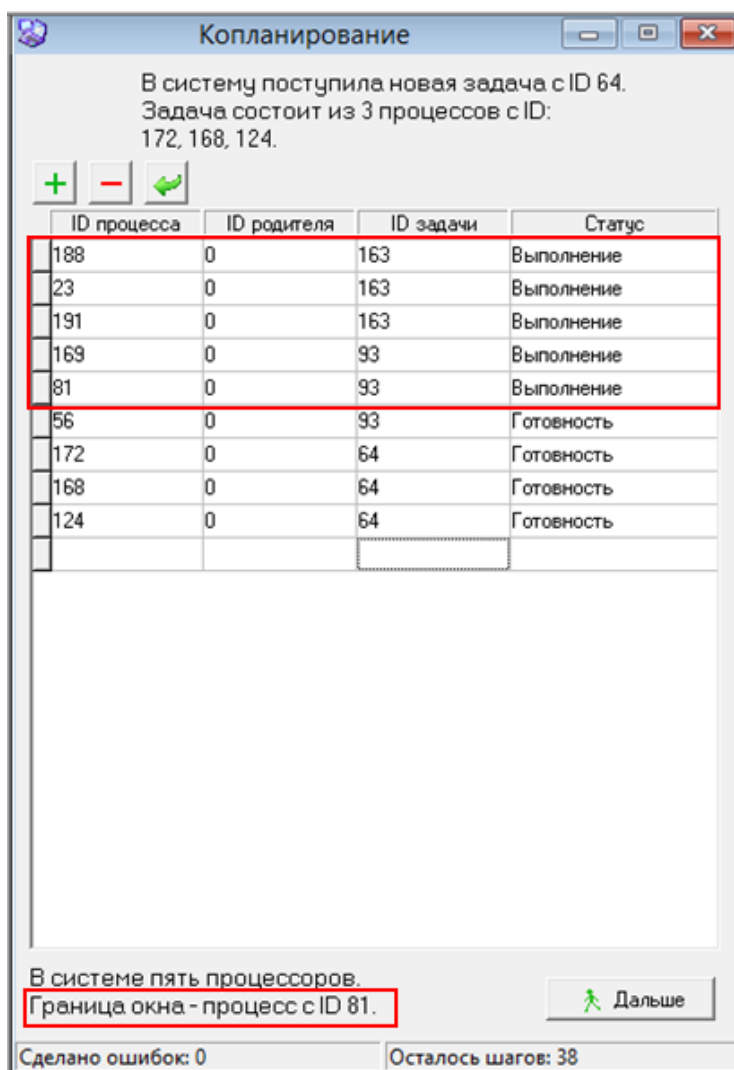


Рис. 1.56. Состояние строки «Граница окна»

2) Порождение дочернего процесса

Если дочерний процесс после создания поместить в конце таблицы (рис. 1.57), установка выдаст ошибку (рис. 1.58). Чтобы её исправить, процессы необходимо упорядочить по полю «ID задачи» (рис. 1.59).

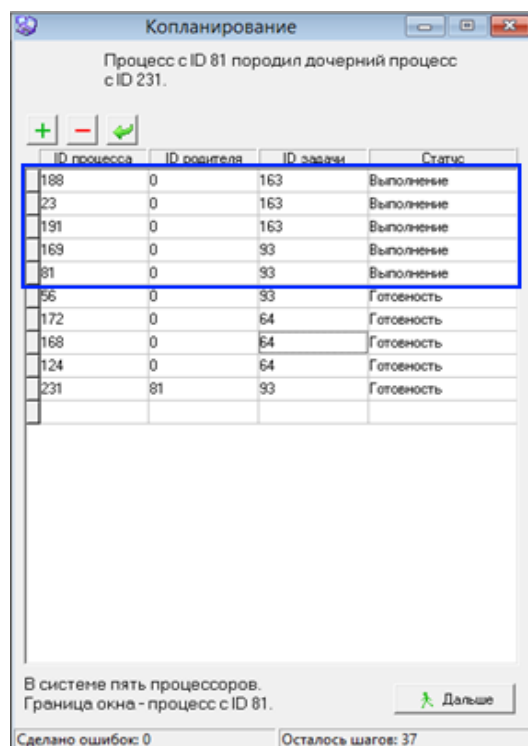


Рис. 1.57. Добавление дочернего процесса в конец таблицы
(в синей рамке – текущее окно планировщика)

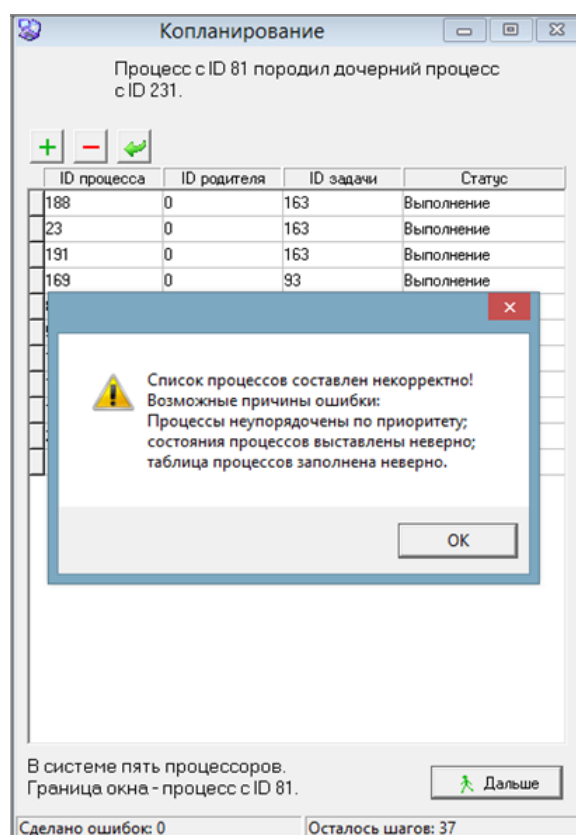


Рис. 1.58. Сообщение об ошибке

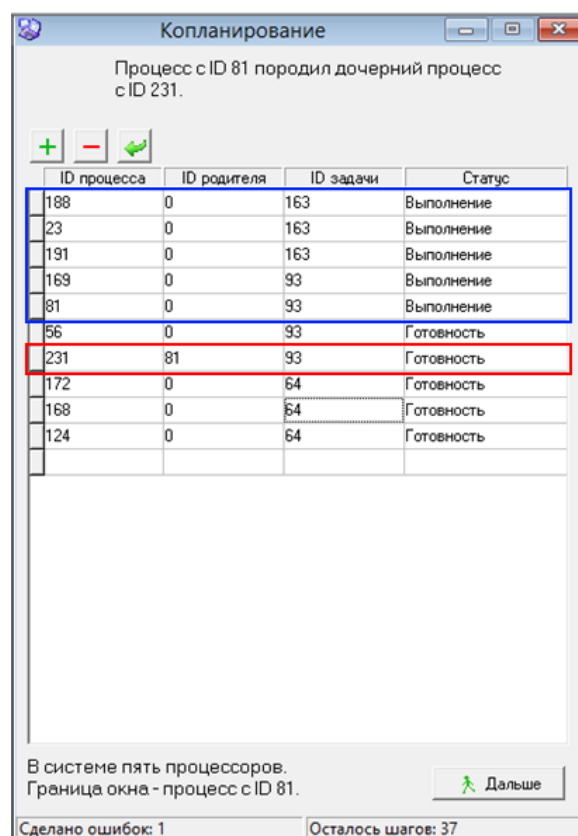


Рис. 1.59. Правильное расположение дочернего процесса

3) Конец кванта времени

После окончания кванта времени окно перемещается к следующей группе процессов, которые будут обрабатываться параллельно. Обработанные процессы принимают статус «Готовность» (рис. 1.60, 1.61).

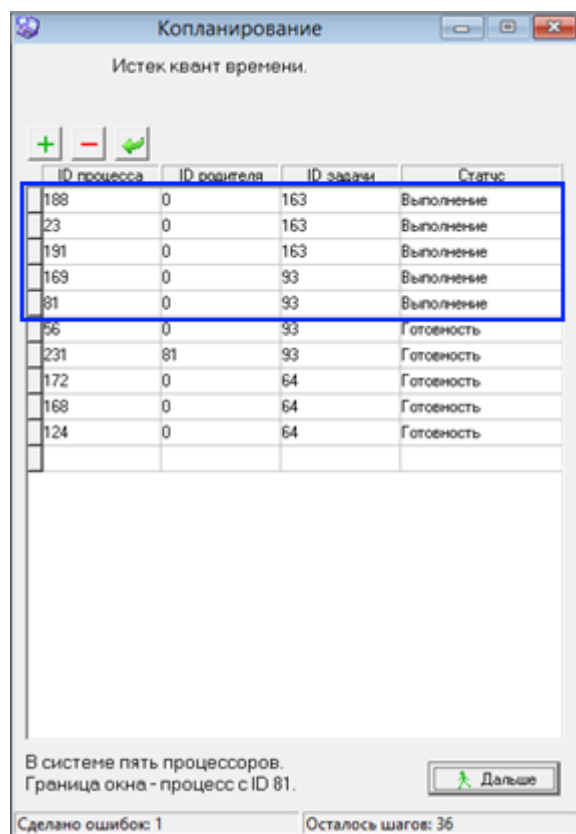


Рис. 1.60. Иstek квант времени

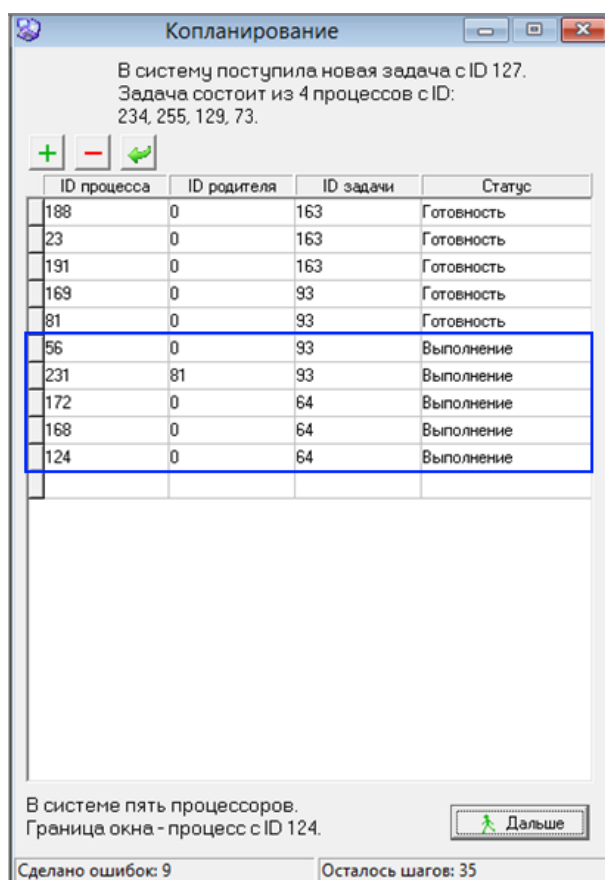


Рис. 1.61. Смещение окна планировщика

4) Вход процесса в состояние ожидания

Если по какой-либо причине процесс в текущем окне не может выполняться, окно расширяется, и до конца кванта времени выполняется следующий процесс в очереди, даже если он не относится к задаче, выполняемой в данный момент (рис. 1.62, 1.63).

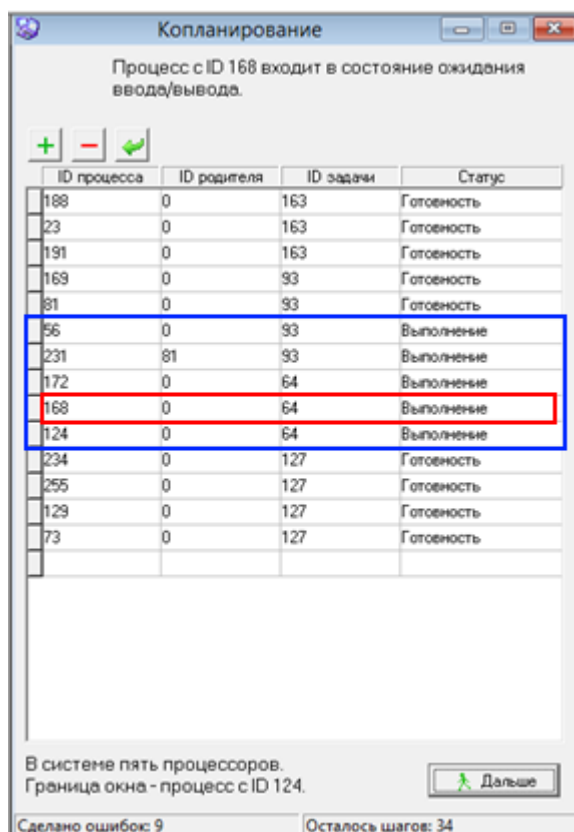


Рис. 1.62. Вход процесса в состояние ожидания ввода/вывода

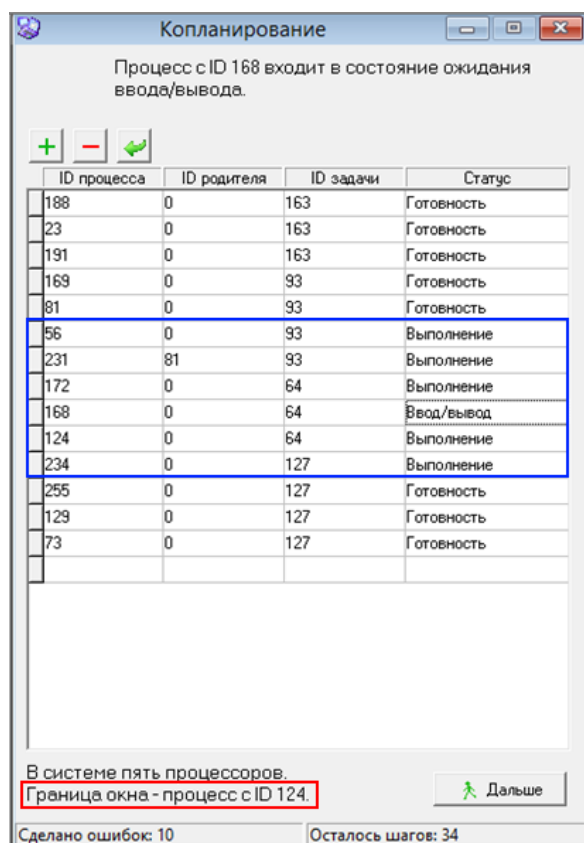


Рис. 1.63. Расширение окна планировщика

5) Конец кванта времени (окно находится около конца очереди)

В алгоритме копланирования процессы выполняются циклически, поэтому окно планировщика будет располагаться так, как изображено на рис. 1.64 и рис. 1.65.

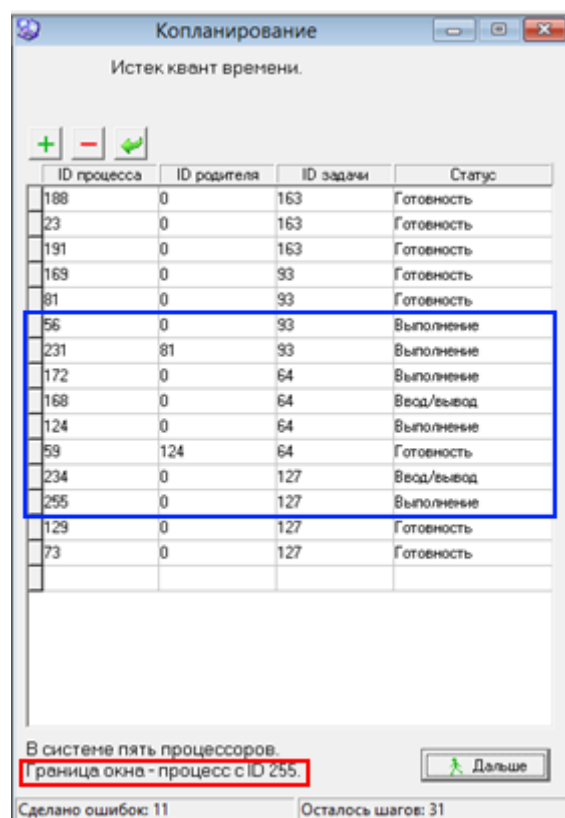


Рис. 1.64. Конец кванта времени

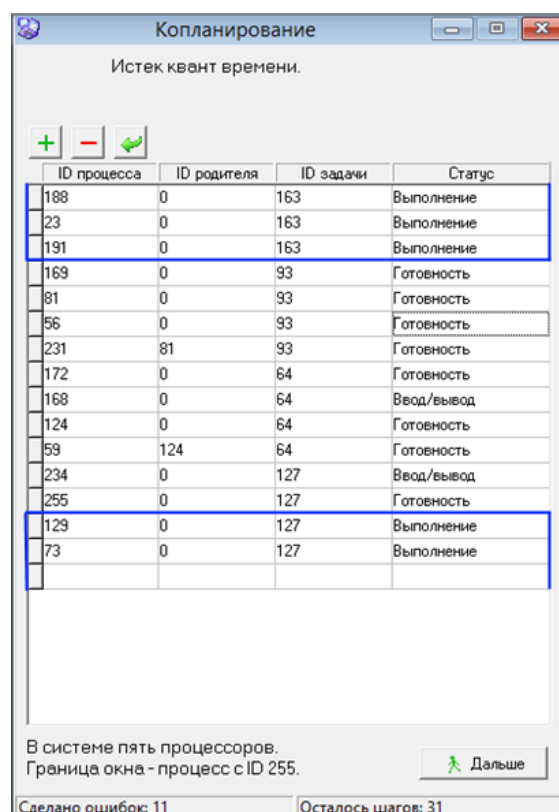


Рис. 1.65. Смещение окна планировщика

6) Снятие задачи с выполнения

Чтобы снять задачу с выполнения, необходимо удалить все процессы, относящиеся к задаче (рис. 1.66, 1.67).

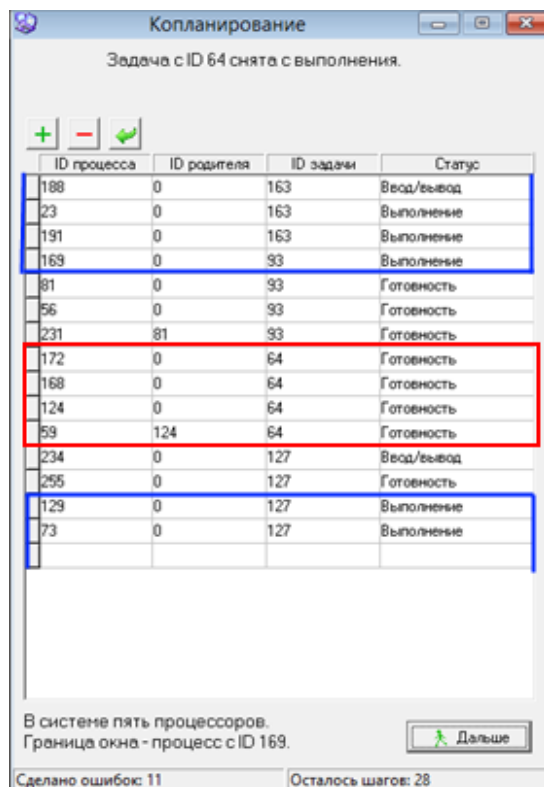


Рис. 1.66. Вид списка процессов до удаления задачи

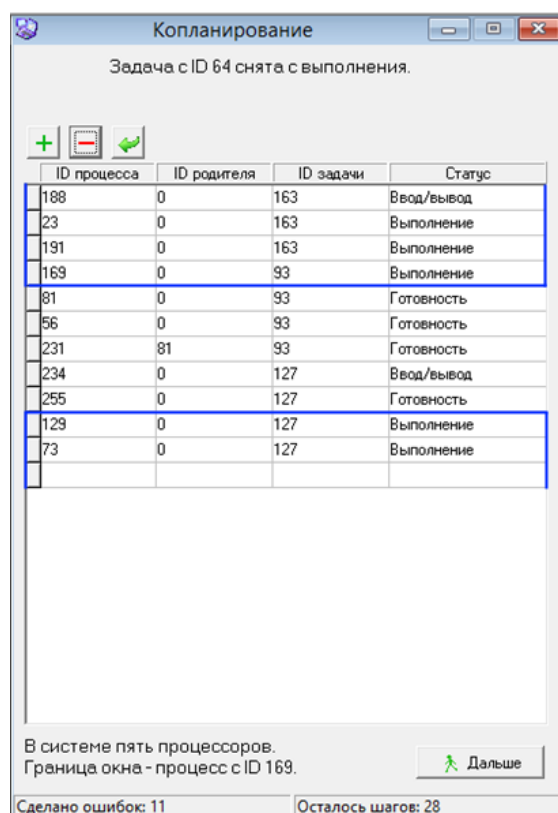


Рис. 1.67. Вид списка процессов после удаления задачи

7) Появление новой задачи с последующим концом кванта времени

В ходе выполнения лабораторной работы возможна такая ситуация, когда окно планировщика и новые процессы располагаются как на рис. 1.68. Выбирая новую позицию окна планировщика, необходимо ориентироваться по строке «Граница окна» (рис. 1.69).

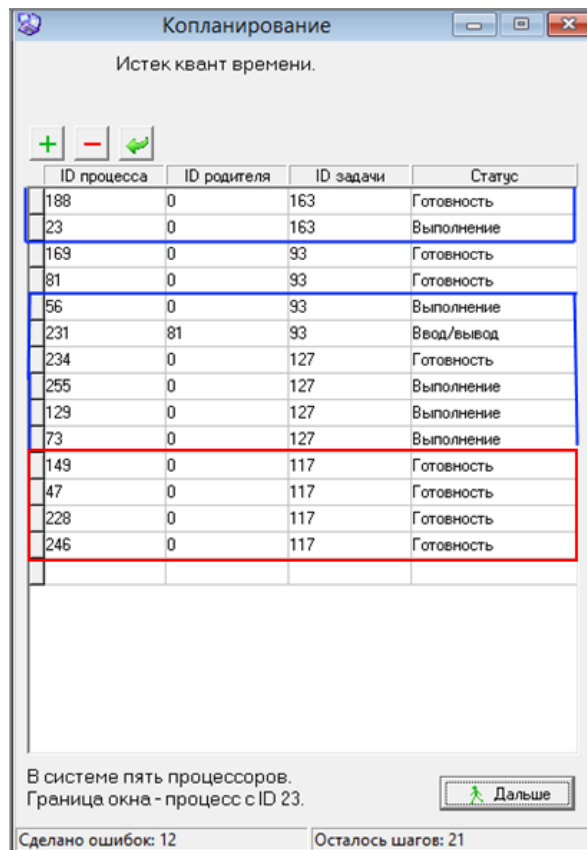


Рис. 1.68. Конец кванта времени

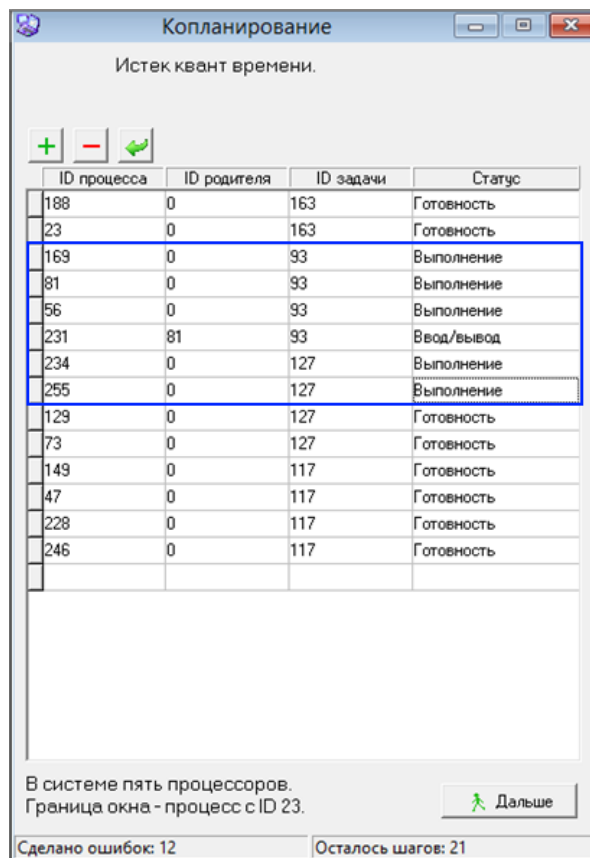


Рис. 1.69. Смещение окна планировщика

1.3.4. Алгоритм динамического разделения

Теоретическая часть

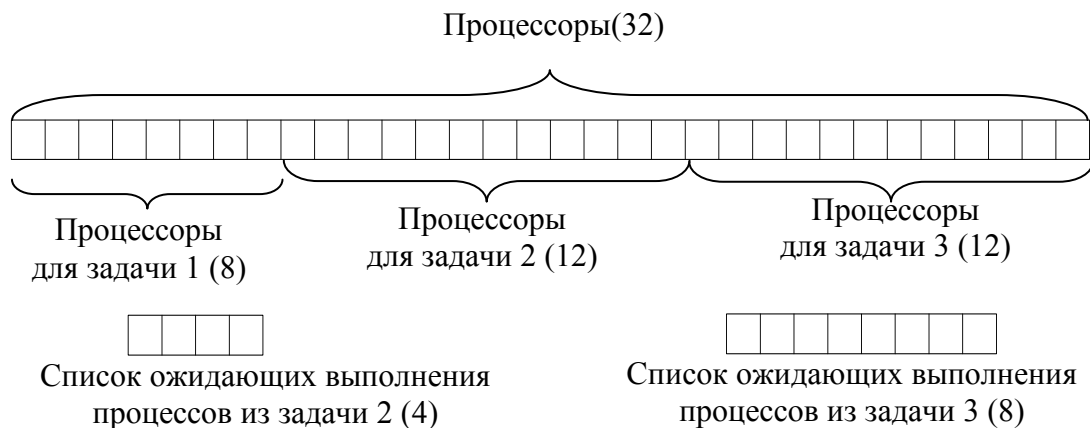
Динамическое разделение (dynamic partitioning) сводит к минимуму потери производительности вследствие промахов в кэше, пытаясь поддерживать максимальную привязку процессов к процессорам. Планировщик поровну распределяет процессоры системы между задачами. Количество процессоров, которое достанется каждой задаче, всегда меньше или равно количеству доступных для выполнения процессов в этой задаче.

В качестве примера можно рассмотреть систему с 32 процессорами, в которой должно выполняться три задачи – первая с 8 процессами, доступными для выполнения, вторая – с 16 процессами, а третья с 20. Если бы планировщик поровну разделял процессоры между задачами, одна задача получила бы 10 процессоров, а две остальные – по 11 каждая. В рассматриваемом примере в первой задаче только 8 процессов, поэтому планировщик выделит ей 8 процессоров и поровну разделит оставшиеся 24 процессора между двумя оставшимися задачами (по 12 процессоров каждой) (пример 1 на Рис. 1.70). Поэтому каждая задача будет всегда выполняться на одних и тех же процессорах, если только в системе не появятся новые задачи. Этот алгоритм можно усовершенствовать так, что каждый процесс будет всегда выполняться на одном и том же процессоре. Если в каждой задаче содержится по одному процессу, то динамическое разделение превратится в обычный круговой алгоритм планирования.

Если в систему поступают новые задачи, система динамически изменяет распределение процессоров. Предположим, что в систему поступила четвертая задача, включающая 10 доступных для выполнения процессов (пример 2 на Рис. 1.70). Первой задаче, как и раньше, выделяется 8 процессоров, но вторая и третья задачи передают часть своих

процессоров четвертой задаче. Процессоры будут поровну распределены между задачами – по 8 процессоров каждой задаче. Алгоритм будет изменять выделение процессоров для каждой задачи каждый раз, когда начинается или завершается выполнение задачи, или процесс в задаче будет изменять свое состояние – из выполнения в ожидание и обратно. Хотя количество процессоров, достающихся задаче, изменяется, задача все равно выполняется на подмножестве или надмножестве процессоров, доставшихся ей изначально, и привязка к процессорам улучшается. Чтобы динамическое разделение было эффективным, прирост производительности от повышения уровня попаданий в кэш должен перевесить падение производительности от перераспределения.

Пример 1: Три задачи



Пример 2: Четыре задачи

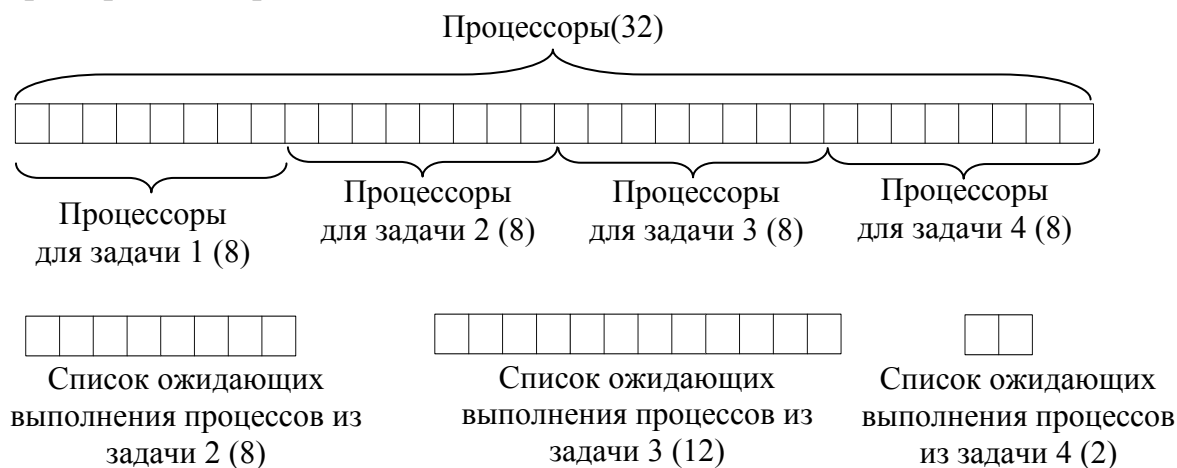


Рис. 1.70. Динамическое разделение процессоров

Описание лабораторной установки

Интерфейс окна для алгоритма динамического разделения представлен на рис. 1.71.

Группа кнопок (1) предназначена для добавления задачи в таблицу задач, удаления выделенной строки в таблице, а также сброса всех значений в начальное состояние (на момент последнего успешного шага).

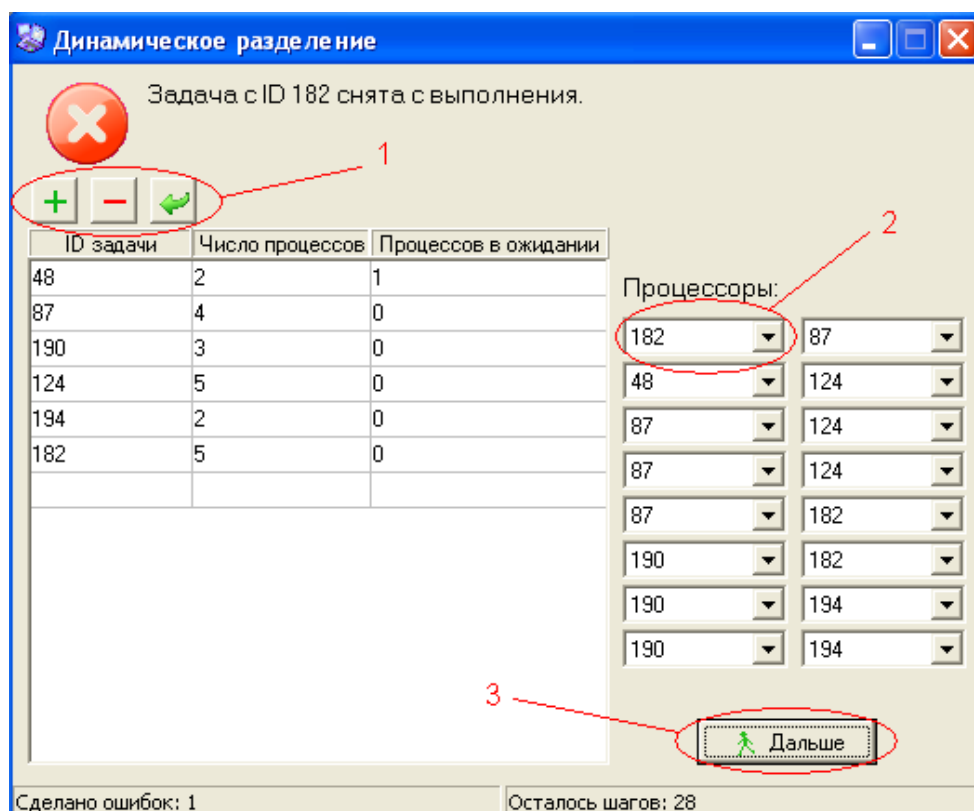


Рис. 1.71. Интерфейс окна для алгоритма динамического разделения

Группа кнопок «Процессоры» (2) позволяет сопоставить каждый процессор одной из задач.

Кнопка (3) осуществляет переход к следующему событию.

Таблица задач содержит три поля: «ID задачи» – идентификатор задачи, «Число процессов» – общее число процессов задачи, «Процессов в ожидании» – число процессов задачи, находящихся в состоянии «ввода/вывода».

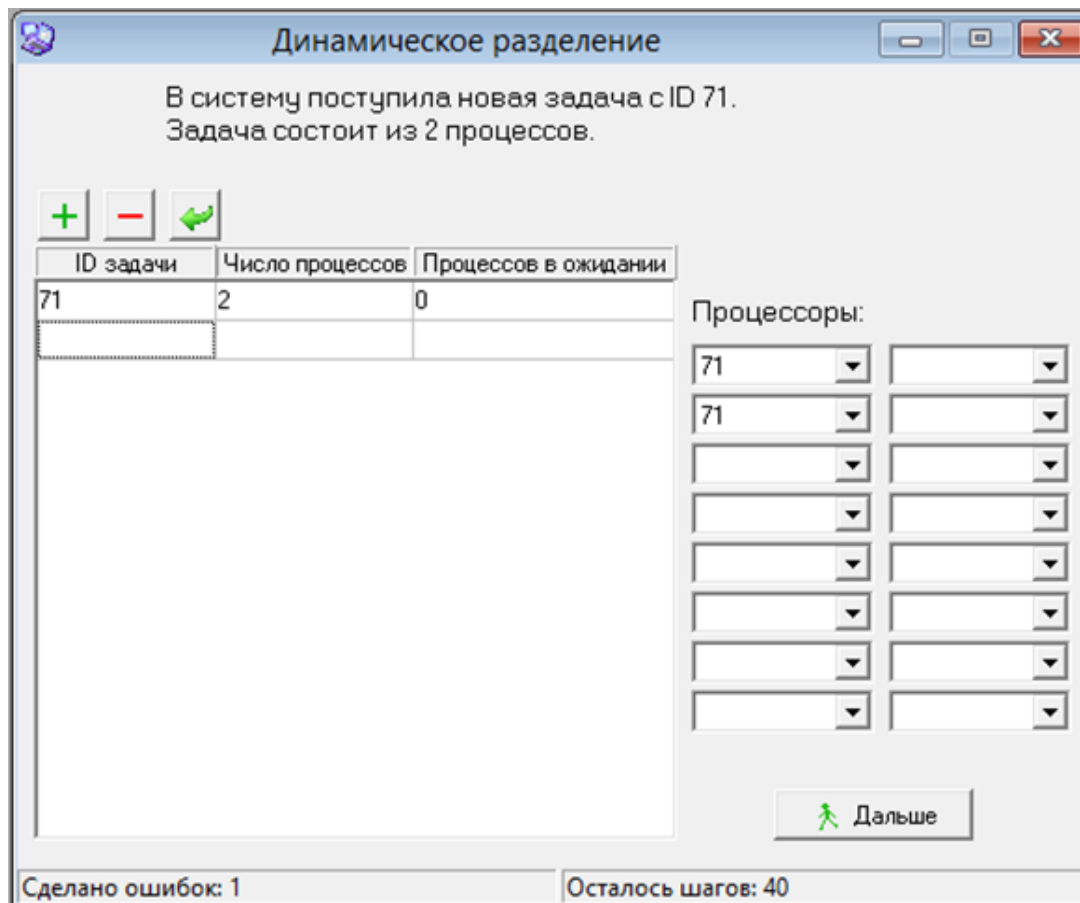
Выполнение лабораторной работы

Процессоры делятся поровну между задачами. «Лишние» процессоры отдаются задачам, поступившим в систему раньше.

Новое множество процессоров задачи должно быть надмножеством или подмножеством старого.

Выполнение шагов:

Если в систему поступают новые задачи, система динамически изменяет распределение процессоров. Примеры приведены на рис. 1.72- 1.89.



ID задачи	Число процессов	Процессов в ожидании
71	2	0

Процессоры:

71	
71	

Дальше

Сделано ошибок: 1

Осталось шагов: 40

Рис. 1.72. Добавление новой задачи

Динамическое разделение

Один из процессов задачи с ID 71 завершил работу.

ID задачи	Число процессов	Процессов в ожидании
71	1	0

Процессоры:

71	

Сделано ошибок: 1 Осталось шагов: 39

Рис. 1.73. Завершение работы процесса

Динамическое разделение

В систему поступила новая задача с ID 32.
Задача состоит из 5 процессов.

ID задачи	Число процессов	Процессов в ожидании
71	1	0
32	5	0

Процессоры:

71	
32	
32	
32	
32	
32	

Сделано ошибок: 1 Осталось шагов: 38

Рис. 1.74. Поступление новой задачи

Динамическое разделение

В систему поступила новая задача с ID 52.
Задача состоит из 4 процессов.

+ - ↩

ID задачи	Число процессов	Процессов в ожидании
71	1	0
32	5	0
52	4	0

Процессоры:

71	52
32	52
32	
32	
32	
32	
52	
52	

Дальше

Сделано ошибок: 1 Осталось шагов: 37

Рис. 1.75. Поступление задачи с ID 52

Динамическое разделение

В систему поступила новая задача с ID 143.
Задача состоит из 3 процессов.

+ - ↩

ID задачи	Число процессов	Процессов в ожидании
32	5	0
52	4	0
63	4	0
143	3	0

Процессоры:

63	52
32	52
32	63
32	63
32	63
32	143
52	143
52	143

Дальше

Сделано ошибок: 2 Осталось шагов: 34

Рис. 1.76. Поступление задачи с ID 143

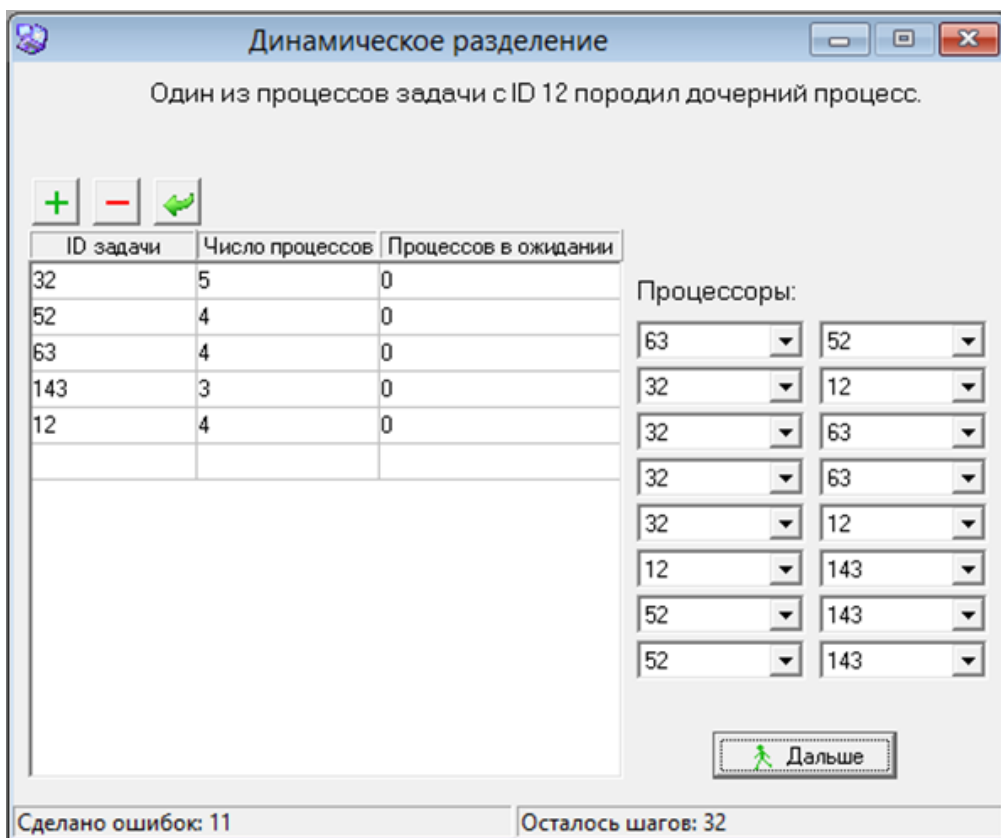


Рис. 1.77. Порождение дочернего процесса

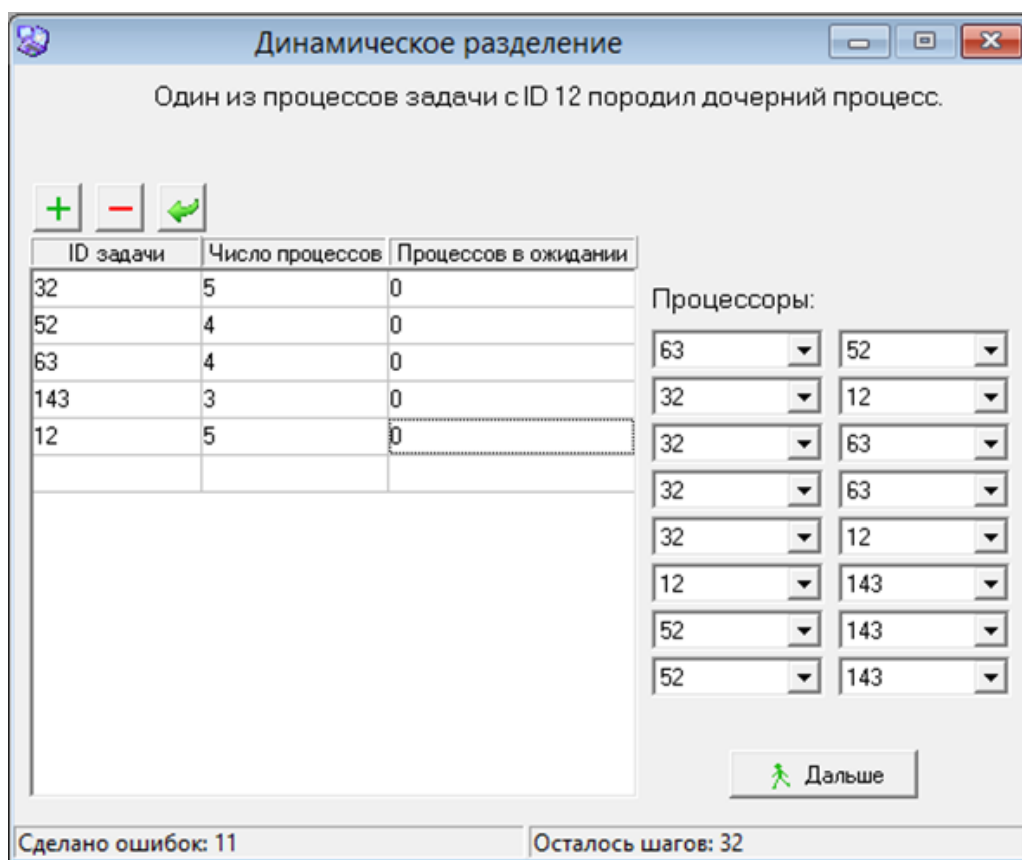


Рис. 1.78. Порождение дочернего процесса задачи с ID 12

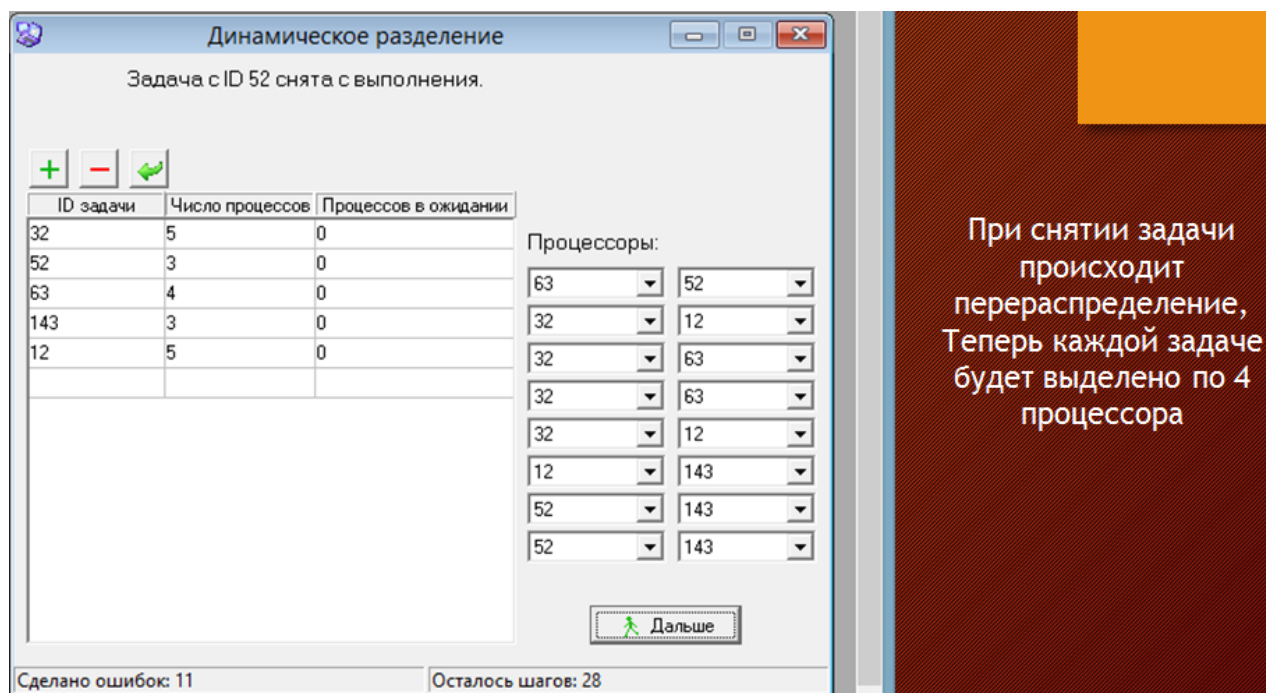


Рис. 1.79. Перераспределение процессоров

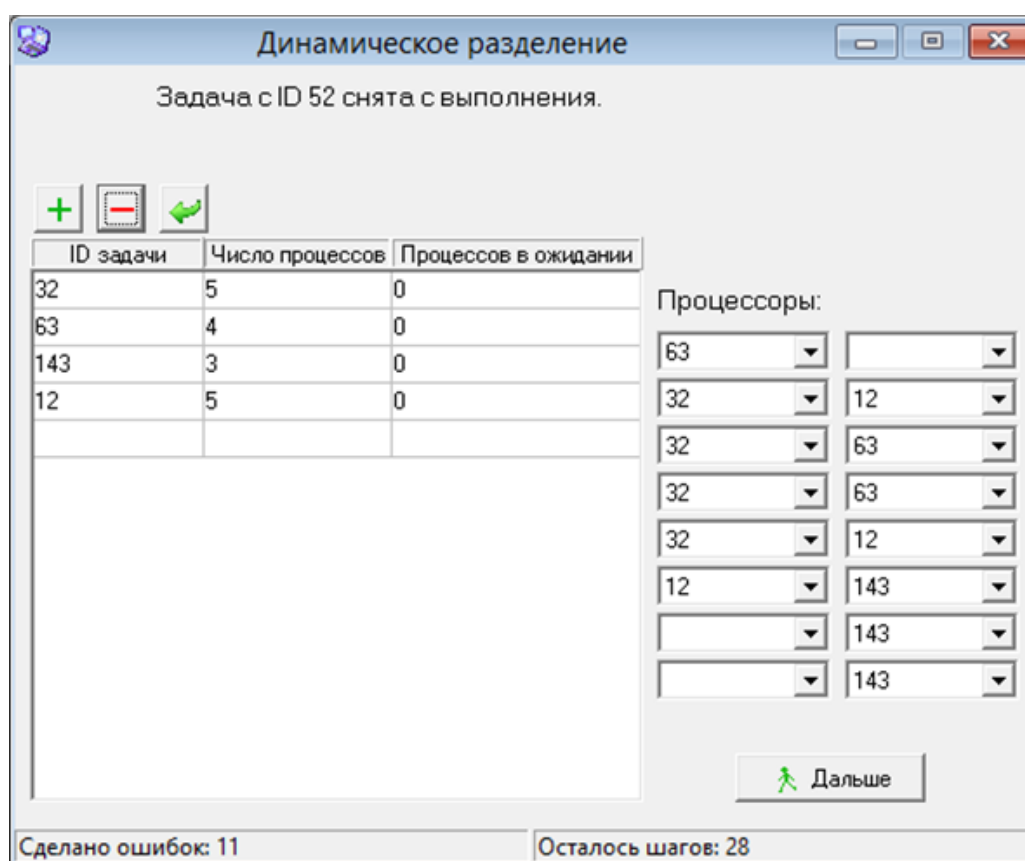
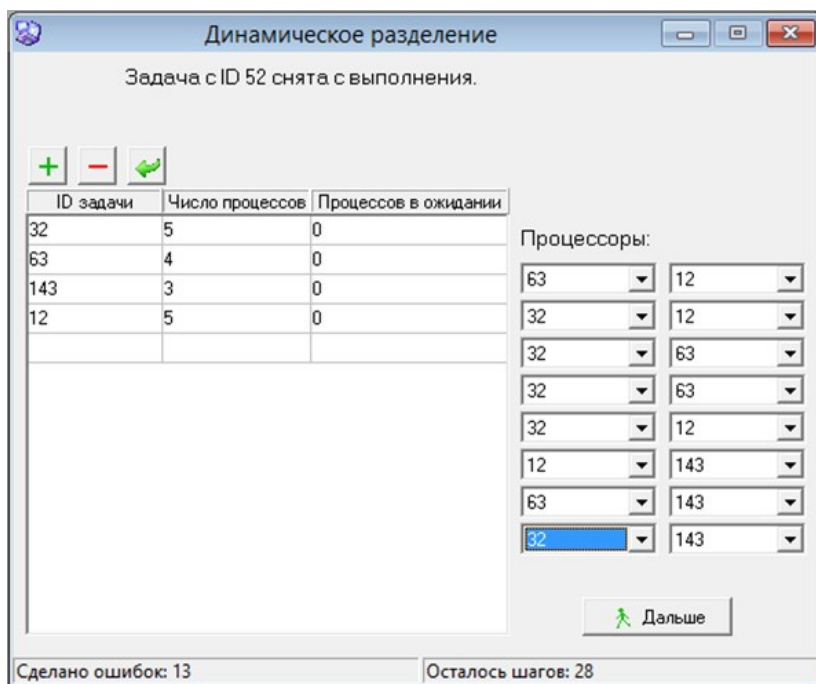


Рис. 1.80. После снятия задачи с выполнения



Так как задаче 143 необходимо только 3 процессора, то 1 «Лишний» достается 32 задаче

Рис. 1.81. Перераспределение процессоров

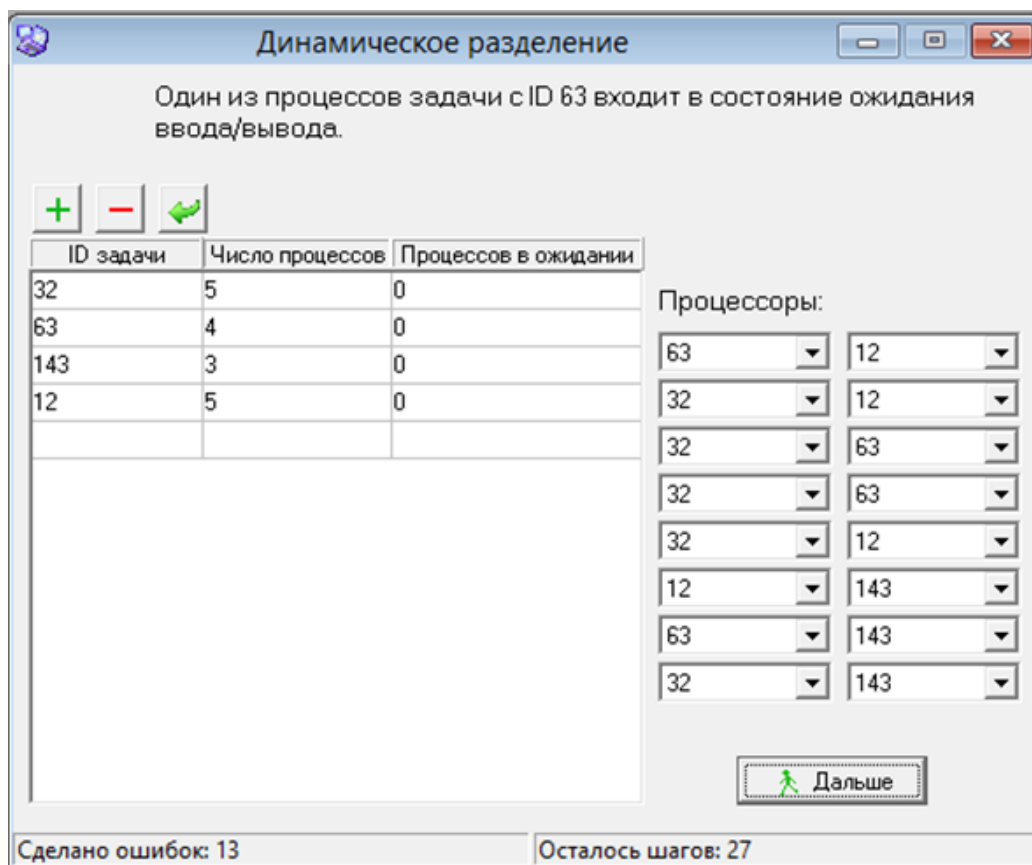


Рис. 1.82. Вход процесса в ожидание ввода/вывода

Динамическое разделение

Один из процессов задачи с ID 63 входит в состояние ожидания ввода/вывода.

ID задачи	Число процессов	Процессов в ожидании
32	5	0
63	4	1
143	3	0
12	5	0

Процессоры:

63	12
32	12
32	63
32	63
32	12
12	143
12	143
32	143

Дальше

Сделано ошибок: 14 Осталось шагов: 27

Освобожденный
процессор №63
достаётся задаче №12
так как больше никому
процессоров не нужно

Рис. 1.83. Перераспределение процессов

Динамическое разделение

Один из процессов задачи с ID 63 завершил работу.

ID задачи	Число процессов	Процессов в ожидании
32	5	0
63	3	1
143	3	0
12	5	0

Процессоры:

63	12
32	12
32	63
32	63
32	12
12	143
12	143
32	143

Дальше

Сделано ошибок: 14 Осталось шагов: 26

Рис. 1.84. Завершение работы процесса

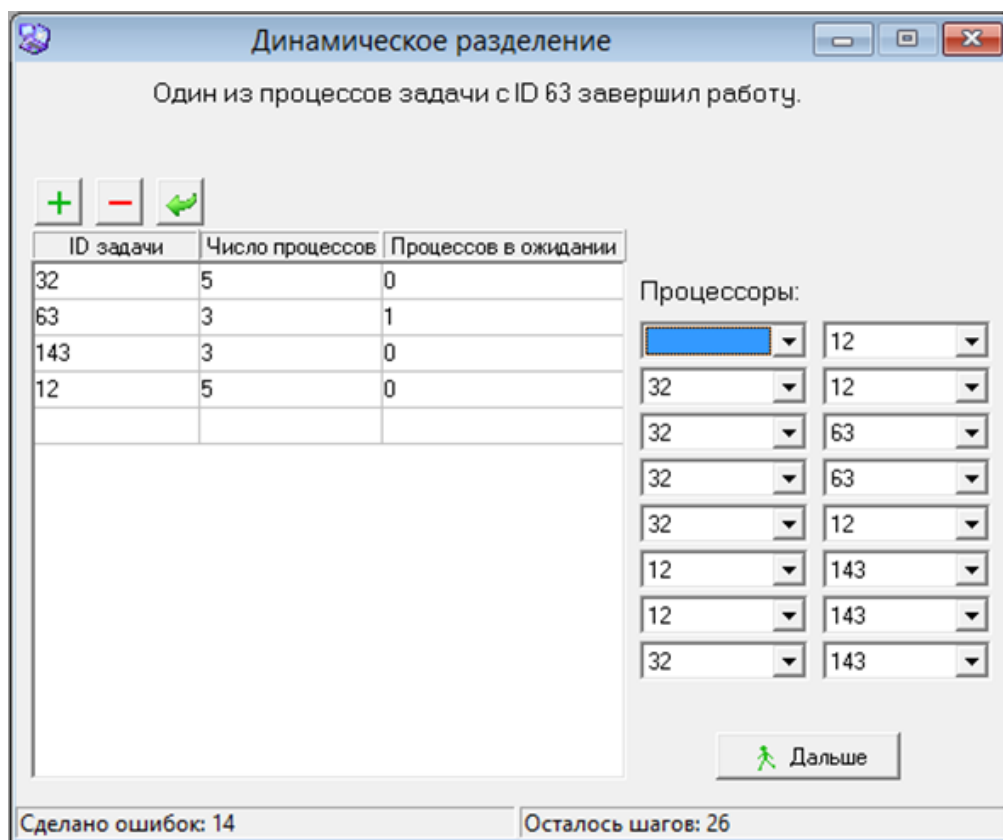


Рис. 1.85. Распределение процессоров

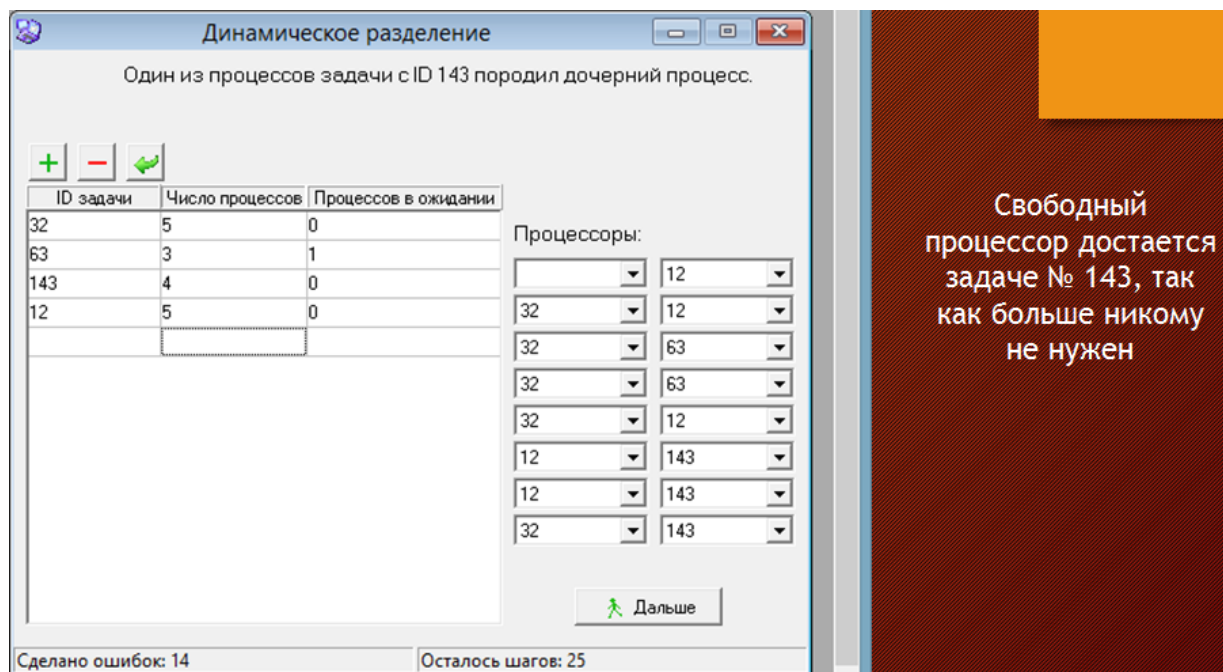


Рис. 1.86. Создание дочернего процесса

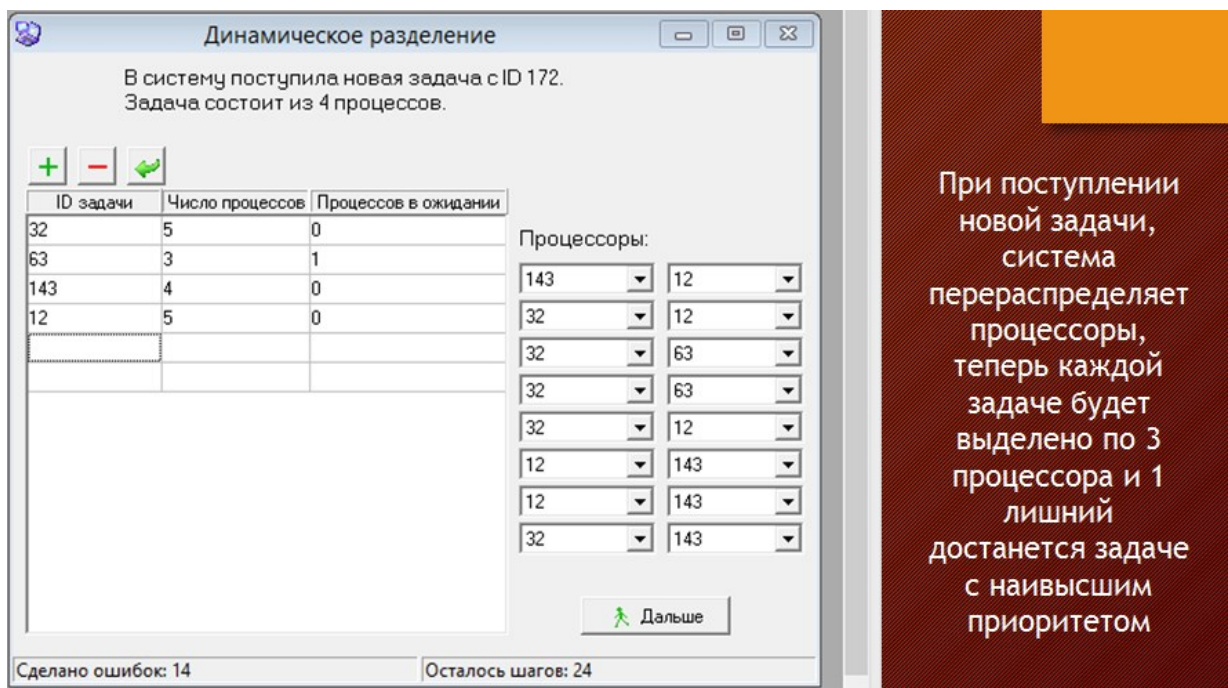


Рис. 1.87. Поступление новой задачи

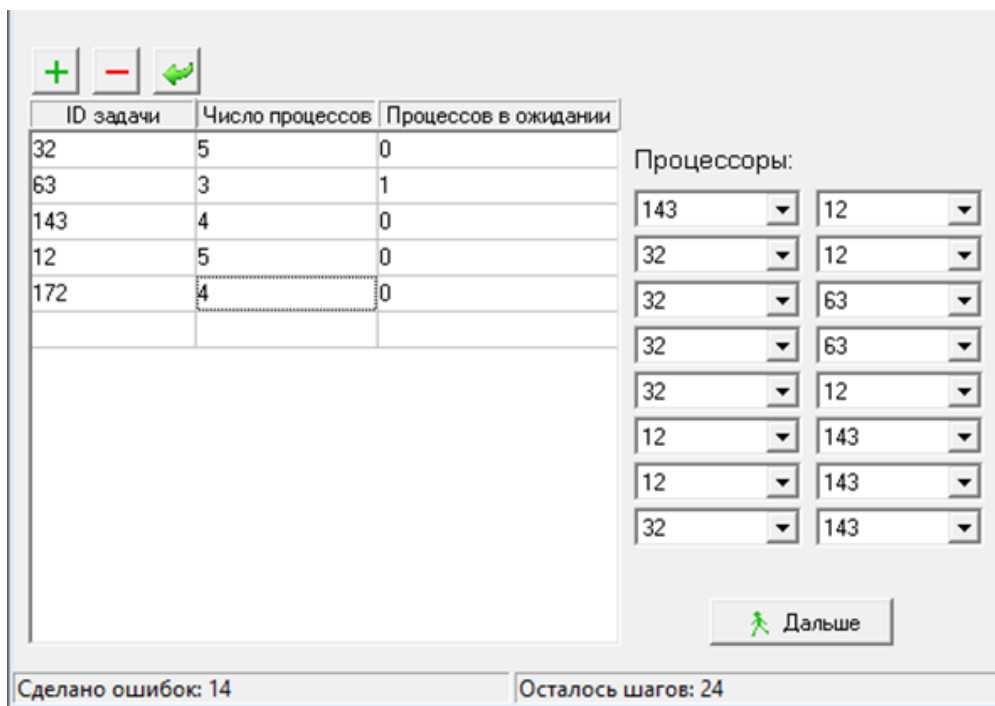


Рис. 1.88. Создание новой задачи

Динамическое разделение

В систему поступила новая задача с ID 91.
Задача состоит из 3 процессов.

ID задачи	Число процессов	Процессов в ожидании
32	5	0
63	4	0
143	4	1
172	3	0
91	3	0

Процессоры:

91	91
32	172
32	63
32	63
32	172
91	143
63	143
172	143

Сделано ошибок: 42 Осталось шагов: 15

Рис. 1.89. Перераспределение процессоров

Результат выполнения лабораторной работы

Результат выполнения лабораторной работы

Выполнение лабораторной работы заканчивается автоматической генерацией отчета. В отчете показан результат теста, время выполнения каждого алгоритма и число допущенных ошибок. ПО количеству допущенных ошибок можно определить степень готовности студента к выполнению лабораторной работе. Большое количество ошибок может говорить о том, что выполнение работы основано не на знаниях, на подборе верного ответа. На рис. 1.90 представлен незаполненный отчет перед выполнением лабораторной работы.

Отчет
<i>Чистяков</i>
<i>ВМ-43</i>
<i>Результат теста: не пройден</i>
<i>Задачно-независимые алгоритмы:</i>
<i>Алгоритм RR</i> <i>Время выполнения: не выполнен</i> <i>Число ошибок: не выполнен</i>
<i>Алгоритм SPF</i> <i>Время выполнения: не выполнен</i> <i>Число ошибок: не выполнен</i>
<i>Задачно-ориентированные алгоритмы:</i>
<i>Алгоритм SNPF</i> <i>Время выполнения: не выполнен</i> <i>Число ошибок: не выполнен</i>
<i>Алгоритм Копланирование</i> <i>Время выполнения: не выполнен</i> <i>Число ошибок: не выполнен</i>

Рис. 1.90. Отчет перед выполнением лабораторной работы

2. Синхронизация в распределённых системах

Распределенная система — это набор независимых компьютеров, представляющий их пользователям единой объединенной системой. В этом определении оговариваются два момента:

- первый относится к аппаратуре: все машины автономны;
- второй касается программного обеспечения: пользователи думают, что имеют дело с единой системой.

Распределенные системы состоят из автономных компьютеров, которые работают совместно, представляя в виде единой связной системы. Их важное преимущество состоит в том, что они упрощают интеграцию различных приложений, работающих на разных компьютерах, в единую систему. Их размер ограничивается только размером базовой сети. Платой за эти преимущества часто является очень сложное программное обеспечение, падение производительности и особенно проблемы с безопасностью. В процессе работы распределённой системы, могут возникнуть следующие конфликтные ситуации:

- несколько процессов не способны одновременно получать доступ к совместно используемым ресурсам, та-ким как принтеры, а должны взаимодействовать, позволяя друг другу временно получать эксклюзивный доступ к этому ресурсу;
- несколько процессов могут время от времени нуждаться в соглашениях о порядке прохождения сообщений, о том, например, что сообщение m_1 от процесса P будет отправлено раньше, чем сообщение m_2 от процесса Q .

С взаимодействием между процессами тесно связан вопрос *синхронизации процессов в распределенных системах*.

Синхронизация в распределенных системах часто значительно сложнее, чем синхронизация в однопроцессорных или

мультипроцессорных системах. В однопроцессорных системах решение задач взаимного исключения, критических областей и других проблем синхронизации осуществлялось с использованием общих методов, таких как семафоры и мониторы. Однако эти методы не совсем подходят для распределенных систем, так как все они базируются на использовании разделяемой оперативной памяти. Например, два процесса, которые взаимодействуют, используя семафор, должны иметь доступ к нему. Если оба процесса выполняются на одной и той же машине, они могут иметь совместный доступ к семафору, хранящемуся, например, в ядре, делая системные вызовы. Однако, если процессы выполняются на разных машинах, то этот метод не применим, для распределенных систем нужны новые подходы.

Проблема распределенных систем и вообще компьютерных сетей состоит в том, что для них не существует понятия единых совместно используемых часов.

Для этого существуют различные способы *синхронизации часов* в распределенных системах, но все они основаны на обмене показаниями часов, а это требует учитывать задержки на посылку и получение сообщений. Точность алгоритмов синхронизации в значительной мере определяют вариации в задержках доступа и способы учета этих вариаций (алгоритмы *Кристиана* и *Беркли*).

Во многих случаях знания абсолютного времени не требуется. Достаточно, чтобы соответствующие события в различных процессах происходили в правильной последовательности (алгоритм Лампорта "*логические часы*").

Синхронизация между процессами часто требует, чтобы один из процессов выступал в роли координатора. В этом случае если координатор не фиксирован, необходимо, чтобы процессы в распределенных вычислениях могли решить, который из них будет координатором.

Это решение принимается посредством *алгоритмов голосования*. Алгоритмы голосования, как правило, задействуются при сбое или отключении существовавшего координатора.

Важный класс алгоритмов синхронизации – *распределенные взаимные исключения*. Эти алгоритмы гарантируют, что в распределенном наборе процессов доступ к совместно используемым ресурсам имеет максимум один процесс. Распределенные взаимные исключения можно легко обеспечить с помощью координатора, который будет следить, чья сейчас очередь. Существуют также и полностью распределенные алгоритмы, но их минус – чересчур большая чувствительность к сбоям процессов и связи.

С взаимными исключениями тесно связаны *распределенные транзакции*. Транзакция состоит из набора операций с совместно используемыми данными, при этом вся транзакция целиком либо полностью выполняется, либо полностью не выполняется. Кроме того, несколько транзакций могут выполняться одновременно, при этом результат оказывается таким, как если бы эти транзакции выполнялись в некоторой заданной очередности. И наконец, транзакции долговечны, в том смысле, что после завершения их результат неизменен.

2.1. Алгоритмы синхронизации часов

Если одна машина имеет приемник WWV, нашей задачей является поддержание синхронизации с ней всех остальных машин. Если приемников WWV нет ни на одной из машин, то каждая из них отсчитывает свое собственное время, и нашей задачей будет по возможности синхронизировать их между собой.

Все алгоритмы имеют одну базовую модель системы. Считается, что каждая машина имеет таймер, который инициирует прерывание N раз в секунду. Когда этот таймер срабатывает, обработчик прерываний

добавляет единицу к программным часам, которые сохраняют число тиков (прерываний), начиная с какого-либо момента в прошлом, о котором была предварительная договоренность.

Если разработчики операционной системы хотят гарантировать, что никакая пара часов не сможет разойтись более чем на q , синхронизация часов должна производиться не реже, чем каждые $q/2p$ сек. Различные алгоритмы отличаются точностью определения момента проведения повторной синхронизации.

2.1.1. Алгоритм Кристиана

Теоретическая часть

Алгоритм основан на работах Кристиана (Cristian) и более ранних.

Периодически, гарантировано не реже, чем каждые $q/2p$ сек, каждая машина посылает серверу времени сообщение, запрашивая текущее время. Эта машина так быстро, как это возможно, отвечает сообщением, содержащим ее текущее время.



Рис. 2.1. Получение текущего времени с сервера времени

В качестве первого приближения, когда отправитель получает ответ, он может просто выставить свои часы в значение C_{UTC} . Однако такой алгоритм имеет две проблемы: главную и второстепенную.

Главная проблема состоит в том, что время никогда не течет назад. Если часы отправителя спешат. C_{utc} может оказаться меньше текущего значения C у отправителя. Простая подстановка C_{utc} способна вызвать серьезные проблемы, связанные с тем, что объектные файлы, скомпилированные после того, как было изменено время, помечены временем более ранним, чем модифицированные исходные тексты, которые поправлялись до изменения времени.

Изменения могут вноситься постепенно. Один из способов таков. Предположим, что таймер настроен так, что он генерирует 100 прерываний в секунду. В нормальном состоянии каждое прерывание будет добавлять ко времени по 10 мс.

При запаздывании часов процедура прерывания будет добавлять каждый раз всего по 9 мс. Соответственно, часы должны быть исправлены так, чтобы добавлять при каждом прерывании 10 мс вместо того, чтобы переключать все время одновременно.

Менее серьезная проблема состоит в том, что перенос ответного сообщения с сервера времени отправителю требует ненулевого времени. Хуже всего, что эта задержка может быть весьма велика и зависеть от загрузки сети. По Кристиану, метод решения проблемы состоит в измерении этой величины. Отправителю достаточно просто аккуратно записать интервал между посылкой запроса и приходом ответа. То и другое время (начальное T_0 и конечное T_1), измеряется по одним и тем же часам, а значит, интервал будет относительно точно измерен даже в том случае, если часы отправителя имеют некоторое расхождение с UTC.

Эта оценка может быть улучшена, если приблизительно известно, сколько времени сервер времени обрабатывает прерывание и работает с пришедшим сообщением. Обозначим время обработки прерывания I . Тогда величина интервала времени между T_0 и T_1 затраченного на прохождение сообщения, будет равна $T_1 - T_0 - I$, и лучшей оценкой

времени на прохождение сообщения в одну сторону будет половина этой величины.

Для повышения точности Кристиан предложил производить не одно измерение, а серию. Все измерения, в которых разность $T1 - T0$ превосходит некоторое пороговое значение, отбрасываются как ставшие жертвами перегруженной сети, а потому недостоверные. Оценка делается по оставшимся замерам, которые могут быть усреднены для получения наилучшего значения. С другой стороны, сообщение, пришедшее быстрее всех, можно рассматривать как самое точное, поскольку оно предположительно попало в момент наименьшего трафика и потому наиболее точно отражает чистое время прохождения.

Описание лабораторной установки

Интерфейс лабораторной установки представлен на рис. 2.2.

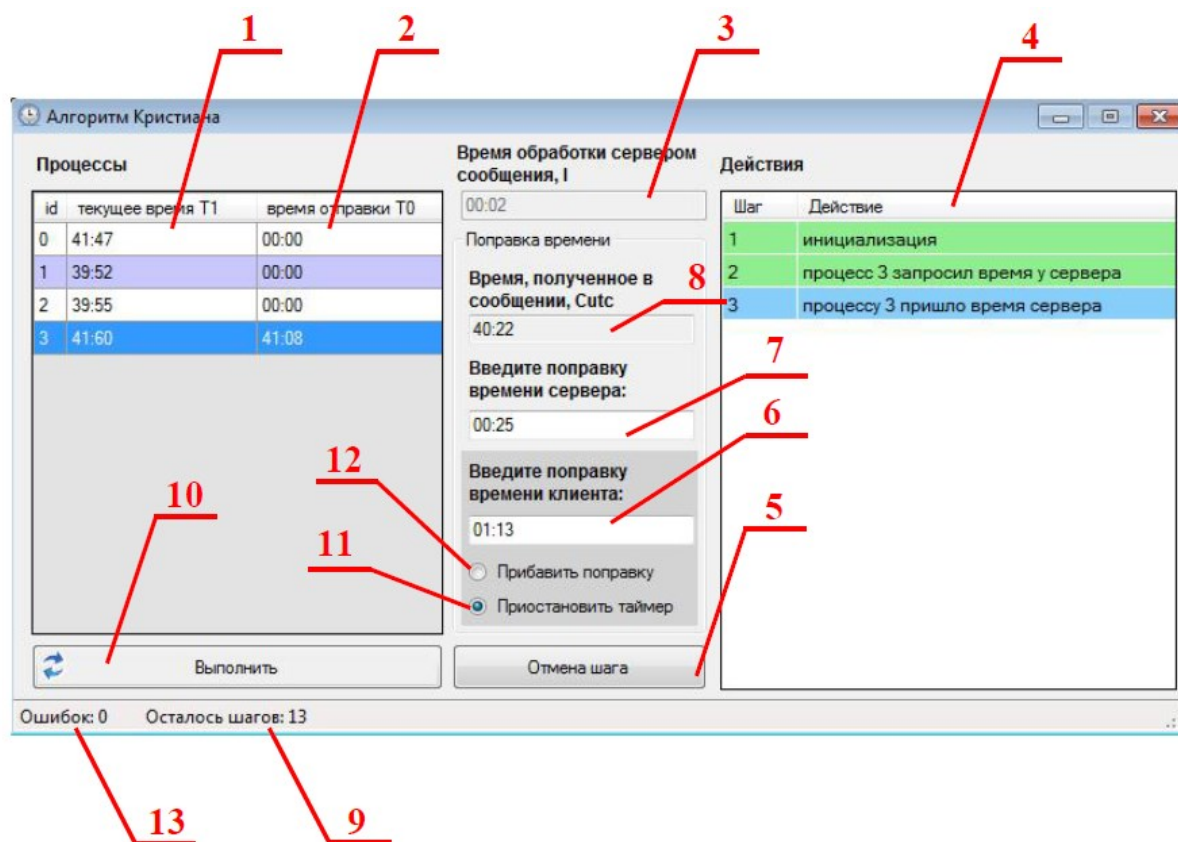


Рис. 2.2. Интерфейс лабораторной установки для алгоритма Кристиана

Обозначения:

Действие, которое необходимо выполнить представлено в таблице (4). Синий цвет обозначает текущий шаг, зелёный – выполненные правильно шаги, красный – шаг выполнен неверно.

После выполнения каждого шага необходимо нажать кнопку «Выполнить» (10).

Для того, чтобы возратить значения времени на начало шага, необходимо нажать кнопку «Отмена шага» (5).

Поле (13) обозначает количество допущенных ошибок, поле (9) – осталось шагов до завершения задания.

Выполнение лабораторной работы

Выполнение шагов:

1) Инициализация.

Чтобы проинициализировать значения времени клиента T_1 необходимо щёлкнуть двойным щелчком мыши в поле (1) и занести значения из задания. В поле (3) необходимо внести время обработки сообщения сервером I из задания.

2) Запрос клиентом времени сервера.

Для сохранения текущего времени клиента T_0 необходимо щёлкнуть двойным щелчком мыши в поле (2) и занести в соответствующее поле значение текущего времени.

3) Получение сообщения о текущем времени сервера.

После получения сообщения о времени сервера, необходимо сосчитать, его поправку (7) $(T_1 - T_0 - I)/2$ (подробнее в описании метода). Причём, если получилось не целое число, дробная часть должна быть отброшена. Следующим шагом является расчёт поправки времени клиента (6), для чего поправка времени сервера (7) складывается с временем сервера в сообщении Cut_s (8) и из неё вычитается текущее время

клиента T1. Если результат отрицательный таймер необходимо приостановить (11), иначе прибавить поправку (12).

2.1.2. Алгоритм Беркли

Теоретическая часть

В алгоритме Кристиана сервер времени пассивен. Прочие машины периодически запрашивают у него время. Все, что он делает, – это отвечает на запросы.

В операционной системе UNIX разработки университета Беркли (Berkeley) принят прямо противоположный подход. Здесь сервер времени (демон времени) активен, он время от времени опрашивает каждую из машин, какое время на ее часах. На основании ответов он вычисляет среднее время и предлагает всем машинам установить их часы на новое время или замедлить часы, пока не будет достигнуто необходимое уменьшение значения времени на сильно ушедших вперед часах. Этот метод применим для систем, не имеющих машин с приемником WWV. Время демона может периодически выставляться вручную оператором. Это продемонстрировано на рис. 2.3.

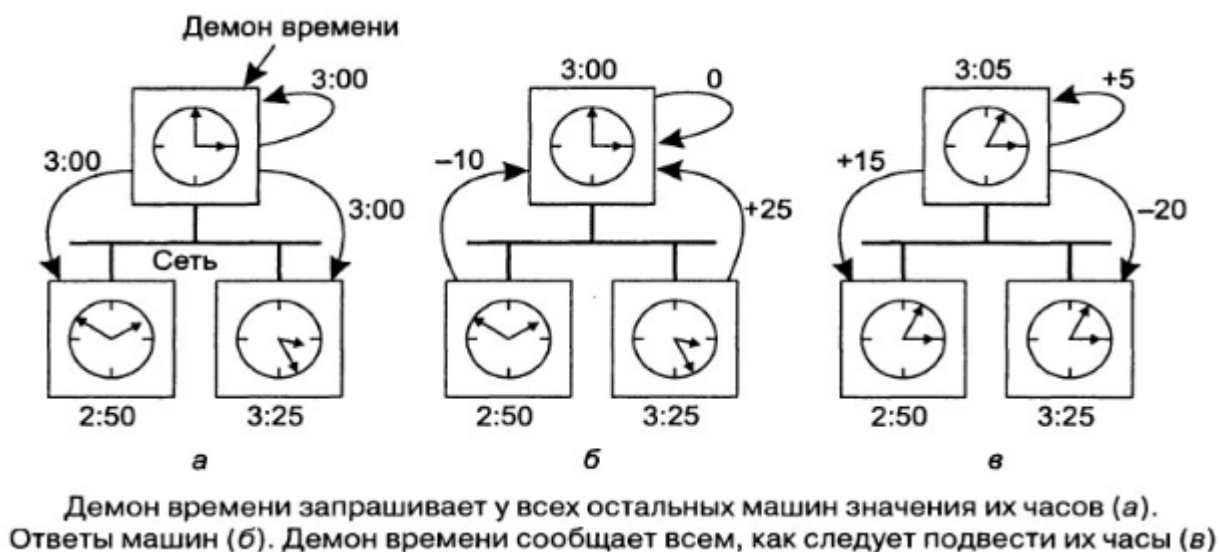


Рис. 2.3. Работа демона времени

В 3:00 демон времени сообщает всем машинам свое время и запрашивает эти машины об их времени (рис. 2.3 а). Затем демон времени получает ответ о том, насколько спешат или отстают часы машин от часов демона (рис. 2.3 б). Зная эти числа, демон времени вычисляет среднее и сообщает каждой из машин, как ей следует подвести свои часы (рис. 2.3 в).

2.1.3. Логические часы

Теоретическая часть

В своей классической статье Лампорт (Lamport) показал, что хотя синхронизация часов возможна, она не обязательно должна быть абсолютной.

Если два процесса не взаимодействуют, нет необходимости в том, чтобы их часы были синхронизированы, поскольку отсутствие синхронизации останется незамеченным и не создаст проблем. Кроме того, он указал, что обычно имеет значение не точное время выполнения процессов, а его порядок. Для синхронизации логических часов Лампорт определил отношение под названием «происходит раньше». Выражение $a > b$ читается как «а происходит раньше b» и означает, что все процессы согласны с тем, что первым происходит событие а, а позже – событие b. Отношение «происходит раньше» непосредственно выполняется в двух случаях:

- Если а и b – события, происходящие в одном и том же процессе, и а происходит раньше, чем b, то отношение $a > b$ истинно;
- Если а – это событие отсылки сообщения одним процессом, а b – событие получения того же сообщения другим процессом, то отношение $a > b$ так-же истинно. Сообщение не может быть получено до отсылки или даже в тот же самый момент, когда оно было послано, поскольку для пересылки необходимо конечное ненулевое время.

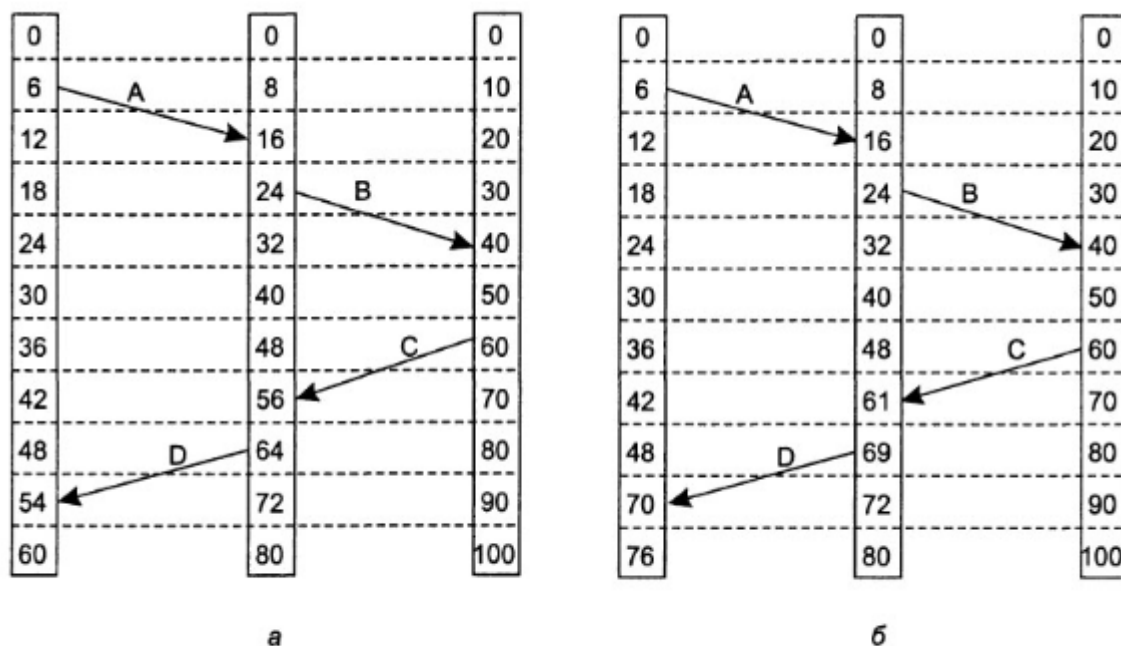
Отношение «происходит раньше» – это транзитивное отношение, то есть в случае, если $a \rightarrow b$ и $b \rightarrow c$, выполняется условие $a \rightarrow c$. Если два события, x и y , происходят в разных процессах, которые не обмениваются сообщениями (ни непосредственно, ни через третий процесс), то отношение $x \rightarrow y$ не истинно, впрочем, как и $y \rightarrow x$. Такие события называются параллельными (concurrent). В данном случае это означает только то, что никто не может (или не хочет) знать о том, где и какое из этих событий произошло.

Соответствие каждому событию a отметки времени $C(a)$. Эти отметки времени должны быть такими, чтобы при $a \rightarrow b$ соблюдалось соотношение $C(a) < C(b)$. Перефразируя условие, которое было установлено ранее, если a и b – два события одного процесса и a происходит раньше, чем b , то $C(a) < C(b)$. Например, если a – это посылка сообщения одним процессом, а b – прием этого сообщения другим процессом, то $C(a)$ и $C(b)$ должны быть назначены так, чтобы значения $C(a)$ и $C(b)$ соответствовали отношению $C(a) < C(b)$. Кроме того, время по часам C , всегда идет вперед (увеличивается), а назад – никогда (не уменьшается). Коррекция времени должна производиться только путем добавления к нему положительного значения, а не его вычитанием.

Для всех различимых событий a и b выполняется соотношение:

$$C(a) \neq C(b).$$

Рассмотрим три процесса, описанные на рис. 2.4. Процессы запущены на различных машинах, каждая из которых имеет собственные часы и скорость работы. Как можно увидеть по рисунку, когда часы процесса 0 поставят отметку времени 6, в процессе 1 они покажут 8, а в процессе 2 – 10. Каждые часы идут с постоянной скоростью, но эти скорости различны из-за разницы в кристаллах.



Три процесса, каждый с собственными часами, которые ходят с разной скоростью (а). Подстройка часов по алгоритму Лампорта (б)

Рис. 2.4. Пример подстройки часов

В момент 6 процесс 0 посылает сообщение А процессу 1. Сколько времени это сообщение проведет в пути, зависит от того, каким часам мы верим. В любом случае, когда оно придет в процесс 1, его часы будут показывать 16. Если сообщение будет содержать время отправки, 6, процесс 1 сочтет, что на пересылку ушло 10 тиков. Это значение вполне разумно. В соответствии с этими рассуждениями сообщение В от процесса 1 процессу 2 будет доставлено за 16 тиков, это вновь разумное значение.

Сейчас рассмотрим нестандартные ситуации. Сообщение С от процесса 2 процессу 1 будет послано на отметке 60 и достигнет цели на отметке 56. Сообщение D от процесса 1 процессу 0 будет послано на отметке 64 и достигнет цели на отметке 54. Эти цифры абсолютно невозможны. Это та ситуация, которую мы должны предотвратить.

Решение, найденное Лампортом, прямо вытекает из отношения «происходит раньше». Поскольку С посылается на отметке 60, оно должно

достичь цели на отметке 61 или позднее. Поэтому каждое сообщение содержит время отправки по часам отправителя. Когда сообщение достигает цели, но часы получателя показывают время, более раннее чем время отправки, получатель быстро подводит свои часы так, чтобы они показывали время на единицу большее времени от-правки. На рис. 6 мы видим, что теперь сообщение С приходит на отметке 61. Таким же образом сообщение D приходит теперь на отметке 70.

Этот алгоритм дает нам возможность полностью упорядочить все события в системе. В отличие от этого множество других распределенных алгоритмов для устранения неясностей требуют упорядоченности, так что этот алгоритм широко цитируется в литературе.

Описание лабораторной установки

Интерфейс лабораторной установки представлен на рис. 2.5.

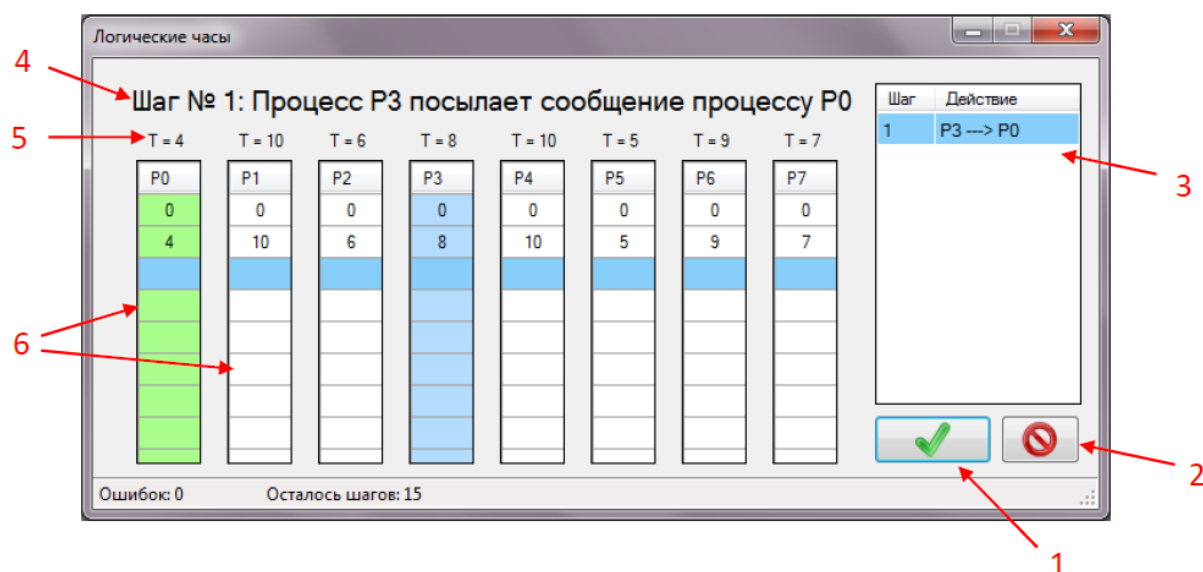


Рис. 2.5. Интерфейс лабораторной установки для логических часов

Обозначения:

1 – кнопка «Выполнить». Проверяется текущий шаг и в случае правильного выполнения осуществляется переход к следующему.

2 – кнопка «Отмена». Позволяет сделать откат изменений на начало текущего шага.

3 – история выполнения алгоритма. Текущий шаг помечен голубым цветом. Если шаг сделан правильно, то строка закрашивается зеленым цветом, в противном случае закрашивается красным.

4 – Сообщение о том, что нужно сделать в текущем шаге.

5 – показывает время такта для каждого процесса.

6 – В алгоритме рассматриваются 8 процессов, которые обмениваются сообщениями. Процесс-отправитель помечен светло-синим, а процесс-получатель светло-зеленым цветом.

Выполнение лабораторной работы:

Выполнение шагов:

Пользователь заполняет значения только для процесса-отправителя и процесса-получателя. Остальные времена в строке заполняются автоматически, при условии что шаг выполнен верно.

Процесс-отправитель посылает сообщение в текущий момент времени, а получателю оно приходит в следующий такт времени (у каждого процесса он свой).

Время отправления должно быть меньше(!) времени получения. В противном случае требуется выполнять синхронизацию. То, что именно нужно сделать, подробно описано в теоретической части методических указаний по лабораторной работе.

2.2. Алгоритмы голосования

Многие распределенные алгоритмы требуют, чтобы один из процессов был координатором, инициатором или выполнял другую специальную роль. Обычно не важно, какой именно процесс выполняет эти специальные действия, главное, что-бы он вообще существовал. В этом

разделе представлены алгоритмы, предназначенные для выбора координатора (это слово будет использоваться в качестве обобщенного имени специального процесса).

Если все процессы абсолютно одинаковы и не имеют отличительных характеристик, способа выбрать один из них не существует. Соответственно, считается, что каждый процесс имеет уникальный номер, например сетевой адрес (для простоты считается, что на каждую машину приходится по процессу).

В общем, алгоритмы голосования пытаются найти процесс с максимальным номером и назначить его координатором. Алгоритмы разрешаются способами поиска.

Также считается, что каждый процесс знает номера всех остальных процессов, но не известно, какие из них в настоящее время работают, а какие нет. Алгоритм голосования должен гарантировать, что если голосование началось, то оно, рассмотрев все процессы, решит, кто станет новым координатором.

2.2.1. Алгоритм «забияки»

Теоретическая часть

Алгоритм «забияки» (bully algorithm). Когда один из процессов замечает, что координатор больше не отвечает на запросы, он инициирует голосование. Например, процесс Р проводит голосование следующим образом:

1. Процесс Р посылает всем процессам с большими, чем у него, номерами сообщение типа ГОЛОСОВАНИЕ;
2. Далее возможно два варианта развития событий:
 - если никто не отвечает, процесс Р выигрывает голосование и становится координатором;

– если один из процессов с большими номерами отвечает, то он становится координатором, а процесс Р прекращает опрос и продолжает работу как обычный процесс.

В любой момент процесс может получить сообщение ГОЛОСОВАНИЕ от одного из своих коллег с меньшим номером. По получении этого сообщения получатель посылает отправителю сообщение ОК, показывая, что он работает и готов стать координатором. Затем получатель сам организует голосование. В конце концов, все процессы, кроме одного, отпадут, этот последний и будет новым координатором. Он уведомит о своей победе посылкой всем процессам сообщения, гласящего, что он новый координатор и приступает к работе.

Если процесс, который находился в нерабочем состоянии, начинает работать, он организует голосование. Если он оказывается процессом с самым большим из работающих процессов номером, он выигрывает голосование и берет на себя функции координатора. Итак, побеждает всегда самый большой парень в городишке, отсюда и название «алгоритм забияки».

На рис. 2.6 приведен пример работы алгоритма забияки. Группа состоит из восьми процессов, пронумерованных от 0 до 7. Ранее координатором был процесс 7, но он завис. Процесс 4 первым замечает это и посылает сообщение ГОЛОСОВАНИЕ всем процессам с номерами больше, чем у него, то есть процессам 5, 6 и 7, как показано на рис. 2.6 а. Процессы 5 и 6 отвечают ОК, как показано на рис. 2.6 б. После получения первого из этих ответов процесс 4 «понимает» что он меньше по приоритету.

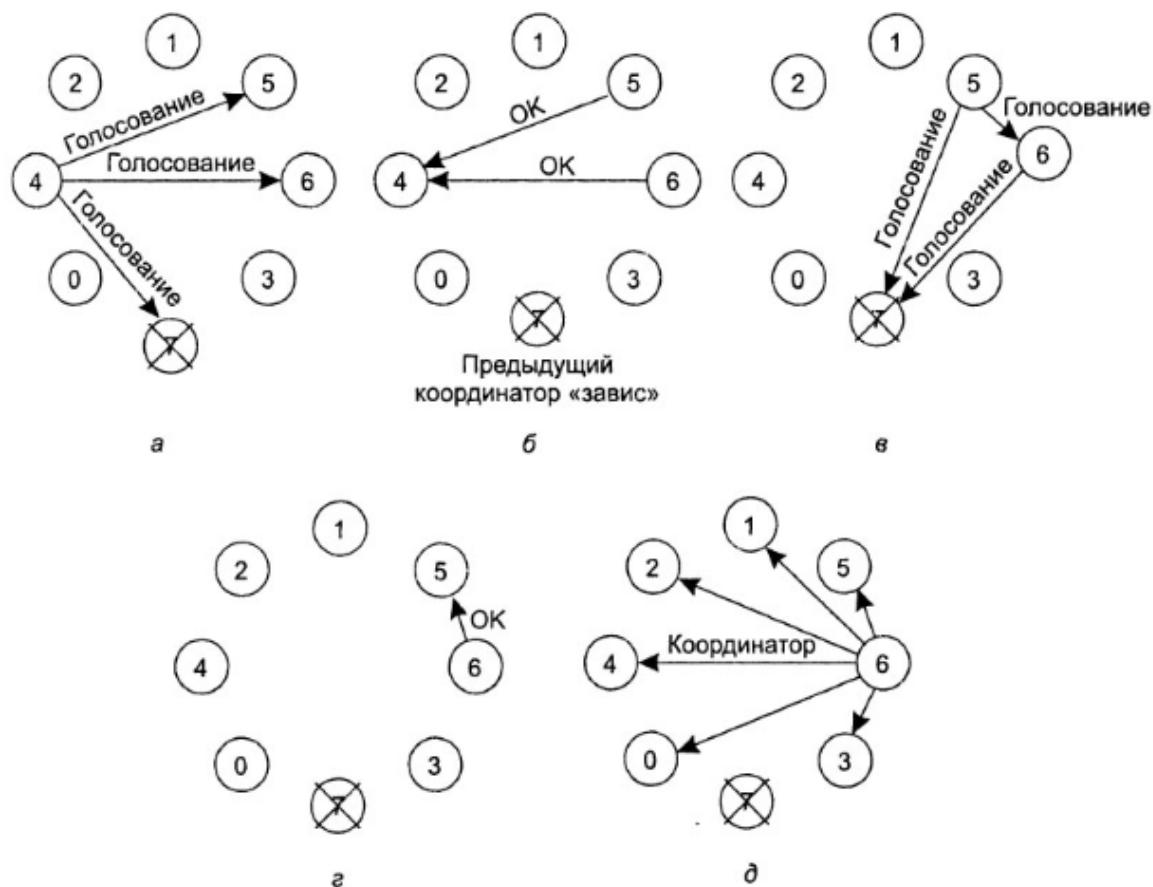


Рис. 2.6. Голосование по алгоритму «Забияки»

На рис. 2.6.в показано, как оба оставшихся процесса, 5 и 6, продолжают голосование. Каждый посылает сообщения только тем процессам, номера у которых больше их собственных. На рис. 2.6 г процесс 6 сообщает процессу 5, что голосование будет вести он. В это время 6 понимает, что процесс 7 не может ответить, а значит, победитель – он сам. Если информация о состоянии сохраняется на диске или где-то еще, откуда ее можно достать, когда с прежним координатором что-нибудь случается, процесс 6 должен записать все что нужно на диск. Готовый занять свою должность процесс 6 заявляет об этом путем рассылки сообщения **КООРДИНАТОР** всем работающим процессам. Когда 4 получит это сообщение, он продолжит работу с той операции, которую пытался выполнить, когда обнаружил, что процесс 7 не отвечает, используя теперь в качестве координатора процесс 6.

Таким образом, можно обойти сбой в процессе 7, и работа продолжается. Если процесс 7 запустится снова, ему будет достаточно послать всем остальным сообщение **КООРДИНАТОР** и вынудить их подчиниться.

Описание лабораторной установки

Интерфейс лабораторной установки представлен на рис. 2.7.

Для изменения состояния процесса и отправки сообщений используется контекстное меню (рис. 2.8).

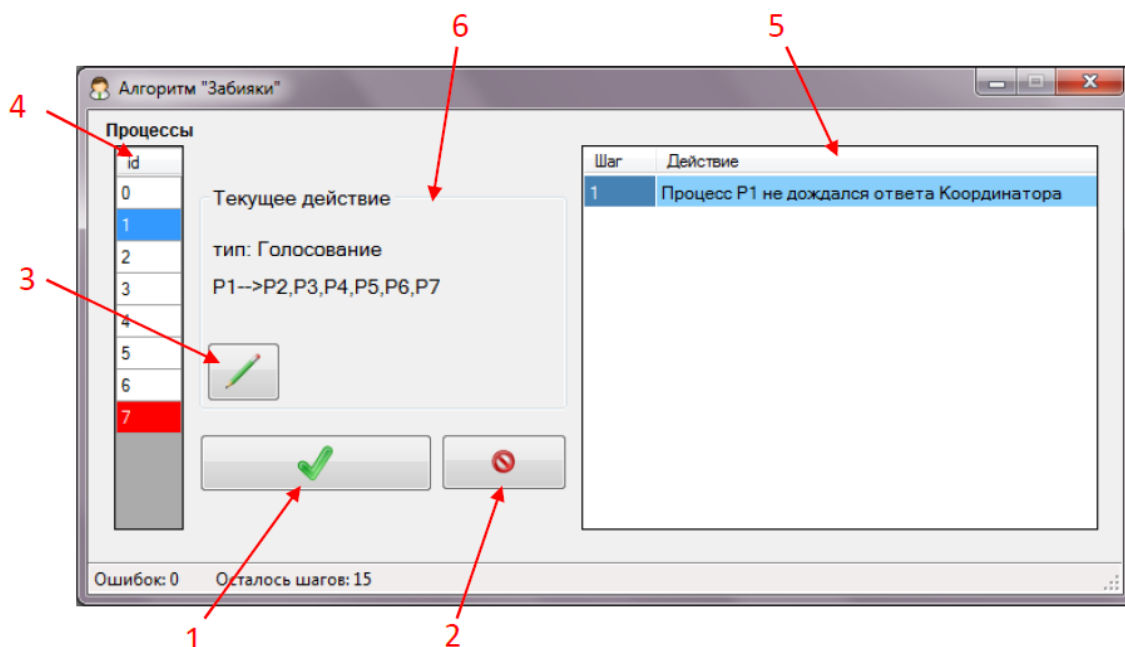


Рис. 2.7. Интерфейс лабораторной установки для алгоритма «забияки»

Обозначения:

1 – кнопка «Выполнить». Проверяется текущий шаг и в случае правильного выполнения осуществляется переход к следующему.

2 – кнопка «Отмена». Позволяет сделать откат изменений на начало текущего шага.

3 – кнопка «Редактирование». Позволяет редактировать сообщение.

4 – список процессов. Рабочие процессы отображены белым цветом, нерабочие – красным, координатор – синим. Отмечать процессы можно

с помощью контекстного меню (кликнуть правой кнопкой по нужному процессу).

5 – история выполнения алгоритма. Текущий шаг помечен голубым цветом. Если шаг сделан правильно, то строка закрашивается зеленым цветом, в противном случае закрашивается красным.

6 – панель «Текущее действие». Показывает какой тип сообщений выбран и каким процессам отправляется сообщение.

Форма для отправки сообщений процесса представлена на рис. 2.9.

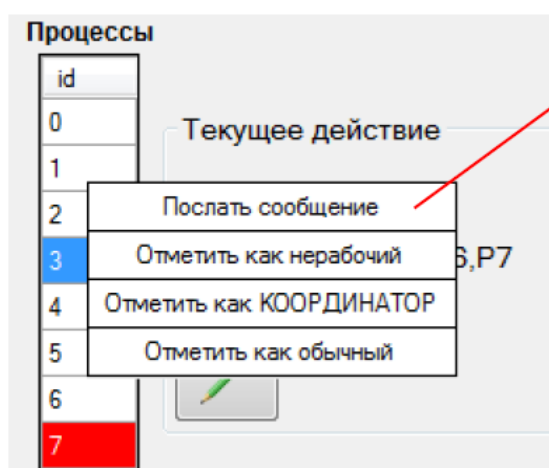


Рис. 2.8. Контекстное меню

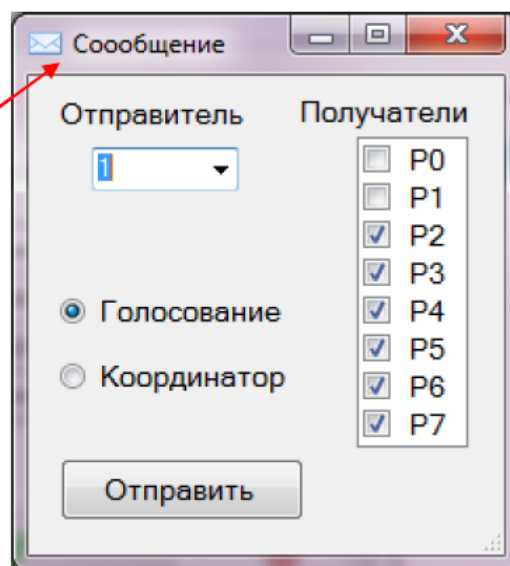


Рис. 2.9. Форма для отправки сообщений

Выполнение лабораторной работы

Выполнение шагов:

1) Если координатор перестает замечать, то его нужно отметить как нерабочий. Отсутствие координатора замечает какой-либо процесс и организует процедуру голосования. Он должен послать сообщение с типом «Голосование» всем процессам с номерами, большими чем у него. Если есть ответившие процессы, то они продолжают голосование аналогично,

если ни один процесс не ответил, то процесс-отправитель сам становится координатором и посылает всем кроме себя(!) сообщение с типом «Координатор».

2) Нерабочие процессы могут возвращаться в рабочее состояние. Вернувшийся процесс «не знает» есть ли в системе координатор и организует голосование, если ему никто не отвечает, то он сам становится координатором. Если уже есть координатор, и его номер больше, чем у вернувшегося процесса, то он отвечает ему «ОК». Однако в лабораторной установке этот шаг опущен и если вернувшийся процесс не самый «большой», то просто показывается следующий шаг для выполнения.

2.2.2. Кольцевой алгоритм

Теоретическая часть

Другой алгоритм голосования основан на использовании кольца. В отличие от некоторых других кольцевых алгоритмов в этом не требуется маркер. Мы предполагаем, что процессы физически или логически упорядочены, так что каждый из процессов знает, кто его преемник. Когда один из процессов обнаруживает, что координатор не функционирует, он строит сообщение ГОЛОСОВАНИЕ, содержащее его номер процесса, и посылает его своему преемнику. Если преемник не работает, отправитель пропускает его и переходит к следующему элементу кольца или к следующему, пока не найдет работающий процесс. На каждом шаге отправитель добавляет свой номер процесса к списку в сообщении, активно продвигая себя в качестве кандидата в координаторы.

В конце концов, сообщение вернется к процессу, который начал голосование. Процесс распознает это событие по приходу сообщения, содержащего его номер процесса. В этот момент тип сообщения изменяется на КООРДИНАТОР и вновь отправляется по кругу, на этот раз

с целью сообщить всем процессам, кто стал координатором (элемент списка с максимальным номером) и какие процессы входят в новое кольцо. После того как это сообщение один раз обойдет все кольцо, оно удаляется и процессы кольца возвращаются к обычной работе.

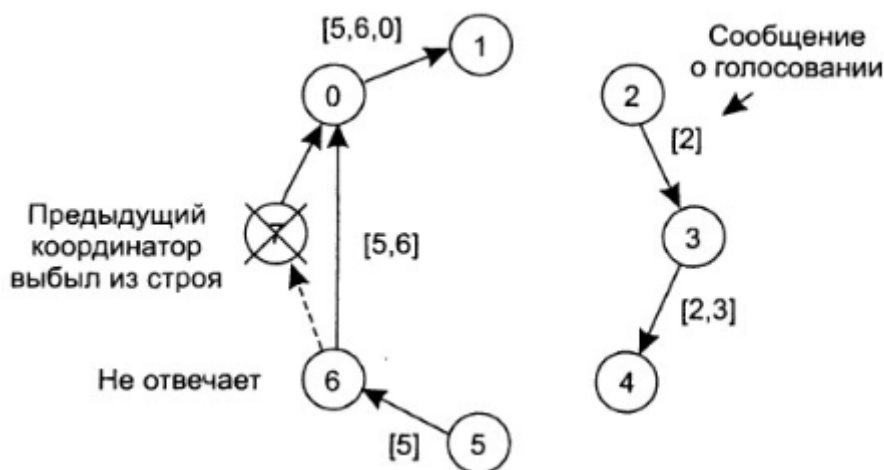


Рис. 2.10. Кольцевой алгоритм голосования

На рис. 2.10 мы видим, что происходит, когда два процесса, 2 и 5, одновременно обнаруживают, что предыдущий координатор, процесс 7, перестал работать. Каждый из них строит сообщение ГОЛОСОВАНИЕ и запускает это сообщение в путь по кольцу независимо от другого. В конце концов, оба сообщения проходят все кольцо, и процессы 2 и 5 превращают их в сообщения КООРДИНАТОР, которые пересылаются тем же элементам кольца и в том же порядке.

Когда эти сообщения, в свою очередь, совершают полный круг, они удаляются. Больше не происходит ничего такого, что потребовало бы дополнительного обмена сообщениями. Даже в наихудшем случае все сделанное очень незначительно загружает сеть.

Описание лабораторной установки

Интерфейс лабораторной установки представлен на рис. 2.11.

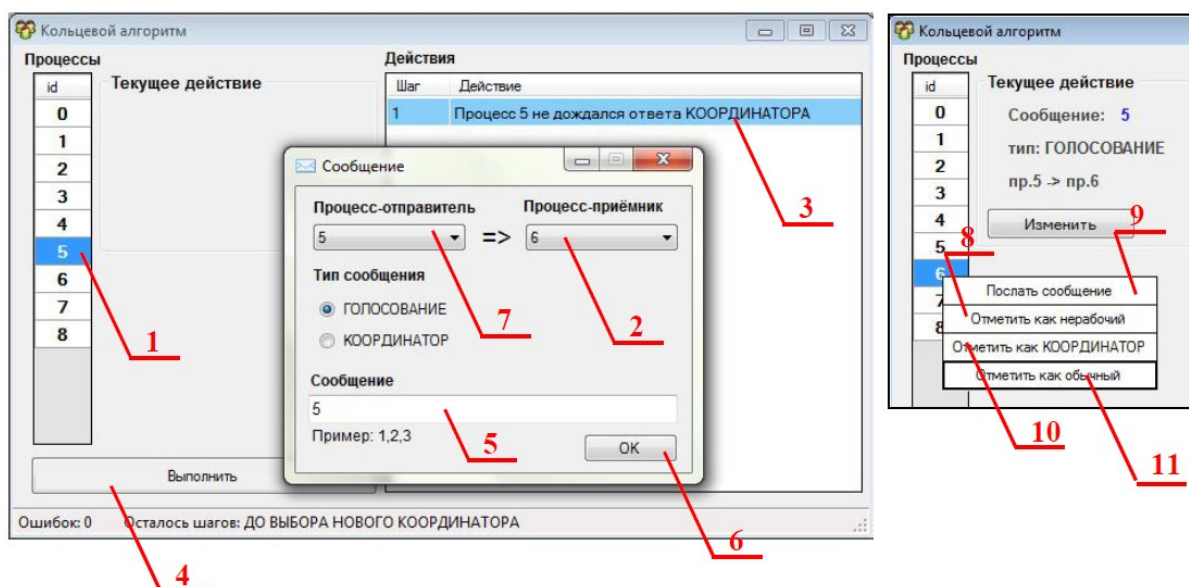


Рис. 2.11. Интерфейс лабораторной установки для кольцевого алгоритма

Обозначения:

Процесс, который начинает опрос, выделяется на панели слева (1). Для Действие, которое необходимо выполнить представлено в таблице (3). Синий цвет обозначает текущий шаг, зелёный – выполненные правильно шаги, красный – шаг выполнен неверно.

После выполнения каждого шага необходимо нажать кнопку «Выполнить» (4).

Выполнение лабораторной работы

Выполнение шагов:

1) Послать сообщение при ГОЛОСОВАНИИ

Когда процесс заметил нерабочего КООРДИНАТОРА, он должен послать сообщение (9) следующему процессу по кольцу. В сообщении необходимо указать процесса-отправителя (7), процесса приемника (следующего по кольцу) (2), указать в сообщении номер текущего процесса (прибавлением через запятую к предыдущему сообщению). Выбрать тип сообщения ГОЛОСОВАНИЕ и подтвердить выбор (6).

2) Процесс не ответил

Необходимо отметить этот процесс как не рабочий (8), и послать сообщение следующему процессу по кольцу.

3) ГОЛОСОВАНИЕ обошло кольцо

Необходимо выбрать нового КООРДИНАТОРА (максимальный рабочий номер) (10), и послать сообщение по новому кольцу (от процесса, с которого началось голосование), чтобы оповестить всех о новом КООРДИНАТОРе. Причем в формате сообщения необходимо указать тип сообщения КООРДИНАТОР с выбранным КООРДИНАТОРОМ.

2.3. Взаимные исключения

Системы, состоящие из множества процессов, обычно проще всего программировать, используя критические области. Когда процесс нуждается в том, чтобы считать или обновить совместно используемые структуры данных, он сначала входит в критическую область, чтобы путем взаимного исключения убедиться, что ни один из процессов не использует одновременно с ним общие структуры данных. В однопроцессорных системах критические области защищаются семафорами, мониторами и другими конструкциями подобного рода. В этом разделе рассматривается несколько примеров того, как именно критические области и взаимные исключения реализуются в распределенных системах.

2.3.1. Централизованный алгоритм

Теоретическая часть

Наиболее простой способ организации взаимных исключений в распределенных системах состоит в том, чтобы использовать методы их реализации, принятые в однопроцессорных системах. Один из процессов выбирается координатором (например, процесс, запущенный на машине с самым большим сетевым адресом).

Каждый раз, когда этот процесс собирается войти в критическую область, он посылает координатору сообщение с запросом, в котором уведомляет, в какую критическую область он собирается войти, и запрашивает разрешение на это. Если ни один из процессов в данный момент не находится в этой критической области, координатор посылает ответ с разрешением на доступ, как показано на рис. 2.12 а. После получения ответа процесс, запросивший доступ, входит в критическую область.

Предположим теперь, что другой процесс, 2, запрашивает разрешение на вход в ту же самую область. Координатор знает, что в этой критической области уже находится другой процесс, и не дает разрешения на вход. Конкретный способ запрещения доступа зависит от особенностей системы. На рис. 2.12 б координатор просто не отвечает, блокируя этим процесс 2, который ожидает ответа. Он также может послать ответ, гласящий «доступ запрещен». В любом случае он ставит запрос от процесса 2 в очередь и ожидает дальнейших сообщений.



Рис. 2.12. Централизованный алгоритм

Когда процесс 1 покидает критическую область, он посылает координатору сообщение, отказываясь от эксклюзивного доступа

на область, как показано на рис. 2.12 в. Координатор выбирает первый элемент из очереди отложенных запросов и посылает процессу разрешающее сообщение. Если процесс был блокирован (то есть для него это первое сообщение от координатора), он разблокируется и входит в критическую область. Если процессу было отослано сообщение с запретом на доступ, он будет опрашивать приходящие сообщения или блоки. В любом случае, увидев разрешение, он войдет в критическую область.

Легко заметить, что алгоритм гарантирует взаимное исключение: координатор позволяет войти в каждую критическую область только одному процессу за раз. Это, помимо всего прочего, честно, поскольку разрешения выдаются в том же порядке, в каком они запрашивались. Никакой процесс никогда не ждет вечно (зависания отсутствуют). Схема также проста в реализации и использует для работы с критической областью всего три сообщения (запросить, разрешить и высвободить). Подобный алгоритм может использоваться и для другого выделения ресурсов, а не только для работы с критическими областями.

Описание лабораторной установки

Интерфейс лабораторной установки представлен на рис. 2.13.

Обозначения:

1 – кнопка «Выполнить». Проверяется текущий шаг и в случае правильного выполнения осуществляется переход к следующему.

2 – кнопка «Отмена». Позволяет сделать откат изменений на начало текущего шага.

3 – добавление процесса в очередь.

4 – выбор процесса для занесения в очередь.

5 – выбор критической секции (КС). Всего в установке три КС.

6 – выбор состояния координатора (разрешение на вход / запрет на вход).

7 – состояния процессов (обычное состояние / в очереди / в критической секции).

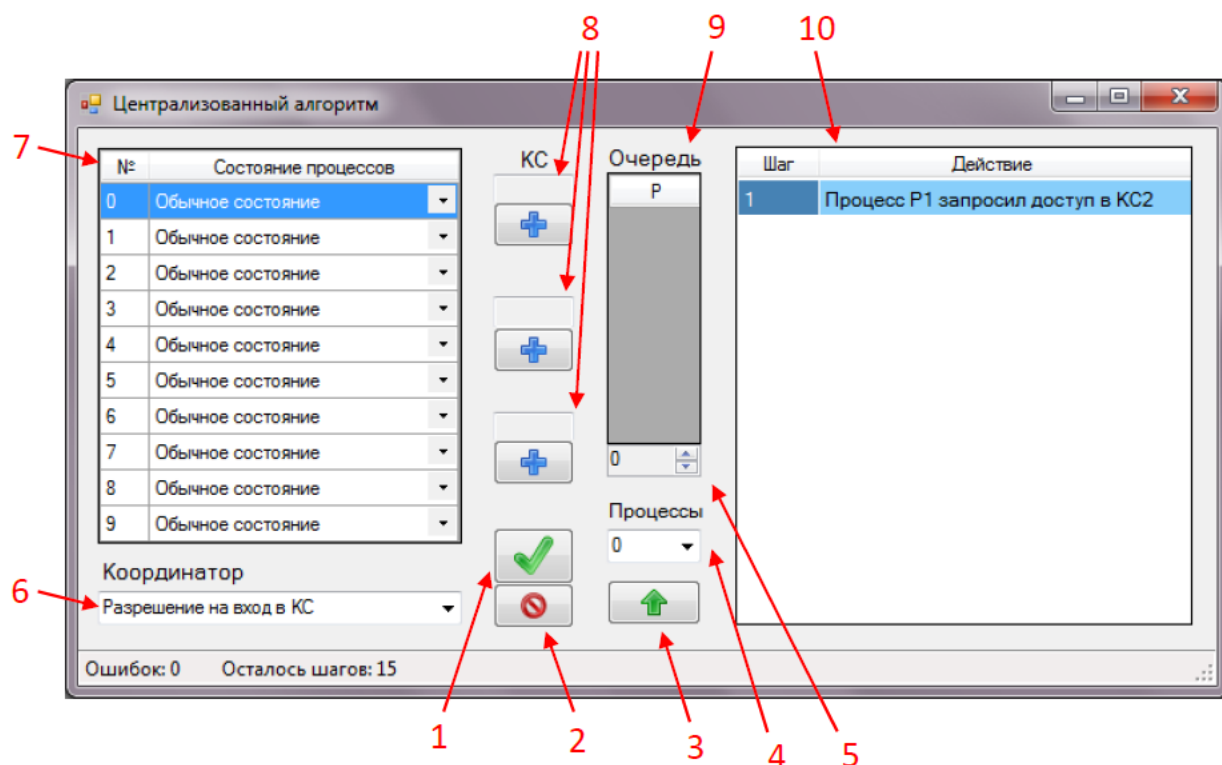


Рис. 2.13. Интерфейс лабораторной установки для централизованного алгоритма

8 – критические секции. Нумерация сверху вниз. С помощью кнопки в КС добавляется процесс из выбранной очереди (обозначение 5).

9 - история выполнения алгоритма. Текущий шаг помечен голубым цветом. Если шаг сделан правильно, то строка закрашивается зеленым цветом, в противном случае закрашивается красным.

Выполнение лабораторной работы

Выполнение шагов:

1) Когда процесс запрашивает вход в КС, сначала его нужно добавить в очередь к нужной КС. После этого, если КС свободна,

то процесс в нее входит, изменив свое состояние. Если КС занята, то процесс остается в очереди и также изменяет свое состояние.

2) Координатор разрешает процессам вход в КС, если она свободна, либо запрещает, если она занята. В случае, когда процесс выходит из КС, он меняет свое состояние, а на его место поступает процесс, находящийся первым в очереди.

2.3.2. Распределенный алгоритм

Теоретическая часть

Наличие даже одного неработающего места в централизованных алгоритмах часто недопустимо, поэтому исследователи обратились к распределенным алгоритмам взаимного исключения.

Рассматриваемый алгоритм требует наличия полной упорядоченности событий в системе. То есть в любой паре событий, например, отправки сообщений, должно быть однозначно известно, какое из них произошло первым. Алгоритм Лампорта, является одним из способов введения подобной упорядоченности и может быть использован для расстановки отметок времени распределенных взаимных исключений.

Алгоритм работает следующим образом. Когда процесс собирается войти в критическую область, он создает сообщение, содержащее имя критической области, свой номер и текущее время. Затем он отправляет это сообщение всем процессам, концептуально включая самого себя. Посылка сообщения, как предполагается, надежная, то есть на каждое письмо приходит подтверждение в получении.

Вместо отдельных сообщений может быть использована доступная надежная групповая связь.

Когда процесс получает сообщение с запросом от другого процесса, действие, которое оно производит, зависит от его связи с той критической

областью, имя которой указано в сообщении. Можно выделить три варианта:

- если получатель не находится в критической области и не собирается туда входить, он отправляет отправителю сообщение ОК;
- если получатель находится в критической области, он не отвечает, а помещает запрос в очередь;
- если получатель собирается войти в критическую область, но еще не сделал этого, он сравнивает метку времени пришедшего сообщения с меткой времени сообщения, которое он отослал. Выигрывает минимальное. Если пришедшее сообщение имеет меньший номер, получатель отвечает посылкой сообщения ОК. Если его собственное сообщение имеет меньшую отметку времени, получатель ставит приходящие сообщения в очередь, ничего не посылая при этом. После посылки сообщения-запроса на доступ в критическую область процесс приостанавливается и ожидает, что кто-нибудь даст ему разрешение на доступ. После того как все разрешения получены, он может войти в критическую область. Когда он покидает критическую область, то отправляет сообщения ОК всем процессам в их очереди и удаляет все сообщения подобного рода из своей очереди.

Если конфликты отсутствуют, все идет хорошо. Однако представим себе, что два процесса пытаются одновременно войти в одну и ту же критическую область, как показано на рис. 2.14 а. Процесс 0 имеет меньшую отметку времени и потому выигрывает (рис. 2.14 б). Когда процесс 0 завершает работы с критической областью, он отправляет сообщение ОК и теперь процесс 2 может войти в критическую область (рис. 2.14 в).

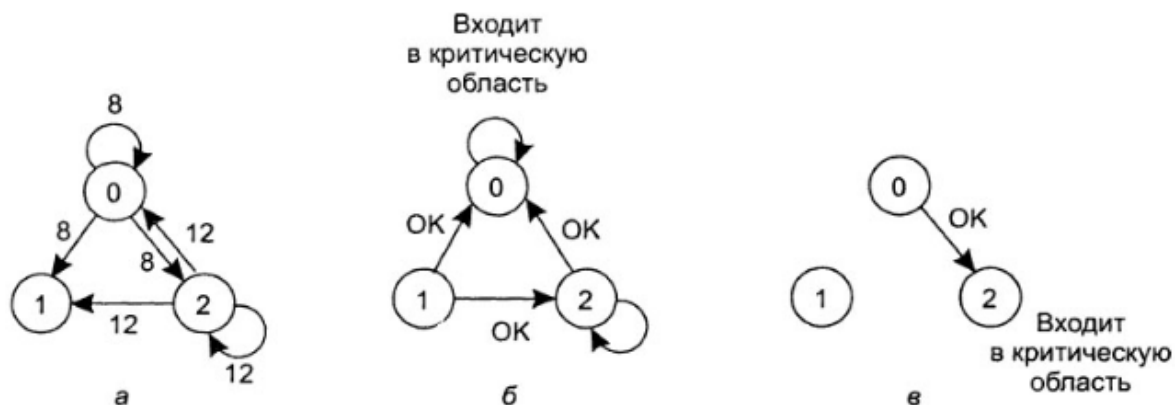


Рис. 2.14. Пример конфликта процессов

Процесс 0 рассылает всем запрос с отметкой времени 8, и одновременно с ним процесс 2 рассылает всем запрос с отметкой времени 12. Процесс 1 не интересуется входом в критическую область и в ответ посылает сообщение ОК им обоим. Процессы 0 и 2 замечают конфликт и сравнивают отметки времени. Процесс 2 видит, что проиграл, и разрешает доступ процессу 0, посылая ему сообщение ОК. Процесс 0 ставит ответ от процесса 2 в очередь для дальнейшей обработки и входит в критическую секцию, как показано на рис. 2.14 б. Когда процесс 0 заканчивает работу в критической области, он удаляет ответ 2 из очереди сообщений и отправляет процессу 2 сообщение ОК, разрешая ему войти в критическую область, что и показано на рис. 2.14 в. Алгоритм работает, поскольку в случае конфликтов наименьшая отметка времени выигрывает, а с очередностью отметок времени способен разобратся любой процесс.

Когда приходит запрос, его получатель посылает ответ всегда, разрешая или запрещая доступ. Всякий раз, когда запрос или ответ утеряны, отправитель выжидает положенное время и либо получает ответ, либо считает, что получатель находится в нерабочем состоянии. После получения запрещения отправитель ожидает последующего сообщения ОК.

Алгоритм можно модифицировать так, чтобы разрешить процессу вход в критическую область после того, как он соберет разрешения простого большинства, а не всех остальных процессов. Разумеется, в этом случае, после того как процесс даст разрешение на вход в критическую

область одному из процессов, он не сможет дать то же самое разрешение другому процессу до тех пор, пока первый не покинет критическую область.

Несмотря на все возможные улучшения, этот алгоритм остается более медленным, более сложным, более затратным и менее устойчивым, чем исходный централизованный алгоритм.

Описание лабораторной установки

Интерфейс лабораторной установки представлен на рис. 2.15.

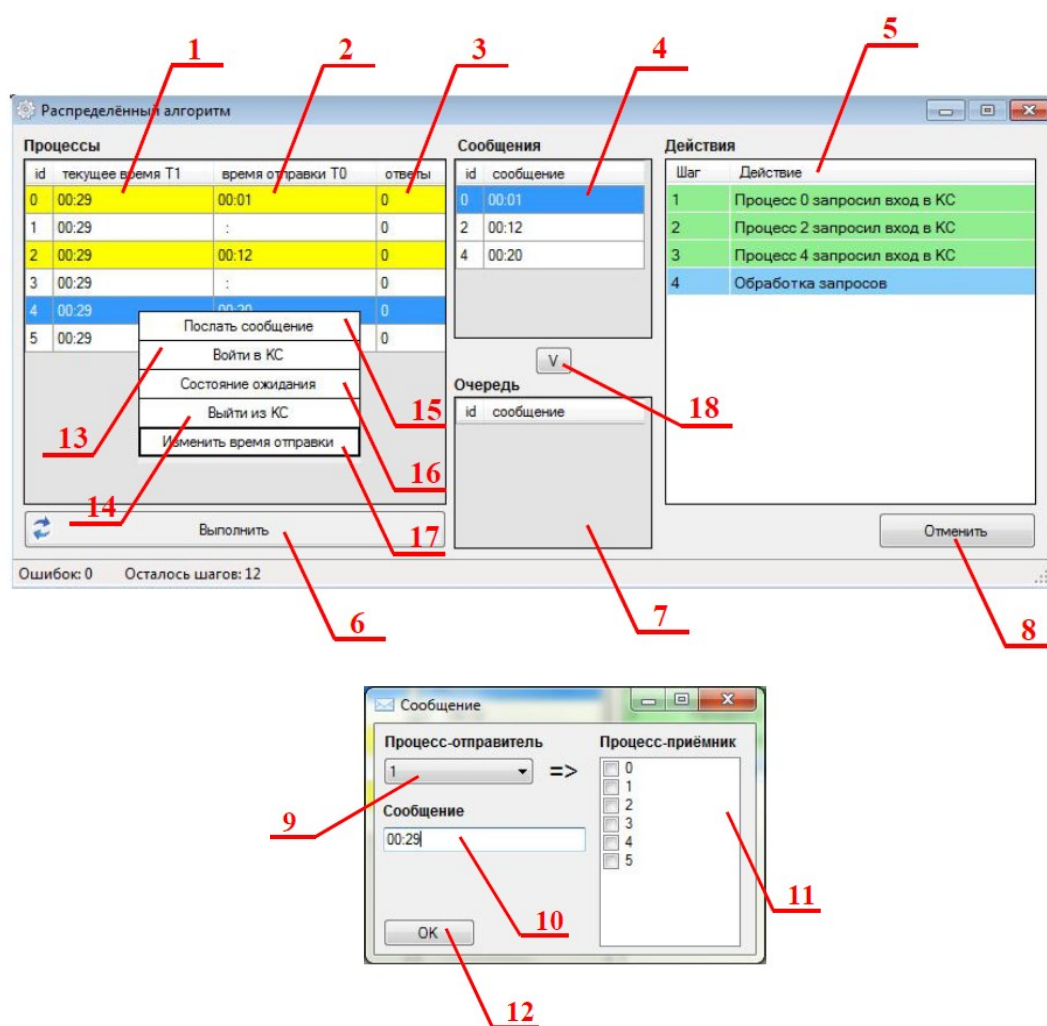


Рис. 2.15. Интерфейс лабораторной установки для распределенного алгоритма

Обозначения:

Действие, которое необходимо выполнить представлено в таблице (5). Синий цвет обозначает текущий шаг, зелёный – выполненные правильно шаги, красный – шаг выполнен неверно.

После выполнения каждого шага необходимо нажать кнопку «Выполнить» (6).

Для того, чтобы возратить значения времени на начало шага, необходимо нажать кнопку «Отмена шага» (5).

Выполнение лабораторной работы

Выполнение шагов:

1) Запрос процессом КС

Чтобы запросить КС, процесс должен отправить сообщения остальным клиентам, включая самого себя. Для этого необходимо щёлкнуть правой кнопкой мыши на поле (1) и выбрать пункт «Послать сообщение» (15). В окне сообщения необходимо выбрать процесс-отправитель (9), выделить процессы, которым необходимо отправить сообщение (11). После ввода сообщения (10) (текущее время процесса) необходимо подтвердить ввод (12). Далее необходимо сохранить время запроса (17) и ввести процесс в состояние ожидания (16).

2) Обработка запросов.

При обработке запросов необходимо ответить на все поступившие сообщения в очередях всех процессов:

- Если процесс запрашивал КС и его время запроса больше, чем в сообщении в очереди (4), он может ответить ОК (для этого необходимо щёлкнуть правой кнопкой мыши на сообщении в очереди). Если время запроса меньше, чем в сообщении в очереди сообщений (4), он добавляет сообщение в свою очередь (7), кнопкой (18).

- Если процесс не запрашивал вход в КС, он может ответить на все сообщения в очереди.

После обработки сообщений, у одного из процессов будут собраны все ответы (3) . Он может войти в КС (13).

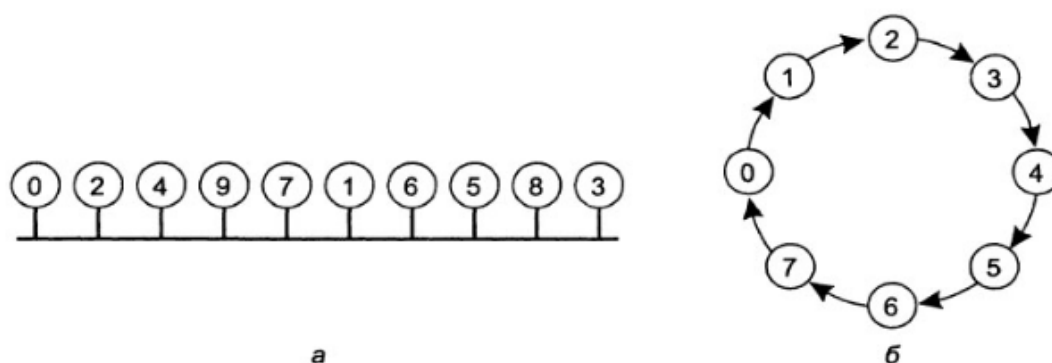
3) Выход из КС.

Когда процесс выходит из КС, необходимо обозначить его как обычный процесс (14), изменить время отправки (17), просто удалив значение и отправить ОК всем процессам из его очереди (7). В этом же шаге выполняется обработка запросов и если есть процесс, у которого будут все ответы, он входит в КС.

2.3.3. Алгоритм маркерного кольца

Теоретическая часть

Абсолютно иной подход к реализации взаимных исключений в распределенных системах предлагает алгоритм маркерного кольца (Token Ring). Пусть мы имеем магистральную сеть (например, Ethernet), но без внутреннего упорядочения процессов (рис. 2.16 а). Программно создается логическое кольцо, в котором каждому процессу назначается положение в кольце, как показано на рис. 2.16 б. Положение в кольце может быть назначено по порядку следования сетевых адресов или как-то иначе. Неважно, как именно задана упорядоченность. Главное – как дать процессу знать, кто в кольце является следующим после него.



Беспорядочная группа процессов в сети (а).
Логическое кольцо, созданное программно (б)

Рис. 2.16. Пример маркерного кольца

При инициализации кольца процесс 0 получает маркер, или токен (token). Маркер циркулирует по кольцу. Он передается от процесса

к процессу $k+1$ (это модуль размера кольца) сквозными сообщениями. Когда процесс получает маркер от своего соседа, он проверяет, не нужно ли ему войти в критическую область. Если это так, он входит в критическую область, выполняет там всю необходимую работу и покидает область. После выхода он передает маркер дальше. Входить в другую критическую область, используя тот же самый маркер, запрещено.

Если процесс, получив от соседа маркер, не заинтересован во входе в критическую область, он просто передает этот маркер дальше. Соответственно, если ни один из процессов не находится в критических областях, маркер просто циркулирует по кольцу с максимально возможной скоростью.

Легко видеть, что этот алгоритм корректен. Только один процесс в любой момент времени обладает маркером, а значит, только один процесс может находиться в критической области. Поскольку маркер перемещается от процесса к процессу в общеизвестном порядке, зависания не происходит. Когда процесс решает войти в критическую область, в худшем случае ему придется ждать, пока все остальные процессы последовательно не войдут в критическую область и не выйдут из нее.

Обычно этот алгоритм также имеет проблемы. Если маркер однажды потеряется, он должен быть восстановлен. На самом деле, понять, что он пропал, довольно сложно, поскольку срок между последовательными появлениями маркера в сети не ограничен. Тот факт, что маркера не было видно в течение часа, вовсе не означает, что он потерян, кто-нибудь просто может его использовать.

Алгоритм также сталкивается с проблемами при сбоях процессов, однако справиться с этим проще, чем в других случаях. Если мы потребуем от процесса, получающего маркер, подтверждать получение, неработающий процесс будет обнаружен при первой же попытке соседа передать ему маркер. В этот момент неработающий процесс можно

удалить из группы, и хранитель маркера сможет перебросить маркер через его «голову» следующему члену кольца или, при необходимости следующему за ним. Разумеется, это требует поддержания текущей конфигурации кольца.

Описание лабораторной установки

Интерфейс лабораторной установки представлен на рис. 2.17.

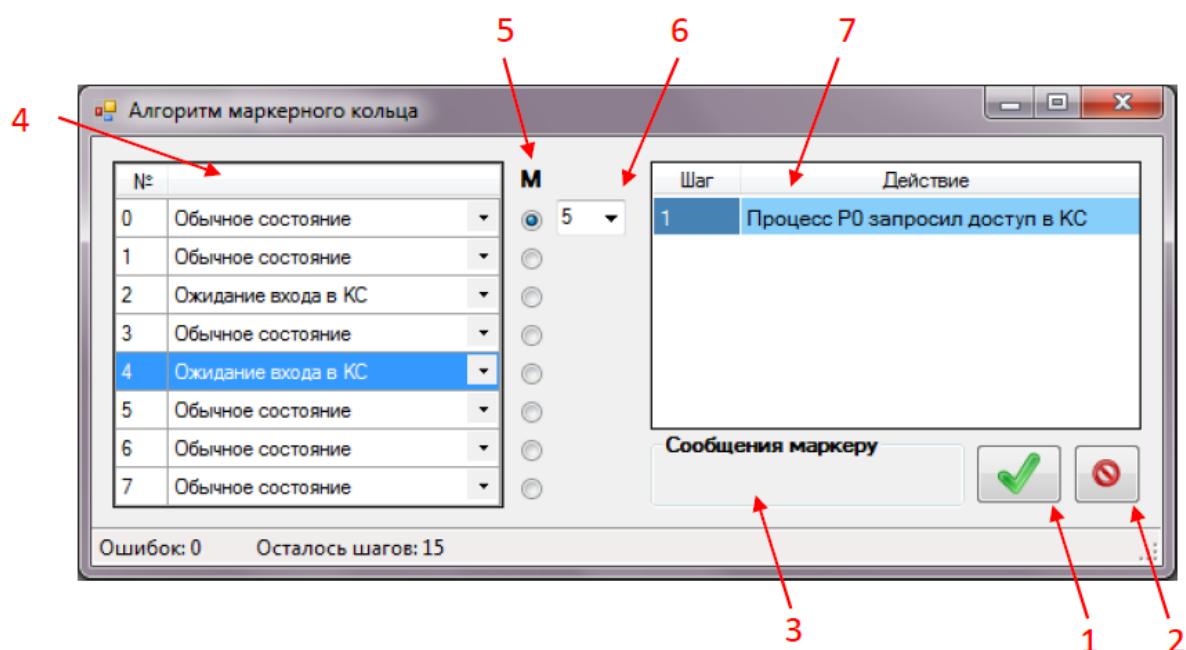


Рис. 2.17. Интерфейс лабораторной установки для алгоритма маркерного кольца

Обозначения:

1 – кнопка «Выполнить». Проверяется текущий шаг и в случае правильного выполнения осуществляется переход к следующему.

2 – кнопка «Отмена». Позволяет сделать откат изменений на начало текущего шага.

3 – Панель для отображения, ответил ли процесс маркеру.

4 – состояния процессов (Обычное состояние / Ожидание входа в КС / В критической секции).

5 – маркер.

6 – сообщение, которое маркер посылает процессу.

7 – история выполнения алгоритма. Текущий шаг помечен голубым цветом. Если шаг сделан правильно, то строка закрашивается зеленым цветом, в противном случае закрашивается красным.

Выполнение лабораторной работы

Выполнение шагов:

В данном алгоритме процесс может зайти в КС только в случае, если у него сейчас есть маркер.

На каждом шаге маркер посылает сообщение следующему процессу. Если процесс рабочий, то он отвечает маркеру «ОК» и на следующем шаге маркер может на него перейти. Если процесс не отвечает, то маркер остается в том же процессе и посылает сообщение следующему процессу.

Нерабочий процесс помечается красным цветом. Для этого нужно щелкнуть один раз по номеру процесса в таблице состояний процесса.

Возможна ситуация, что маркер отослал сообщение процессу, процесс ответил «ОК» и в это же время процесс, в котором находится маркер, запрашивает вход в КС. В этом случае маркер перемещается в процесс, который ответил «ОК», а предыдущий процесс помечается как ожидающий КС.

2.4. Распределенные транзакции

Концепция транзакций тесно связана с концепцией взаимных исключений. Алгоритмы взаимного исключения обеспечивают одновременный доступ не более чем одного процесса к совместно используемым ресурсам, таким как файл, принтер и т. п. Транзакции, в общем, также защищают общие ресурсы от одновременного доступа нескольких параллельных процессов. Однако транзакции могут и многое другое. В частности, они превращают процессы доступа и модификации множества элементов данных в одну атомарную операцию. Если процесс

во время транзакции решает остановиться на полпути и повернуть назад, все данные восстанавливаются с теми значениями и в том состоянии, в котором они были до начала транзакции.

Компьютерная модель транзакций выглядит так. Один процесс объявляет, что хочет начать транзакцию с одним или несколькими другими процессами. После этого они могут согласовывать различные условия, создавать и удалять сущности, выполнять операции. Затем инициатор объявляет, что он предлагает всем остальным подтвердить, что работа сделана. Если все это подтверждают, результаты утверждаются и становятся постоянными. Если один или несколько процессов отказываются (или до соглашения в них возникают сбои), ситуация возвращается к тому состоянию, которое имело место до начала транзакции, со всеми вытекающими отсюда последствиями для файлов, баз данных и т. д., все изменения в которых волшебным образом исчезают. Такое свойство «все или ничего» сильно упрощает работу программистам.

Программирование с использованием транзакций требует специальных примитивов, которые могут поддерживаться как базовой распределенной системой, так и исполняющей системой языка программирования. Типичные примеры приводятся в таблице. Полный список примитивов зависит от того, какие объекты используются в транзакции.

Таблица

Набор примитивов для выполнения транзакции

BEGIN_TRANSACTION	Пометить начало транзакции
END_TRANSACTION	Прекратить транзакцию и попытаться завершить ее
ABORT_TRANSACTION	Прервать транзакцию и восстановить прежние значения
READ	Считать данные из файла, таблицы или другого источника
WRITE	Записать данные в файл, таблицу или другой приемник

Свойство транзакций «все или ничего» – это лишь одно из характерных свойств транзакции. Говоря более конкретно, транзакции:

- атомарны (atomic) – для окружающего мира транзакция неделима;
- непротиворечивы (consistent) – транзакция не нарушает инвариантов системы;
- изолированы (isolated) – одновременно происходящие транзакции не влияют друг на друга;
- долговечны (durable) – после завершения транзакции внесенные ею изменения становятся постоянными.

На эти свойства часто ссылаются по их первым буквам – ACID.

2.4.1. Закрытое рабочее пространство

Теоретическая часть

В соответствии с первым подходом, когда процесс начинает транзакцию, то он работает в индивидуальном рабочем пространстве, содержащем все файлы и другие объекты, к которым он имеет доступ. Пока транзакция не зафиксирована или не прервется, все изменения данных происходят в этом рабочем пространстве, а не в «реальном», под которым мы понимаем обычную файловую систему. Главная проблема этого подхода состоит в больших накладных расходах по копированию большого объема данных в индивидуальное рабочее пространство, хотя и имеются несколько приемов уменьшения этих расходов.

Когда файл открывается на запись, он может быть найден таким же образом, как и при чтении, за исключением того, что сначала он будет скопирован в закрытое рабочее пространство. Однако при втором варианте оптимизации удастся отказаться и от большей части копирования. Вместо

копирования целого файла в закрытое рабочее пространство можно копировать только индекс файла.

Индекс – это блок данных, ассоциированных с каждым файлом и хранящий информацию о том, где на диске расположены его блоки. В UNIX индекс – это индексный узел (inode). При использовании внутреннего индекса файл может быть считан точно так же, поскольку диск, к которому происходит адресация, – это тот же самый диск с исходным расположением блоков. Однако когда блок файла модифицируется, создается копия этого блока, и в индекс вставляется адрес копии, как показано на рис. 2.19 а. После этого блок можно изменять, не оказывая влияния на оригинал. Добавляемые блоки обрабатываются точно так же. Новые блоки иногда называют теневыми блоками (shadow block).

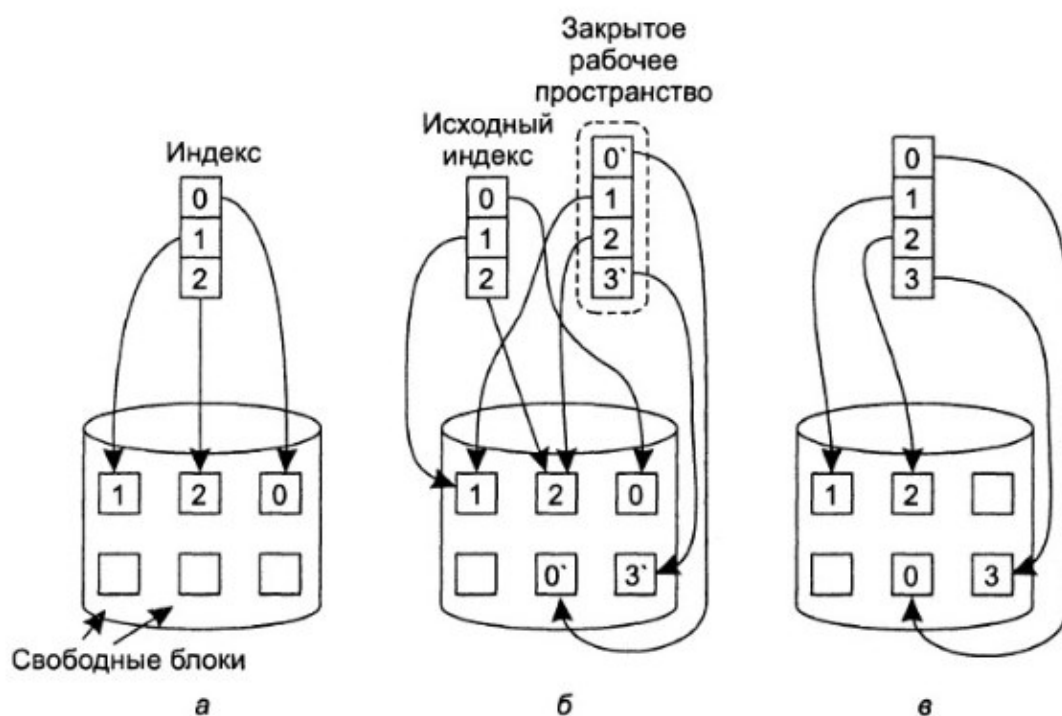


Рис. 2.19. Транзакция в закрытом рабочем пространстве

На рисунке 2.19 показано: индекс файла и дисковые блоки для файла из трех блоков (а); ситуация после того, как транзакция модифицировала

блок 0 и добавили блок 3 (б); ситуация после подтверждения транзакции (в).

Как можно видеть на рис. 2.19 б, процесс, выполняющий транзакцию, видит модифицированный файл, а все остальные процессы продолжают работать с исходным файлом. В особенно сложных транзакциях закрытое рабочее пространство может содержать не один, а большое количество файлов. Если транзакция прерывается, закрытое рабочее пространство просто удаляется и все закрытые блоки, которые оно содержало, возвращаются в список свободных. Если транзакция завершается, закрытые блоки в ходе атомарной операции перемещаются в рабочее пространство родителя, как показано на рис. 2.19 в. Блоки, которые больше не нужны, возвращаются в список свободных.

Эта схема работает также и для распределенных транзакций. В этом случае аналогичный процесс начинается на каждой машине, содержащей файлы, к которым в ходе транзакции будет производиться доступ. Каждый процесс получает свое собственное закрытое рабочее пространство так, как описано выше. Если транзакция прерывается, все процессы просто отказываются от своих закрытых рабочих пространств. С другой стороны, при подтверждении транзакции производятся локальные обновления, после чего транзакции можно считать полностью завершенными.

2.4.2. Журнал с упреждающей записью

Теоретическая часть

Другой традиционный метод реализации транзакций – это журнал с упреждающей записью (write-ahead log). Этот метод заключается в том, что модифицируются сами файлы, а не их копии, но перед изменением любого блока производится запись в специальный файл – журнал регистрации, где отмечается, какая транзакция делает изменения, какой

файл и блок изменяется и каковы старое и новое значения изменяемого блока. Только после успешной записи в журнал регистрации делаются изменения в исходном файле. Если транзакция фиксируется, то и об этом делается запись в журнал регистрации, но старые значения измененных данных сохраняются. Если транзакция прерывается, то информация журнала регистрации используется для приведения файла в исходное состояние, и это действие называется откатом.

В распределенных системах фиксация транзакций может потребовать взаимодействия нескольких процессов на разных машинах, каждая из которых хранит некоторые переменные, файлы, базы данных. Для достижения свойства неделимости транзакций в распределенных системах используется специальный протокол, называемый протоколом двухфазной фиксации транзакций. Хотя он и не является единственным протоколом такого рода, но он наиболее широко используется.

Суть этого протокола состоит в следующем. Один из процессов выполняет функции координатора (рис. 2.20).

Координатор начинает транзакцию, делая запись об этом в своем журнале регистрации, затем он посылает всем подчиненным процессам, также выполняющим эту транзакцию, сообщение «подготовиться к фиксации». Когда подчиненные процессы получают это сообщение, то они проверяют, готовы ли они к фиксации, делают запись в своем журнале и посылают координатору сообщение-ответ «готов к фиксации». После этого подчиненные процессы остаются в состоянии готовности и ждут от координатора команду фиксации. Если хотя бы один из подчиненных процессов не откликнулся, то координатор откатывает подчиненные транзакции, включая и те, которые подготовились к фиксации.



Рис. 2.20. Двухфазный протокол фиксации транзакции

Выполнение второй фазы заключается в том, что координатор посылает команду «фиксировать» (commit) всем подчиненным процессам. Выполняя эту команду, последние фиксируют изменения и завершают подчиненные транзакции. В результате гарантируется одновременное синхронное завершение (удачное или неудачное) распределенной транзакции.

Описание лабораторной установки

Интерфейс лабораторной установки представлен на рис. 2.21.

Обозначения:

Действие, которое необходимо выполнить представлено в таблице (5). Синий цвет обозначает текущий шаг, зелёный – выполненные правильно шаги, красный – шаг выполнен неверно.

После выполнения каждого шага необходимо нажать кнопку «Выполнить» (11).

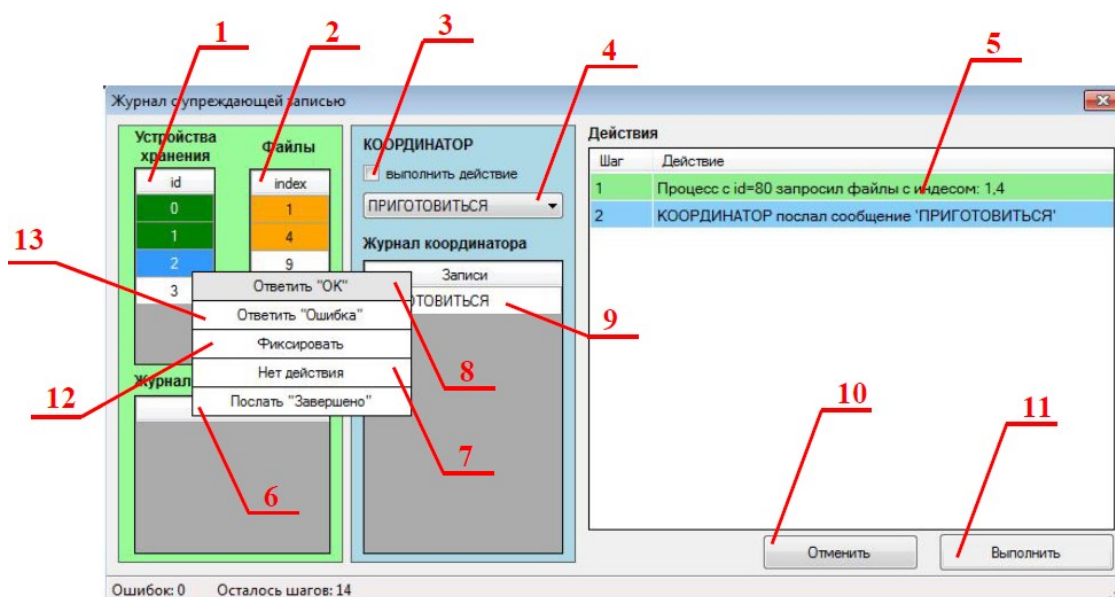


Рис. 2.21. Интерфейс лабораторной установки для журнала с предупреждающей записью

Для того, чтобы вернуть значения времени на начало шага, необходимо нажать кнопку «Отмена шага» (10).

Для очистки полей устройств хранения, необходимо нажать кнопку «Нет действия» (7).

Выполнение лабораторной работы

Выполнение шагов:

1) Запрос процессом файлов с определёнными индексами

Когда процесс запросил изменение файлов с определёнными индексами, КОординАТОР должен запросить (3) готовность этих файлов, которые могут храниться на различных устройствах. Для этого необходимо послать команду ПРИГОТОВИТЬСЯ (4), она автоматически запишется в журнал (9).

2) Ответ от устройств хранения

После того, как послана команда ПРИГОТОВИТЬСЯ, файлы, с запрошенными индексами проверяются на ошибки, и если они доступны

для изменения, устройство отвечает ОК. Необходимо для каждого устройства хранения (1) выбрать файлы (2) с запрошенными индексами и ответить ОК КООРДИНАТОРУ (8), если файлы присутствуют и нет ошибки (выделенный файл не помечен красным цветом), иначе послать сигнал ошибки (13).

3) Все устройства ответили ОК

В этом случае КООРДИНАТОР разрешает процессу изменять файлы, посылая команду ФИКСИРОВАТЬ (4).

4) Какое-либо устройство ответило сигналом об ошибке

В этом случае КООРДИНАТОР должен сделать ОТКАТ, посылая всем соответствующую команду (4). На этом процесс обработки запроса прерывается.

5) Фиксирование изменений

После команды ФИКСИРОВАТЬ, каждое устройство, содержащее файлы с запрошенными индексами фиксирует изменения, которые вносит процесс. Для этого необходимо отметить соответствующие устройства хранения командой ФИКСИРОВАТЬ (12).

6) Завершение работы

После того, как изменения завершены, каждое устройство хранения посылает сообщение о том, что оно завершило работу (6).

7) Завершение транзакции

После того, как каждое устройство послало сообщение о завершении работы, КООРДИНАТОР должен завершить транзакцию соответствующей командой (4).

Результат выполнения лабораторной работы

По окончании лабораторной работы автоматически формируется отчет, в котором показано количество выполненных шагов, количество ошибок, процент выполнения и время выполнения каждого задания. Здесь

подведено суммарное время выполнения лабораторной работы. По полученному отчету преподаватель может судить о качестве выполнения каждого задания студентом. Вид формируемого отчета представлен на рис. 2.22.

Результаты					
№	Название задания	Шагов	Ошибок	Выполнено (%)	Время выполнения
1	Алгоритм Кристиана	15	54	21%	00:33
2	Логические часы	16	12	57%	00:06
3	Алгоритм забияки	20	11	64%	00:06
4	Кольцевой алгоритм	11	11	50%	00:08
5	Централизованный алгоритм	15	17	48%	00:09
6	Распределённый алгоритм	15	17	46%	00:27
7	Алгоритм маркерного кольца	15	15	50%	00:12
8	Журнал супреждающей записью	15	4	78%	00:07
Суммарное время выполнения: 01:48					

Рис. 2.22. Результат выполнения лабораторной работы

Заключение

Многомашинные и многопроцессорные системы в настоящее время являются основными решателями большинства современных задач с использованием вычислительной техники. Знание процессов, происходящих внутри современных компьютеров, в первую очередь изучение алгоритмов работы разнообразных модулей операционных систем, несомненно, поможет профессионалам-программистам создавать программные продукты, эффективные, надежные, отвечающие современным требованиям и способные решать задачи любой сложности.

Библиографический список

1. Воеводин, В. В. Вычислительная математика и структура алгоритмов [Текст] : учебник / В. В. Воеводин ; МГУ. – 2-е изд., стер. – Москва : Изд-во МГУ, 2010. – 166 с.
2. Гергель, В. П. Высокопроизводительные вычисления для многопроцессорных многоядерных систем [Текст] : учеб. пособие / В. П. Гергель ; ННГУ. – Москва : Изд-во МГУ : Физматлит : Изд-во ННГУ, 2010. – 539 с.
3. Илюшечкин, В. М. Операционные системы [Текст] : учеб. пособие / В. М. Илюшечкин. – Москва : Бином. Лаборатория знаний, 2009. – 109 с.
4. Мельцов, В. Ю. Основы проектирования параллельных алгоритмов и программ [Электронный ресурс] : учеб. пособие / В. Ю. Мельцов, В. С. Князьков ; ВятГУ, науч.-образоват. центр супервычислит. технологий и систем. – Киров, 2010.
5. Олифер, В. Г. Сетевые операционные системы [Текст] : учебник / В. Г. Олифер, Н. А. Олифер. – Санкт-Петербург, Москва, Харьков : Питер, 2006. – 539 с.
6. Стивенс У. UNIX: взаимодействие процессов [Текст] / У. Стивенс. – Санкт-Петербург : Питер, 2005. – 576 с.
7. Эндрюс, Г. Р. Основы многопоточного, параллельного и распределенного программирования [Текст] / Г. Р. Эндрюс. – Москва : Вильямс, 2003. – 512 с.

Электронные ресурсы

8. Организация ЭВМ и систем. Дистанционное образование [Электронный ресурс]. – Режим доступа : http://paralichka85.px6.ru/1architecture/glava01_3.htm. – 05.12.2014.
9. Гергель В. П. Теория и практика параллельных вычислений [Электронный ресурс] / В. П. Гергель. – Режим доступа : <http://www.intuit.ru/departament/calculate/paralltp>. – 05.12.2014.

Учебное издание

Караваева Ольга Владимировна

Чистяков Геннадий Андреевич

**ПЛАНИРОВАНИЕ И СИНХРОНИЗАЦИЯ
ПРОЦЕССОВ В МНОГОПРОЦЕССОРНЫХ
ОПЕРАЦИОННЫХ СИСТЕМАХ**

Учебно-методическое пособие

Подписано к использованию 05.12.2014. Заказ № 2362.

Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Вятский государственный университет»

610000, г. Киров, ул. Московская, 36, тел.: (8332) 64-23-56, <http://vyatsu.ru>